

Sheffield
Hallam
University

Deep Neural Networks and Learning Systems

Deep Neural Networks and Learning Systems

Lecture Notes

MSc in Artificial Intelligence
Sheffield Hallam University

Dr. Alejandro Jimenez Rodriguez
Dr. Jamie Duell

Contents

1	The running case study: predicting what happens next	1
1.1	The task	1
1.2	The central trap: an average can win	2
1.3	The full system, seen from a distance	3
1.4	How to read the architecture boxes	4
2	Learning from examples	5
2.1	Several traditions, one modern subject	5
2.1.1	Statistics: fitting a rule to observations	5
2.1.2	Computation and artificial intelligence: what can a machine do? . . .	6
2.1.3	Neuroscience: inspiration without identity	6
2.1.4	From hand-designed features to learned representations	6
2.2	Examples are not the world	7
2.2.1	Sampling variability	8
2.2.2	Training, validation and test data	8
2.3	What exactly is the machine being asked to do?	9
2.3.1	Classification	9
2.3.2	Regression	9
2.3.3	Density estimation and generative modelling	9
2.3.4	Structured prediction	10
2.3.5	Ranking and retrieval	10
2.4	Ways of obtaining a teaching signal	11
2.4.1	Supervised learning	11
2.4.2	Unsupervised learning	11
2.4.3	Self-supervised learning	11
2.4.4	Semi-supervised learning	11
2.4.5	Reinforcement learning: a different feedback structure	12
2.5	The supervised-learning skeleton	12
2.5.1	1. A dataset	12
2.5.2	2. A parameterised model	12
2.5.3	3. A loss	13
2.5.4	4. Training	13
2.5.5	5. Inference	13
2.5.6	6. Evaluation	13
2.6	Learning is not merely fitting	14
2.6.1	Empirical risk and the unseen process	15
2.6.2	The distribution matters	15
2.7	The loss is part of the problem statement	15
2.8	Better than what? Baselines and floors	16
2.8.1	A baseline can diagnose the metric	16

2.9	A task is a contract	17
2.9.1	An example: sentiment analysis	18
2.9.2	An example: image generation	18
2.10	Where neural networks enter	18
2.11	Connections to the Week 1 practical work	19
2.12	Optional exercises	19
2.13	Chapter summary	20
3	Regression, loss and gradient descent	22
3.1	The simplest adjustable prediction	22
3.1.1	Predictions are not targets	23
3.2	From a line to a linear layer	24
3.2.1	Several outputs at once	24
3.2.2	A batch is one larger matrix multiplication	25
3.3	The loss is a specification of error	25
3.3.1	What if the model is forced to predict a constant?	25
3.3.2	Training objective and evaluation metric need not be identical	26
3.4	A derivative is local sensitivity	27
3.4.1	Finite differences: a slope without symbolic calculus	27
3.4.2	Many parameters give a gradient	27
3.5	Gradient descent: turning sensitivity into an update	28
3.5.1	Why training is iterative	29
3.5.2	The gradient of MSE for a line	29
3.6	The practical meaning of a training loop	30
3.7	Why not use the entire dataset for every update?	30
3.7.1	Stochastic and mini-batch gradients	31
3.7.2	Batch, update and epoch are different counters	31
3.7.3	Why shuffle?	32
3.8	When decreasing loss is not enough	32
3.8.1	Baselines return	33
3.9	What stochasticity changes — and what it does not	33
3.10	A complete linear-learning system	33
3.11	What to carry into Week 3	34
4	From linear rules to neural networks	36
4.1	From a linear model to an artificial neuron	36
4.1.1	A score before a class	37
4.2	Classification is geometry	37
4.2.1	Optimisation failure or representational failure?	37
4.3	Why more linear layers do not solve the problem	38
4.4	The nonlinearity	39
4.4.1	Step activation	39
4.4.2	Sigmoid	40
4.4.3	Hyperbolic tangent	40
4.4.4	ReLU	40
4.5	From neurons to a multilayer perceptron	41
4.5.1	A hidden layer is a change of coordinates	41
4.5.2	Depth as repeated re-representation	42
4.6	ReLU networks divide space into regions	43
4.7	The forward pass: one composed calculation	43
4.8	Local sensitivity and the chain rule	44

4.8.1	What happens at an inactive ReLU?	45
4.9	Backward pass and backpropagation	46
4.9.1	Backpropagation is not gradient descent	46
4.9.2	Why the forward pass must remember things	46
4.10	Representation learning and optimisation meet	47
4.11	What a neural network is — and is not	48
4.12	Connections to the Week 3 practicals	48
4.13	A compact map of the computation	49
4.14	Where we go next	49
5	Training, generalisation and diagnosis	50
5.1	Backpropagation in a complete network	51
5.1.1	Automatic differentiation is bookkeeping, not an optimiser	51
5.2	Classification: from scores to objectives	52
5.2.1	Binary classification	52
5.2.2	Multi-class classification	53
5.3	A loss can decrease while learning goes wrong	54
5.3.1	Plateaus, saddles and local minima	54
5.3.2	Vanishing and exploding gradients	54
5.3.3	Dead ReLUs	55
5.4	Initialisation: beginning in a learnable regime	55
5.5	Optimisers change the trajectory	56
5.5.1	Momentum	56
5.5.2	Adam and AdamW	56
5.5.3	Learning-rate schedules	56
5.5.4	Curriculum learning	57
5.6	Keeping scales under control: normalisation	57
5.7	Generalisation: fitting the sample is not the final task	57
5.7.1	Underfitting and overfitting	58
5.7.2	Capacity and the limits of the simple bias–variance cartoon	58
5.7.3	Regularisation	58
5.8	Shortcuts: learning can succeed for the wrong reason	59
5.9	Calibration: confidence is another behaviour to evaluate	59
5.10	Experimental discipline: ask one diagnostic question at a time	60
5.10.1	Better than what?	60
5.10.2	Does the output depend on the intended input?	60
5.10.3	What is happening inside the representation?	61
5.10.4	Train and validation answer different questions	61
5.10.5	Multi-task learning: one representation, several objectives	61
5.11	A compact diagnostic workflow	62
5.12	What to carry forward	63
6	Convolution and inductive bias	65
6.1	From prediction to representation	65
6.2	Why not flatten the image?	66
6.3	The convolutional idea: one local detector, reused	67
6.3.1	Locality	67
6.3.2	Weight sharing	68
6.4	Translation equivariance is not invariance	68
6.5	Images have channels; convolutions create channels	68
6.6	Stride, padding and output size	70

6.6.1	Padding	70
6.6.2	Stride	70
6.7	Receptive fields grow through depth	70
6.8	Pooling: useful forgetting	71
6.9	The architecture chooses what must be relearned	72
6.10	A short history of convolutional architecture ideas	72
6.10.1	LeNet: local features and shared weights	72
6.10.2	AlexNet: scale, compute and deep visual learning	72
6.10.3	VGG: repeated small convolutions	73
6.10.4	ResNet: make depth easier to optimise	73
6.11	Pretraining and transfer: reusing the encoder	73
6.12	Autoencoders: learning a visual bottleneck without labels	74
6.12.1	The bottleneck is a design decision	75
6.13	Application to the architecture: a visual currency for StoryReasoning	76
6.14	Representation is not reasoning	76
6.15	What architecture has bought us	77
6.16	A practical reading of a CNN	77
6.17	Summary	78
7	Explanation as evidence	79
7.1	What are we trying to explain?	79
7.1.1	A prediction, a representation, or a mechanism	79
7.1.2	Local and global explanations	80
7.1.3	Model-specific and model-agnostic methods	80
7.1.4	Post-hoc and intrinsically interpretable models	80
7.2	Sensitivity is not the same as importance	80
7.3	Integrated Gradients: explaining a change from a reference	82
7.3.1	Expected Gradients	82
7.4	Perturbation turns an explanation into a test	83
7.4.1	Occlusion and feature perturbation	83
7.4.2	Permutation importance	83
7.5	Local surrogate models: approximate the black box nearby	83
7.6	SHAP: contributions through coalitions	84
7.7	Counterfactual explanations	85
7.8	What is inside a representation? Probes	85
7.8.1	From probe to intervention	86
7.9	Faithfulness: an explanation should predict behaviour	86
7.10	Attention, gates and other inspectable quantities	87
7.11	Intrinsic interpretability and sparse representations	88
7.12	Transfer learning as a question about reusable information	89
7.13	A hierarchy of claims	89
7.14	What should you ask when shown an explanation?	90
7.15	Chapter summary	91
8	Language, sequence and memory	92
8.1	Turning language into numbers	92
8.1.1	Words, tokens and subwords	92
8.1.2	One-hot vectors and learned embeddings	93
8.1.3	A bag of words forgets order	93
8.2	Recurrent neural networks: a state that moves through time	94
8.2.1	The same rule, unrolled	94

8.2.2	Backpropagation through time	95
8.3	Gating: learning what to keep and what to change	96
8.4	Language modelling: predicting one token from its history	96
8.4.1	Teacher forcing	97
8.4.2	Cross-entropy and perplexity	97
8.5	The model learns a distribution; decoding chooses an output	97
8.5.1	Greedy decoding	97
8.5.2	Sampling and temperature	98
8.6	When a decoder ignores its condition	98
8.6.1	Why teacher forcing can make the condition easy to ignore	99
8.6.2	Making the easy signal less reliable	99
8.7	Sequence-to-sequence models: encoding one sequence, decoding another	99
8.8	A fixed hidden state is an information bottleneck	100
8.9	Attention: retrieve instead of remembering everything in one vector	101
8.10	The first transformer view	102
8.11	What to carry into generative modelling	102
9	From one prediction to a distribution	104
9.1	One recorded target does not imply one possible future	105
9.1.1	Squared error predicts a conditional mean	105
9.1.2	The irreducible part of point prediction	106
9.2	Autoencoders: reconstruction is not yet generation	106
9.2.1	The latent space may have holes	107
9.3	Latent variables: put the alternatives inside the model	108
9.4	Variational autoencoders: make the latent sampleable	108
9.4.1	Reconstruction and KL play different roles	109
9.4.2	The reparameterisation trick	109
9.4.3	How much KL?	110
9.5	Conditional generation: a distribution that depends on context	111
9.5.1	Conditioning can still be ignored	112
9.6	GANs: learn through a critic rather than reconstruction	112
9.7	Diffusion: generation as repeated denoising	113
9.7.1	Predict the corruption	114
9.7.2	Conditioning a denoiser	114
9.8	CLIP: align modalities without generating either one	115
9.9	How should a generative model be evaluated?	115
9.9.1	Fidelity and diversity	115
9.9.2	Average sample quality versus best-of-K	116
9.9.3	Use the condition test again	116
9.9.4	Nearest neighbours and memorisation	116
9.10	Four generative mechanisms, four different contracts	117
9.11	What to carry into Week 9	117
10	Attention, transformers and information routing	119
10.1	From a memory lookup to attention	119
10.1.1	Keys and values do different jobs	120
10.2	Scaled dot-product attention	120
10.3	A query should depend on the question	122
10.4	Self-attention: the sequence queries itself	123
10.4.1	Self-attention does not know order by itself	123
10.5	Position: where did this token occur?	123

10.6	Multi-head attention: several interaction spaces	124
10.7	Masks: attention is also an access-control mechanism	125
10.7.1	Bidirectional attention answers a different question	125
10.8	The transformer block: route, transform, preserve	125
10.9	Three common transformer arrangements	126
10.9.1	Encoder-style models	126
10.9.2	Decoder-style models	127
10.9.3	Encoder–decoder models	127
10.10	Self-supervision: the data can manufacture the target	127
10.11	Contrastive learning and aligned spaces	127
10.12	Cross-attention: one representation queries another	128
10.12.1	Early fusion and routed fusion	129
10.13	Application to the architecture: when a weight is only a prior	130
10.14	Retrieval is another form of routing	130
10.15	What attention does not solve	131
10.16	What to carry into Week 10	132
11	Scaling, representation geometry and what the metric sees	133
11.1	Scaling is not one knob	133
11.1.1	Empirical scaling laws	134
11.2	Pretraining changes where the capacity comes from	135
11.3	Conditional computation: grow capacity without using all of it	136
11.3.1	The router is part of the model	136
11.3.2	Routing has its own failure modes	136
11.4	A representation is a geometry, not a list of neurons	137
11.5	Anisotropy: when everything points partly the same way	138
11.6	Collapse: when the distinctions actually disappear	139
11.7	Accuracy can stop while the representation keeps moving	139
11.7.1	Neural collapse	140
11.8	Delayed generalisation: grokking as a warning about training time	140
11.9	Superposition: more features than obvious coordinates	141
11.10	Sparse autoencoders: try to recover a feature basis	142
11.11	How should a representation be measured?	142
11.11.1	Similarity and centred similarity	142
11.11.2	Nearest-neighbour retrieval	142
11.11.3	Linear probes	143
11.11.4	Activation spread and spectrum	143
11.11.5	Ablations and interventions	143
11.12	Application to the architecture: scale moved some heads, not the bottleneck	143
11.13	Metric validity: does the score distinguish the failures we care about?	144
11.13.1	Target-aligned semantic similarity	144
11.13.2	Set-level Fréchet distance	144
11.13.3	Sharpness is yet another property	144
11.14	When the metric becomes the loss	145
11.15	A final scientific habit: locate the bottleneck before changing the model	145
11.16	The compulsory learning thread, completed	146
11.17	Optional continuation: where does learning end?	147

12 Optional extension: learning during inference	149
12.1 The old boundary: training changes parameters, inference uses them	149
12.2 In-context learning: task information in the input	150
12.2.1 What was learned during training?	151
12.2.2 More context is not automatically better	151
12.3 An induction-style hypothesis: retrieve a pattern, then copy	151
12.3.1 Content and position can interact	152
12.4 External memory: knowledge outside the parameters	153
12.4.1 Parametric and non-parametric knowledge	153
12.4.2 Retrieval depth is a computational choice	153
12.5 Test-time adaptation: parameters can change after deployment	154
12.5.1 A proxy objective carries assumptions	154
12.5.2 Adaptation creates state and persistence questions	155
12.6 Test-time compute: inference need not be one forward pass	156
12.6.1 Sampling and selection	156
12.6.2 Two-stage retrieval and reranking	156
12.7 Application to the architecture: retrieval instead of drawing the future	157
12.8 Where does the information live?	158
12.8.1 Parameters	158
12.8.2 Activations and context	158
12.8.3 External memory	158
12.8.4 Inference procedure	158
12.9 A diagnostic workflow for adaptive inference	159
12.10 What this optional week adds to the learning thread	159

Chapter 1

The running case study: predicting what happens next

These notes use one system repeatedly as a case study. It is useful precisely because it is not a clean toy problem. The system is asked to continue a short visual story: from four consecutive movie frames and four textual descriptions, it must predict a fifth frame and a fifth description. The data also identify characters, objects, settings and character bounding boxes. At first sight this sounds like an ordinary supervised-learning problem. It is not.

The difficulty is that films are edited. In most five-frame windows the fifth frame is a different camera shot from the preceding frames. The next image is therefore only partly determined by what has already been observed. Many visually different frames can be perfectly plausible continuations. A learner that is asked for a single pixel-perfect answer is being given an impossible contract: the data contain several possible futures, while the objective insists that exactly one of them was correct.

This tension — between what the data determine and what the objective rewards — is the reason the architecture is useful throughout the module. We will not introduce every component at once. Each week will return to one mechanism or one diagnostic when the relevant concept has become available.

1.1 The task

Let a story provide a sequence of frames and descriptions,

$$(I_1, s_1), (I_2, s_2), \dots$$

For one training example the system receives

$$x = ((I_1, s_1), \dots, (I_4, s_4))$$

and the target is

$$y = (I_5, s_5).$$

The frame is represented at a modest resolution of 60×125 pixels. The descriptions are short natural-language sentences. Structured annotations tell us which characters occur in each frame, where they are, which objects are present, and the setting. [Figure 1.1](#) shows a typical five-frame window and the shot change that makes the task ambiguous.

The phrase “predict the next frame” hides several different problems. We might ask for the exact target pixels, a compact representation of the frame, the most plausible frame among a set of candidates, or a distribution from which several plausible futures can be

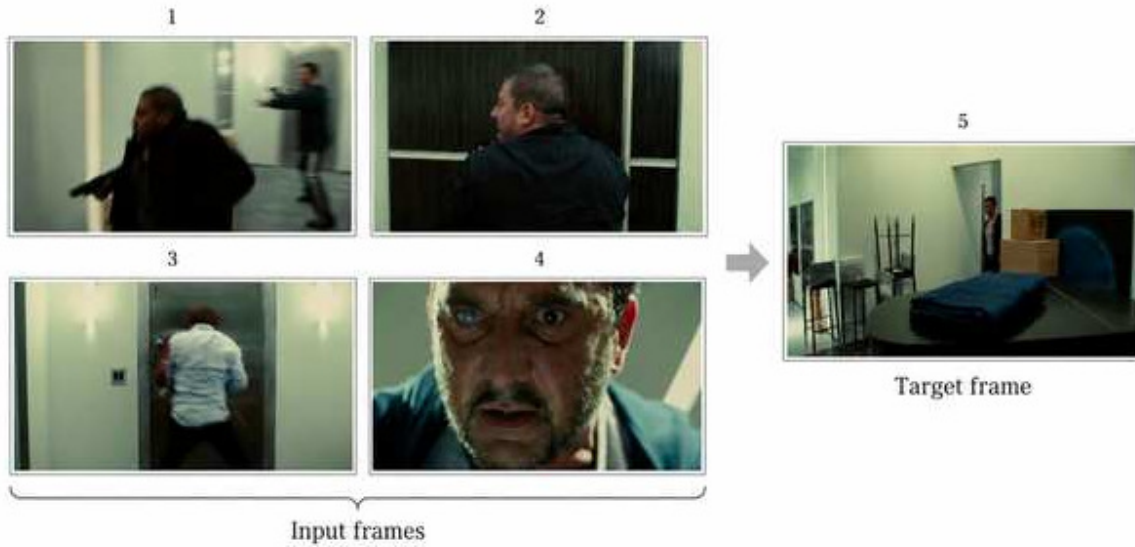


Figure 1.1: A five-frame window from StoryReasoning (story 3920). Frames 1–4 form the observed context and frame 5 is the prediction target. Although the frames are temporally consecutive, the target can differ sharply in viewpoint and visible content from the immediately preceding frame. The example makes the central difficulty concrete: predicting what happens next is not always the same as extrapolating recent pixel motion.

sampled. These are not interchangeable formulations. They expose the learner to the same raw observations but ask different questions of them.

The full system used as the running case study is summarised in [figure 1.2](#). At this stage the diagram is a map rather than a specification: its components will only become meaningful as the relevant ideas are introduced.

1.2 The central trap: an average can win

Suppose two futures are plausible: in one the camera cuts to character A, and in another it cuts to character B. A learner trained with a pixelwise absolute-error loss is not rewarded for producing a sharp image of either plausible future unless it happens to match the recorded target. Across many ambiguous examples, the safe prediction moves toward a per-pixel median of the possible outcomes. The result can be visually unconvincing and nevertheless obtain a better pixel score than a sharp but different plausible frame.

The repository makes this visible before introducing a complex model. On the held-out StoryReasoning windows, a constant per-pixel median image obtains an image L1 of about 0.151. Copying the last real frame gives roughly 0.170. Even an oracle that is allowed to inspect the target and choose the best of the four input frames reaches only about 0.130. On shot cuts, which account for roughly 85% of the windows, a real frame from the same story can therefore score worse than the bland constant image.

Key idea

A numerical objective is not a neutral observer. It defines which errors count. If the target is genuinely multimodal, a loss on one deterministic output can reward averaging even when averaging destroys the property we care about.

This is why the case study begins with floors and diagnostics rather than with an elaborate neural network. Before asking whether a model is sophisticated, we ask what a trivial predictor can achieve and whether the metric rewards the behaviour we actually want.

1.3 The full system, seen from a distance

The eventual architecture contains several familiar pieces, but they enter the course one at a time.

A convolutional autoencoder first learns a compact latent representation of each frame. Text descriptions are represented using a frozen pretrained sentence encoder, while a recurrent language model learns how descriptions in the corpus are written. For each of the four observed time steps, visual and textual representations are fused and passed through a recurrent sequence model. From that sequence representation, different heads predict visual information, textual information and character presence.

Several later mechanisms are responses to specific failures. Content-dependent attention can select useful inputs, but selection must be tested rather than assumed. Pretrained components require protection against fine-tuning drift. Per-character prediction works better when each character’s own history is preserved rather than compressed into one shared vector. The text generator benefits from attending directly to descriptions and character names. Finally, because the next image is genuinely uncertain, a variational latent models a distribution over possible futures rather than forcing all uncertainty into one deterministic vector.

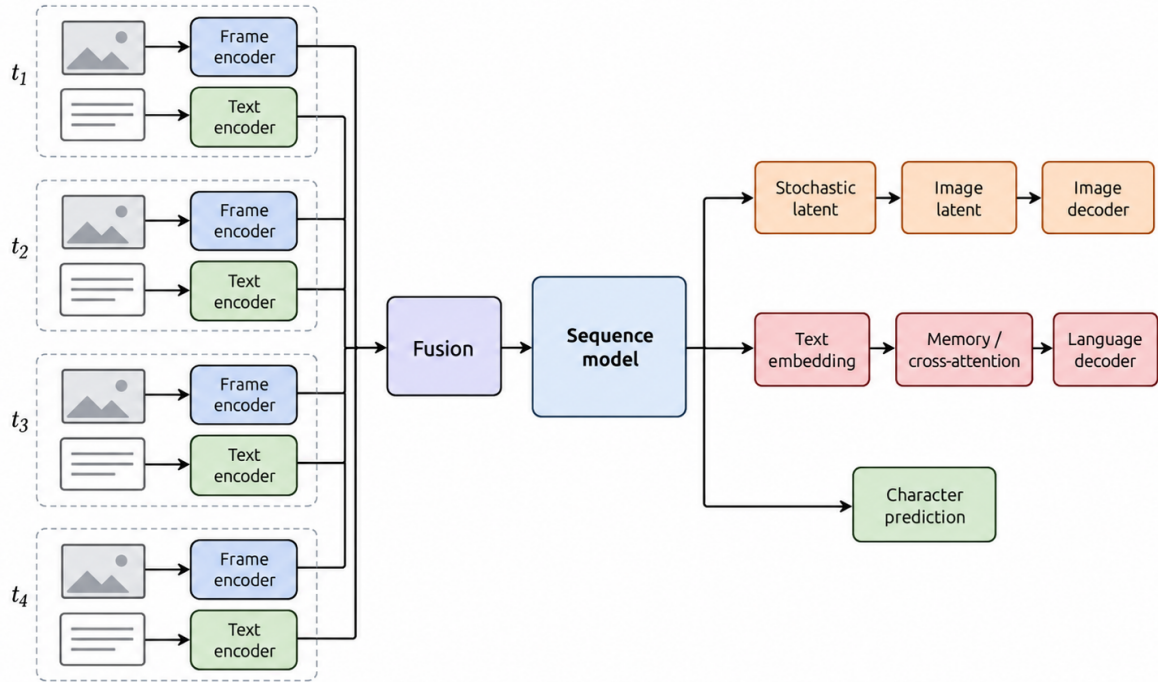


Figure 1.2: The StoryReasoning architecture at a glance. Four frame–description pairs are encoded and fused before a sequence model integrates information across time. From the shared sequence representation, separate branches support image prediction, text generation and character prediction. The image branch includes a stochastic latent variable so that more than one visual continuation can be represented, while the text branch uses a text embedding followed by memory and cross-attention before language decoding.

The important point is not the inventory of modules. The architecture is a record of questions:

- Can a frame be compressed without losing the information needed to reconstruct it?
- Does the sequence model actually use the textual condition, or can the decoder ignore it?
- Does attention vary with the content of each example, or merely learn a positional prior?
- When several losses share one representation, do they cooperate or compete?
- Do pretrained encoders retain the representations that made them useful?
- If several futures are plausible, should the output be a point estimate, a retrieval ranking, or a distribution?

Each question has a diagnostic. That habit is as important as any individual architectural component.

1.4 How to read the architecture boxes

Most weekly chapters contain a small box titled *Application to the architecture*. The box does not attempt to teach the complete system. It takes one concept from the week and asks what that concept reveals about the running case study.

In Week 1 the issue is task definition and baselines. In Week 2 it is loss as specification. Later weeks turn to representation collapse, convolutional encoders, memory, attention, generative latent variables and representation geometry. The architecture therefore develops twice: once as an engineering system, and once as a sequence of increasingly precise questions we learn how to ask of it.

That second development is the more important one.

Chapter 2

Learning from examples

A machine can be made to perform a task in at least two very different ways. We may attempt to write down the rules ourselves, or we may arrange matters so that the machine acquires a rule from examples. The second possibility is what concerns us here.

This distinction sounds simple, but nearly every difficulty in machine learning can be traced to a question hidden inside it. Which examples were shown? What information did they contain? What rule was the machine allowed to represent? What counted as a mistake? And, after the machine has adapted itself to the examples, why should the resulting rule work on an example it has never seen?

This chapter establishes the vocabulary with which the rest of the module will answer those questions. We will deliberately resist the temptation to begin with a large neural network. The first object of study is the *learning problem* itself.

Key idea

Machine learning is the construction of a rule from a finite sample under a chosen objective. The algorithm, the architecture and the dataset matter because each restricts which rule can be found; the objective matters because it states what “better” means.

2.1 Several traditions, one modern subject

Modern machine learning did not descend from a single idea. It sits at a meeting point of statistical inference, computation, pattern recognition, neuroscience and artificial intelligence. These traditions asked different questions, and the field still carries the marks of all of them.

2.1.1 Statistics: fitting a rule to observations

Statistical modelling begins from a stubborn fact: observations vary. If we measure the height of a population, the demand for a product or the position of points in a noisy experiment, no finite table of measurements is the phenomenon itself. We infer regularities from a sample.

The classical statistical tradition developed methods for estimation, regression, likelihood and uncertainty. Pearson’s work on fitting low-dimensional lines and planes to clouds of points is an early example of a theme that will reappear throughout this module: choose a representation and an error criterion, then find the parameters that make the observations least surprising or least discrepant (Pearson 1901). What we now call “training a model” inherits much of this logic.

The statistical contribution is not merely a bag of techniques. It supplies a discipline of thought: a finite dataset is evidence about a larger process, and any conclusion drawn from

it is conditional on how that evidence was obtained.

2.1.2 Computation and artificial intelligence: what can a machine do?

The development of programmable digital computers made a different question urgent. Could tasks associated with intelligence be expressed as computation? Turing’s famous discussion of machine intelligence shifted attention away from defining an essence called “thinking” and toward observable capabilities and procedures (Turing 1950). Early artificial intelligence often pursued explicit symbolic rules: if the world could be described in the right symbols, perhaps reasoning could be written as transformations over those symbols.

Symbolic systems remain important. But they expose a practical difficulty. Many tasks that humans perform effortlessly — recognising a face, parsing speech in a noisy room, judging whether two images depict the same object — are difficult to reduce to a complete list of hand-written rules. The world supplies more variation than the programmer can conveniently enumerate.

Machine learning offers another route: specify examples and an objective, then let the parameters of a model absorb some of that variation.

2.1.3 Neuroscience: inspiration without identity

A third influence came from attempts to understand nervous systems. McCulloch and Pitts described simplified threshold units as logical elements (McCulloch and Pitts 1943); Hebb proposed a principle by which co-activation might strengthen connections (Hebb 1949); Rosenblatt’s perceptron turned the idea of an adjustable weighted unit into a learning system (Rosenblatt 1958).

These ideas mattered enormously, but a warning is useful from the beginning. An artificial neural network is not a miniature biological brain. A modern unit that computes a weighted sum followed by a non-linearity is at most a distant abstraction of a biological neuron. Backpropagation, tokenisation, batch normalisation, attention and GPU matrix multiplication are engineering and mathematical devices, not claims about the literal operation of cortex.

Biology supplied questions and metaphors: distributed representation, adaptation, local interactions, hierarchical processing. Deep learning is best understood as a mathematical technology that borrowed some of those broad ideas and then developed according to its own constraints.

2.1.4 From hand-designed features to learned representations

For much of the history of pattern recognition, a large fraction of the intelligence in a system was supplied by the designer. An image classifier might first be given edges, corners, texture descriptors or other carefully chosen measurements; a separate learning algorithm then mapped those measurements to labels.

Neural networks alter this division of labour. Intermediate representations can themselves be adjusted by learning. The revival of multilayer networks through efficient backpropagation (Rumelhart et al. 1986), followed decades later by larger datasets, faster hardware and successful deep architectures, made this strategy practical at scale. Deep learning came to dominate many perceptual tasks because multiple layers could learn progressively useful transformations rather than relying entirely on a fixed feature pipeline (LeCun, Bengio, et al. 2015).

The recent development of large pretrained and foundation models extends the same tendency. A representation may be learned from a very broad corpus with a self-supervised

objective, then reused or adapted for many later tasks (Bommasani et al. 2021). The centre of gravity has moved from “which features shall we program?” toward “what objective, data and architecture will cause useful features to emerge?”

The convergence of these traditions is summarised in figure 2.1. Statistics contributes methods for estimation and reasoning under uncertainty; computation and artificial intelligence contribute algorithmic and symbolic approaches; neuroscience and pattern recognition contribute ideas about adaptive weighted units and distributed representations. Modern deep learning inherits from all three while increasingly shifting the burden of feature design toward learned representations.

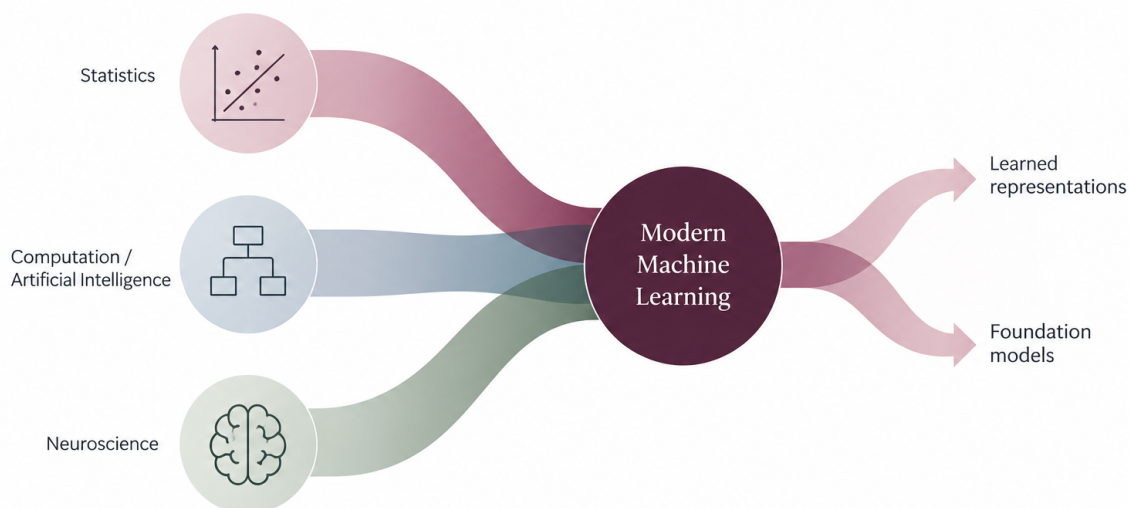


Figure 2.1: Three intellectual traditions that shaped modern machine learning. Statistics contributed estimation and reasoning under uncertainty; computation and artificial intelligence contributed algorithms and symbolic representations; neuroscience and pattern recognition contributed ideas about adaptive weighted units and distributed representation. Modern deep learning combines these influences in systems that learn useful representations from data, a tendency extended by large pretrained foundation models.

Check your understanding

Why is it misleading to say that deep neural networks are “models of the brain”? Give one idea that was biologically inspired and one important mechanism in a modern network that is primarily an engineering construction.

2.2 Examples are not the world

Consider a dataset of handwritten digits. It may contain sixty thousand training images, which sounds large. Yet there are vastly more possible handwritten digits than those sixty thousand. Different writers use different pens, angles, pressures and styles; future users may come from a different population from the people whose handwriting was collected.

The dataset is therefore not the task. It is a finite observation of the process from which future cases will arise.

We will use the following notation. An input example is x . When a target is available,

it is y . A supervised dataset of n examples is

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n.$$

A useful idealisation is that these examples were produced by some underlying data-generating distribution $p(x, y)$:

$$(x_i, y_i) \sim p(x, y).$$

The symbol p need not mean that we know the distribution analytically. Usually we do not. It expresses a more modest claim: there is a process that makes some examples common, some rare, and some perhaps impossible.

2.2.1 Sampling variability

If we draw two small samples from the same population, they need not look alike. One sample of 20 MNIST digits might contain several zeros and no nines; another might suggest the opposite. Neither sample is “wrong”. They are different finite observations.

As sample size increases, empirical proportions usually become more stable. But “usually” is an important word. A particular sample of 100 can by chance be more representative of a population than a particular sample of 1000. Statistical regularity is a statement about repeated sampling, not a guarantee attached to every individual dataset.

This is the first reason machine learning is empirical. A measured accuracy is not a permanent property stamped onto a model. It is a measurement obtained on a particular sample.

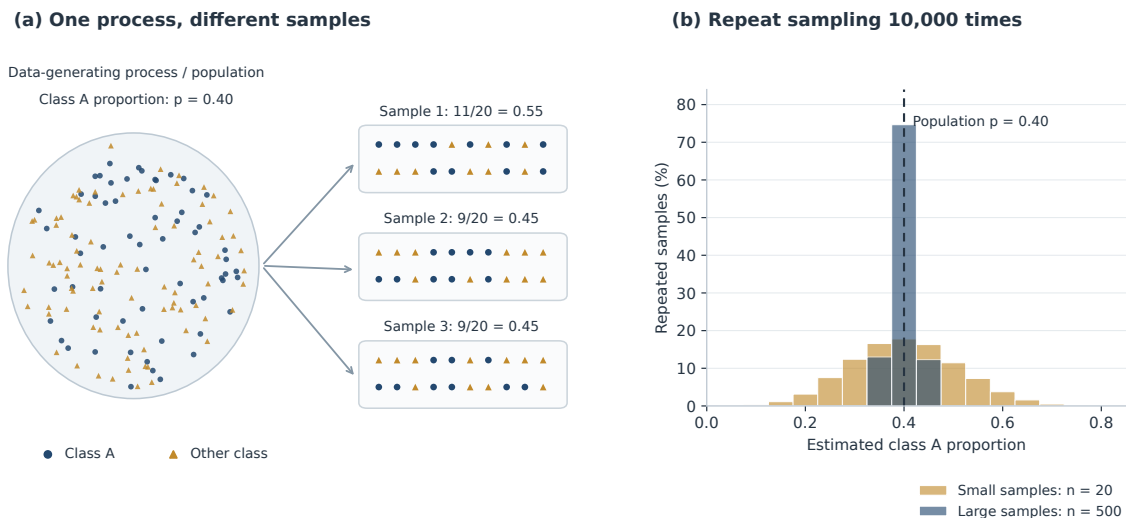


Figure 2.2: A dataset is a sample from a larger process. Repeated small samples can give noticeably different estimates; larger samples tend to reduce, but do not abolish, sampling variability.

This idea is explored directly in the Week 1 sampling practical. The purpose of the lecture is not to reproduce that experiment on paper, but to give it a place in the larger argument: every later claim about a model will be made through finite samples, so uncertainty enters before the first neural network has even been constructed.

2.2.2 Training, validation and test data

Once parameters are adjusted to fit examples, those examples can no longer provide an independent measurement of how well the rule works on unseen cases. A learner can exploit

peculiarities of its training data. We therefore separate data according to purpose.

The *training set* is used to fit model parameters. The *validation set* is used during development to compare designs, tune hyperparameters or decide when to stop. The *test set* is reserved for a final estimate after those choices have been made.

This is not bureaucratic bookkeeping. Repeatedly looking at a test set while changing a model turns the test set, indirectly, into training information. The model developer begins adapting decisions to that finite sample. The measurement becomes less independent.

We will return to overfitting and experimental design in Week 4. For now, keep one principle: the data used to choose a rule and the data used to judge its generality play different roles.

2.3 What exactly is the machine being asked to do?

The phrase “learn from data” is incomplete. We must specify the form of the question and the form of the answer. Different tasks can use the same raw data while defining different learning problems.

2.3.1 Classification

In classification the target is a discrete category. Given an image x , a digit classifier might predict

$$\hat{y} \in \{0, 1, \dots, 9\}.$$

Other examples include spam detection, sentiment classification, medical triage categories and identifying species in an image.

The output need not be only a final label. A model often computes a score for each possible class and the label is derived from those scores. We will study that mechanism in detail in later weeks.

Classification answers a question of the form: *which of these categories best describes the input?*

2.3.2 Regression

In regression the target is numerical and usually treated as continuous. Examples include predicting energy consumption, estimating the age of an object, forecasting a temperature or predicting a physical coordinate.

A regression model may have one output,

$$\hat{y} \in \mathbb{R},$$

or many numerical outputs,

$$\hat{\mathbf{y}} \in \mathbb{R}^d.$$

Week 2 will use regression to build our first complete differentiable learning system.

2.3.3 Density estimation and generative modelling

Sometimes the objective is not to predict a supplied label at all. We may want to model how the observations themselves are distributed.

A density model attempts, explicitly or implicitly, to assign high probability to plausible observations and low probability to implausible ones. A generative model uses what it has learned to produce new samples. Modern image generators and language models belong

broadly to this family, although the probability models and training procedures can be very different.

Text completion is a useful example. Given preceding text, a language model predicts a distribution over the next token. Repeating that operation generates a sequence. It is still a prediction problem, but the object predicted is a probability distribution and there may be many acceptable continuations.

2.3.4 Structured prediction

Some outputs contain many related parts. A semantic segmentation model predicts a category for each pixel. An object detector predicts a set of bounding boxes, class labels and confidence scores. Speech recognition and translation produce sequences whose elements depend on one another.

Calling these tasks “classification” misses an important fact: the structure of the output matters. The model must predict not merely *what* is present, but an organised collection of interdependent quantities.

2.3.5 Ranking and retrieval

In ranking and retrieval, the model need not manufacture the answer. It may instead score existing candidates.

A search system ranks documents for a query. A recommendation system ranks items for a user. An image–text retrieval model can be asked to find which image best matches a caption. The output is therefore a comparison:

$$s(x, c_1), s(x, c_2), \dots, s(x, c_m),$$

followed by an ordering of candidates c_j .

This distinction becomes important in our running architecture. Predicting the exact pixels of the next movie frame is very different from retrieving the true next frame from a candidate set, even though both are called “next-frame prediction” in casual conversation.

Problem type	Typical output	Example questions
Classification	discrete class or class scores	Which digit? Is this review positive?
Regression	one or more real values	What temperature? What position?
Generative / density	distribution or sample	What text/image could plausibly come next?
Structured prediction	sequence, mask, set, boxes	Which word sequence? Which pixels belong to each class?
Ranking / retrieval	scores or ordering over candidates	Which document, image or item is most relevant?

Table 2.1: Several common machine-learning problem types. The categories overlap in modern systems, but each foregrounds a different form of output.

Optional exercise — one dataset, several tasks

Take a collection of street photographs with object labels, bounding boxes and captions. Formulate five different learning problems from the same raw data: one classification problem, one regression problem, one structured-prediction problem, one generative problem and one retrieval problem. For each, state precisely what x and y

are.

The useful difficulty is not inventing an application; it is discovering how much the mathematical problem changes when the target changes.

2.4 Ways of obtaining a teaching signal

Problem type describes the form of the task. A second distinction concerns where the learning signal comes from.

2.4.1 Supervised learning

In supervised learning, training examples contain inputs paired with desired targets:

$$(x_i, y_i).$$

The target may be a class, a number, a mask, a sequence or some more complicated structure. The learner receives evidence about what answer should be produced for each training input.

Supervision does not mean that a human necessarily typed every label. Labels can come from instruments, existing databases, simulations or other automated processes. The defining point is the pairing of an input with a target for the task being learned.

2.4.2 Unsupervised learning

In unsupervised learning, there is no task-specific target y_i supplied with each observation. The learner is asked to discover structure in the observations x_i themselves: clusters, low-dimensional factors, density, recurring patterns or useful representations.

The word “unsupervised” can be misleading if taken to mean “without an objective”. There is always some computational criterion. What is absent is the externally supplied target for the final task.

2.4.3 Self-supervised learning

Self-supervised learning constructs a prediction problem from the data themselves. The target is present inside the observation rather than supplied as a separate human annotation.

A language model can hide or shift part of a text sequence and predict it from the rest. An image system can learn by matching different views of the same image. A video system can predict withheld temporal information. In each case the data generate their own supervisory signal.

This strategy has become central because raw text, images, audio and video are abundant while carefully labelled examples are expensive. Large pretrained models can learn representations from broad unlabelled corpora and later be adapted to a smaller supervised task.

2.4.4 Semi-supervised learning

Semi-supervised learning combines a smaller labelled collection with a larger unlabelled collection. The aim is to use structure learned from the unlabelled data to reduce the amount of expensive supervision required.

In practice the boundaries between these paradigms are porous. A system might be pretrained self-supervised on millions of images, fine-tuned with supervised labels, and then

used inside a larger system trained with another objective. The paradigms describe sources of learning signal, not mutually exclusive species of algorithm.

2.4.5 Reinforcement learning: a different feedback structure

Reinforcement learning is not a focus of this module, but it belongs on the map. An agent acts, observes consequences and receives rewards rather than a correct target for every action. The feedback can be delayed: an action may matter because of what it makes possible many steps later.

For our purposes, notice the contrast. Supervised learning often says “for this input, here is the desired output”. Reinforcement learning says “after interacting with the environment, here is a scalar indication of how well things went”. Both are learning from experience, but the information supplied by experience is different.

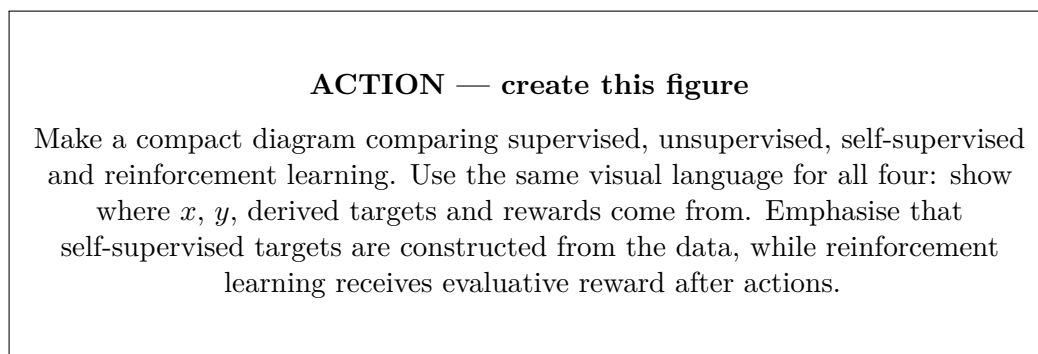


Figure 2.3: Learning paradigms differ mainly in the source and structure of the teaching signal. Modern systems often combine more than one paradigm.

2.5 The supervised-learning skeleton

We now write down the smallest complete supervised-learning system. The notation introduced here will persist for the rest of the module.

2.5.1 1. A dataset

We observe

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n.$$

The x_i are inputs; the y_i are targets. Their form depends on the task.

For digit recognition, x_i is an image and y_i is a digit class. For house-price prediction, x_i might be a vector of measurements and y_i a price. For translation, both x_i and y_i are sequences.

2.5.2 2. A parameterised model

A model is a function with adjustable parameters:

$$f_{\theta}(x).$$

The symbol θ stands for all adjustable quantities: perhaps two numbers in a tiny regression model, perhaps billions of weights in a large neural network.

For an input x_i , the model produces a prediction

$$\hat{y}_i = f_{\theta}(x_i).$$

The parameters are not the same thing as the input. An input changes from example to example. The parameters persist and are changed by learning.

2.5.3 3. A loss

A loss function measures discrepancy between prediction and target:

$$\ell(\hat{y}_i, y_i).$$

Training usually aggregates losses over many examples:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i).$$

At this stage we will not ask how θ is changed. That is the subject of Week 2. The important point is logical: the learner can only improve relative to a criterion. “Make better predictions” becomes operational only when “better” has been encoded in an objective.

2.5.4 4. Training

Training is the process that changes θ so as to reduce the chosen objective on training data. In modern neural networks, this is usually done iteratively using gradients. But the abstract idea is broader. The Week 1 perceptron practical already gives an example of iterative parameter adaptation without requiring the gradient-descent machinery we will develop next.

Training is therefore not the same thing as running the model. It is a process that modifies the persistent rule.

2.5.5 5. Inference

After training, inference means applying the learned model to inputs:

$$x \mapsto f_{\theta}(x).$$

The parameters are normally held fixed. Inference asks what the learned rule predicts; training asks how the rule itself should change.

Later in the module we will deliberately blur this boundary when studying systems that retrieve, adapt or perform optimisation-like computation during a forward pass. For now, keeping the distinction sharp is useful.

2.5.6 6. Evaluation

Finally, we measure behaviour on examples not used to fit the parameters. Evaluation requires a metric that corresponds to the question we care about: accuracy, absolute error, top- k retrieval, intersection-over-union, calibration error, human preference, or something else.

The training loss and evaluation metric can be related without being identical. A differentiable loss may be convenient for learning while a task metric is more meaningful for reporting. This difference will matter repeatedly.

ACTION — create this figure

Draw the six-part supervised-learning loop: dataset $\rightarrow$ model $f_\theta \rightarrow$ prediction $\hat{y} \rightarrow$ loss against $y \rightarrow$ parameter update, with a separate evaluation arrow to held-out data. Visually distinguish the training loop from the inference path $x \rightarrow f_\theta(x) \rightarrow \hat{y}$.

Figure 2.4: The skeleton of supervised learning. Week 1 names the parts; Week 2 will explain how the loss produces parameter updates.

PyTorch: the notation in code

```
import torch
from torch import nn

# x: a batch of input examples
# y: the corresponding targets

model = nn.Linear(in_features=4, out_features=1) # f_theta
y_hat = model(x) # prediction

loss_fn = nn.MSELoss() # ell(...)
loss = loss_fn(y_hat, y) # L(theta)
```

This code is intentionally incomplete. It creates a parameterised model, performs a forward computation and evaluates a loss. It does not yet change the parameters. In Week 2 we will add the mechanism that connects the loss to an update.

Notice also how little of the mathematics depends on PyTorch. ‘nn.Linear’ is one implementation of f_θ ; ‘nn.MSELoss’ is one implementation of ℓ . The conceptual structure survives if both are replaced.

Check your understanding

For a speech-recognition system, identify x , y , $\hat{y}$, θ , a possible training loss and a possible evaluation metric. Which of these change from one example to the next, and which persist across examples?

2.6 Learning is not merely fitting

It is easy to produce a rule that works on the data already observed. The difficult claim is that the rule captures something that persists beyond them.

Suppose a student memorises the answers to a practice sheet. On the same sheet, performance can be perfect. A new exam reveals whether a transferable rule was learned. A machine-learning model faces the same logical test.

2.6.1 Empirical risk and the unseen process

The training objective

$$\frac{1}{n} \sum_i \ell(f_{\theta}(x_i), y_i)$$

is an *empirical* quantity: it is computed on observed examples. What we actually care about is expected performance on future examples from the relevant process:

$$\mathbb{E}_{(x,y) \sim p} [\ell(f_{\theta}(x), y)].$$

We cannot usually compute this expectation exactly because we do not possess the full distribution p . Held-out test data provide another finite estimate.

This gap between what is optimised and what is ultimately desired is the home of generalisation.

Generalisation is not a mysterious extra property sprinkled on a trained network. It is the consequence we hope for when the regularities used to fit the sample are also regularities of future data.

2.6.2 The distribution matters

The phrase “unseen data” is not enough. A model trained on daytime road images in Sheffield may perform poorly on snowy night-time images in another country. Those examples are unseen, but they may also come from a changed distribution.

Thus a model does not generalise in the abstract. It generalises from one collection process to another to the extent that the useful regularities are shared.

This is why dataset design is part of model design. A perfect optimiser cannot recover information that the examples never provide.

Optional mathematics — empirical and expected error

Suppose a classifier makes an error with probability q on the population from which a test set is independently sampled. If the test set contains m examples, the number of observed errors is binomial with mean mq .

Without using a normal approximation, write the variance of the measured error rate. What happens to that variance when m is multiplied by four? Relate the result to the widening and narrowing histograms in the Week 1 sampling practical.

2.7 The loss is part of the problem statement

For the moment, treat a loss as a scoring rule that tells the learner which changes count as improvement. This modest definition is already enough to expose an important principle.

Two models can see exactly the same data and be trained on what appears verbally to be the same task, yet learn different behaviours because their objectives differ.

For a numerical target we might penalise squared error,

$$\ell_{\text{MSE}}(\hat{y}, y) = (\hat{y} - y)^2,$$

or absolute error,

$$\ell_{\text{L1}}(\hat{y}, y) = |\hat{y} - y|.$$

These expressions disagree about how strongly large errors should matter. For classification, other losses are designed around class scores and probabilities. For retrieval, a contrastive objective may reward the correct candidate being closer than incorrect candidates.

Week 2 will make this idea concrete. For now the lesson is conceptual: a model never sees the teacher’s intention directly. It sees data and an objective.

Key idea

The loss is a specification written in mathematics. If the specification rewards the wrong behaviour, successful optimisation can make the system worse according to our actual intention.

This is one reason “the training loss went down” is never, by itself, a scientific conclusion. It tells us that the learner became better at the quantity we asked it to optimise. We still have to justify that quantity.

2.8 Better than what? Baselines and floors

A score in isolation carries surprisingly little information. Suppose a classifier achieves 90% accuracy. Is that impressive?

If the ten classes are balanced, 90% may be excellent. If one class appears 95% of the time, a model that ignores the input and always predicts the majority class reaches 95%. The apparently sophisticated classifier is then worse than a constant rule.

A *baseline* is a deliberately simple comparison that tells us what can be achieved without the behaviour we hope the model has learned. Depending on the task, useful baselines include:

- random or chance prediction;
- always predicting the majority class;
- always predicting the mean or median target;
- copying the previous observation in a temporal problem;
- nearest-neighbour or other simple retrieval;
- a small linear model before trying a deep model.

The purpose of a baseline is not to win. It is to remove easy explanations.

If a complex model barely beats a constant predictor, we should not begin by celebrating the complexity. We should ask what information the task and metric make available, whether the model uses the intended input, and whether the comparison itself is sensible.

2.8.1 A baseline can diagnose the metric

Baselines do more than tell us whether a model is good. They can reveal that a metric rewards an unintended solution.

If copying a real previous frame scores worse than a blurry constant image under a next-frame metric, the surprising result is not merely “the copy baseline is bad”. It is evidence that the metric favours central tendency over visual plausibility.

This turns a baseline into an experimental instrument.

Application to the architecture

The StoryReasoning system in [chapter 1](#) predicts the fifth movie frame from four preceding frame–description pairs. Before training the sequence model, the repository

evaluates three image floors under pixel L1:

Predictor	all windows	shot continues	shot cuts
Per-pixel median image	0.151	0.111	0.154
Copy last input frame	0.170	0.041	0.178
Best of four inputs (oracle)	0.130	0.040	0.142

Only about 15% of windows continue the same shot. On those windows, copying a real frame is excellent. On the much more common shot cuts, the constant median image beats copying the last real frame.

This Week 1 observation is already architectural: it tells us that “predict the next frame” has been underspecified. Pixel prediction, latent prediction, retrieval and distributional prediction are different learning problems. A stronger network cannot remove that distinction. Before asking how to optimise the system, we must decide which problem we mean.

Optional coding — a baseline before a model

Create a synthetic binary-classification dataset with 90 examples from class 0 and 10 from class 1. Do not train a model.

Write three predictors: random guessing with equal probability, always class 0, and a predictor that samples according to the observed class proportions. Estimate their accuracies over many repeated predictions.

Then answer the important question: if a learned classifier reports 89% accuracy on this dataset, what evidence would you need before claiming that it learned anything useful about x ?

2.9 A task is a contract

We can now assemble the pieces.

A machine-learning problem specifies at least:

1. a source of examples and a sampling process;
2. what information is presented as input x ;
3. what output or target is desired;
4. a model family f_θ that determines which rules are representable;
5. an objective that decides which parameter changes are rewarded;
6. an evaluation procedure on data not used to fit the parameters;
7. baselines that reveal how much of the score is obtainable trivially.

Change any one of these and, in an important sense, you have changed the learning problem.

This perspective prevents a common mistake: treating “the neural network” as the only interesting object. A network can be perfectly implemented and still fail because the data

omit the needed signal, the target is ambiguous, the loss rewards a shortcut, the evaluation set is unrepresentative, or the baseline already solves the task.

The model is one participant in a system of choices.

2.9.1 An example: sentiment analysis

Suppose we want to classify film reviews as positive or negative.

The input x might be the raw review text. The target y might be a binary sentiment label. A supervised dataset contains many paired reviews and labels. A model f_θ maps text to class scores. A classification loss rewards assigning higher score to the labelled class. Training adjusts θ . Evaluation estimates accuracy on held-out reviews.

Now alter one element at a time.

If the dataset contains reviews from one website and the test data come from another, the sampling process changes. If the input includes the numerical star rating, the model may solve the task by using a trivial leakage feature rather than understanding text. If the target changes from positive/negative to a five-star score, classification may become ordinal classification or regression. If the evaluation metric weights rare negative reviews more heavily, the practical meaning of “good” changes.

None of these alterations requires changing the word “sentiment”, yet the learning problem has changed substantially.

2.9.2 An example: image generation

Suppose a system is given a text prompt and asked to generate an image. There is usually no single correct pixel array. A loss that compares one generated image pixel by pixel with one recorded target may punish a plausible image simply because it differs in harmless details.

The ambiguity is not a defect in the data. It is part of the task. The correct formulation may need a distribution, a perceptual criterion, adversarial comparison, diffusion objective, retrieval metric or human judgement.

This is the same logical difficulty that appears in StoryReasoning, only in a more familiar modern form.

2.10 Where neural networks enter

Nothing so far requires a neural network. That is deliberate.

A neural network is a flexible family of parameterised functions. Its appeal is that the same basic machinery can learn useful mappings in images, text, audio, tabular data and mixtures of these modalities. But the network still lives inside the skeleton we have already built:

$$\text{examples} \rightarrow f_\theta \rightarrow \text{predictions} \rightarrow \text{loss} \rightarrow \text{parameter change}.$$

The next weeks will open the box f_θ . We will first study the simplest numerical transformation, then the mechanism by which a loss changes parameters, then the non-linear layers that give neural networks their expressive power.

The order matters. If we begin with architecture names, it is easy to mistake machine learning for a catalogue: MLP, CNN, RNN, Transformer. If we begin with the learning problem, each architecture can instead be understood as an answer to a question about information and computation.

PyTorch: a tiny neural network is still just a function

```

from torch import nn

model = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 3),
)

# For a batch x with shape [batch, 4],
# the model returns three output scores per example.
scores = model(x)

```

We will not yet analyse why the hidden layer helps, how the parameters are initialised, or how gradients pass through the two linear layers. Those questions belong to Weeks 2–4. At this stage, the code should be read only as a concrete instance of $f_{\theta}(x)$: a parameterised computation from input to output.

2.11 Connections to the Week 1 practical work

The practical exercises are designed to make several claims from this chapter observable rather than merely plausible.

The sampling experiment uses repeated subsamples of MNIST to show that finite samples fluctuate. It is evidence for the distinction between a dataset and a population-level process.

The distribution and summary exercises examine what is preserved or lost when data are reduced to statistics. That theme will later become representation: a model must compress information without discarding what the task needs.

The simple transformation exercise builds geometric intuition for matrices before Week 2 turns a linear transformation into a trainable regression model.

Finally, the mistake-driven classifier shows a primitive but important learning loop. The learner is never handed the correct decision boundary. It receives examples, makes errors, changes persistent parameters and can end at different valid solutions depending on the path through the data. This anticipates optimisation without requiring us to introduce gradient descent prematurely.

The lecture and practical therefore meet in the same chain:

examples $\rightarrow$ sampling $\rightarrow$ representation $\rightarrow$ prediction $\rightarrow$ error signal $\rightarrow$ adaptation.

The rest of the module will make each arrow more precise.

2.12 Optional exercises

1. Formalise a modern task

Choose one: semantic segmentation, speech recognition, text completion, image generation, recommendation, or image–text retrieval.

State:

1. the input x ;
2. the target or learning signal;

3. the form of the output;
4. whether the task is best described as classification, regression, structured prediction, generative modelling or retrieval;
5. one sensible baseline;
6. one reason a high training score might fail to imply useful generalisation.

2. A constant predictor

Suppose a binary dataset contains a proportion p of positive examples.

1. What accuracy is obtained by always predicting the most common class?
2. For which values of p does this baseline exceed 80%?
3. Why does this show that accuracy cannot be interpreted without class proportions?

Then consider a regression target y . Under squared error the best constant prediction is the mean; under absolute error it is a median. You may prove these statements now if you know the necessary calculus, or return to them after Week 2.

3. Same data, different objective

Imagine a dataset of photographs of products, each with a category, price, caption and product ID. Define two tasks that use exactly the same photographs as input but would reward very different internal representations. Explain what information each task encourages the model to preserve.

4. PyTorch reading exercise

For the two-layer network shown earlier, identify which objects are data and which are parameters. Use `list(model.parameters())` and print the tensor shapes. Without training the network, predict what will remain unchanged when a new batch x is passed through the model and what will change.

2.13 Chapter summary

The essential ideas of Week 1 are simple enough to state compactly, but they will govern everything that follows.

A dataset is a finite sample, not reality itself. The learning problem must specify inputs, outputs and the source of its teaching signal. Classification, regression, generation, structured prediction and retrieval ask different kinds of questions even when they share raw data. Supervised, unsupervised and self-supervised learning differ in how the learning signal is obtained.

In supervised learning we will repeatedly write

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \quad \hat{y}_i = f_\theta(x_i), \quad \mathcal{L}(\theta) = \frac{1}{n} \sum_i \ell(\hat{y}_i, y_i).$$

Training changes θ ; inference applies the resulting rule; evaluation asks how well it transfers to held-out observations.

And every reported score should provoke one immediate question:

Better than what?

That question is not scepticism for its own sake. It is the beginning of experimental discipline.

Chapter 3

Regression, loss and gradient descent

In Week 1 we described a learning system from the outside. There were examples, a model, a target, a loss and a procedure for changing parameters. We can now open that box and watch the mechanism move.

We shall begin with the least mysterious useful model: a straight line. This is deliberate. If the model has only two adjustable numbers, then nearly every quantity involved in learning can be drawn. We can see what a parameter does, what an error means, where the loss comes from, which direction improves it, and why an update that is locally sensible can still fail if its step is badly chosen. Nothing essential is hidden by depth, non-linearity or millions of parameters.

The point of linear regression is therefore larger than fitting lines. It gives us the first complete differentiable learning system. Later networks will contain far more elaborate transformations, but the central loop will remain recognisable:

input $\longrightarrow$ prediction $\longrightarrow$ loss $\longrightarrow$ gradient $\longrightarrow$ parameter update.

Key idea

A model does not improve because it has been told the correct parameters. It improves because the loss changes when its parameters change, and the gradient tells us how sensitive that loss is to small parameter changes.

3.1 The simplest adjustable prediction

Suppose an input is a single number x , and the quantity we wish to predict is a number y . We choose the model

$$\hat{y} = wx + b, \tag{3.1}$$

where w and b are adjustable parameters.

This model is almost embarrassingly simple, which is why it is so useful. The parameter w changes how strongly the prediction responds to the input. In a graph of x against $\hat{y}$, it changes the slope. The parameter b changes the prediction even when $x = 0$; geometrically, it translates the line vertically.

If $w = 0$, the input has no effect at all:

$$\hat{y} = b.$$

The model has become a constant predictor. That apparently trivial limiting case will matter repeatedly. A learner can reduce its loss substantially by learning a useful constant while learning almost nothing about the relationship between input and target.

3.1.1 Predictions are not targets

For one observed pair (x_i, y_i) , the model produces

$$\hat{y}_i = wx_i + b.$$

The difference between the observed target and the prediction is the *residual*. We shall use

$$r_i = y_i - \hat{y}_i. \quad (3.2)$$

A positive residual says that the model predicted too low; a negative residual says that it predicted too high. On a scatter plot, the residual can be drawn as the vertical displacement from the prediction line to the observed point.

A residual is not yet a training objective. A dataset gives us many residuals, some positive and some negative. Simply adding them is a bad idea because errors in opposite directions can cancel. We need a rule for combining them into a scalar quantity whose value reflects the kinds of mistakes we care about.

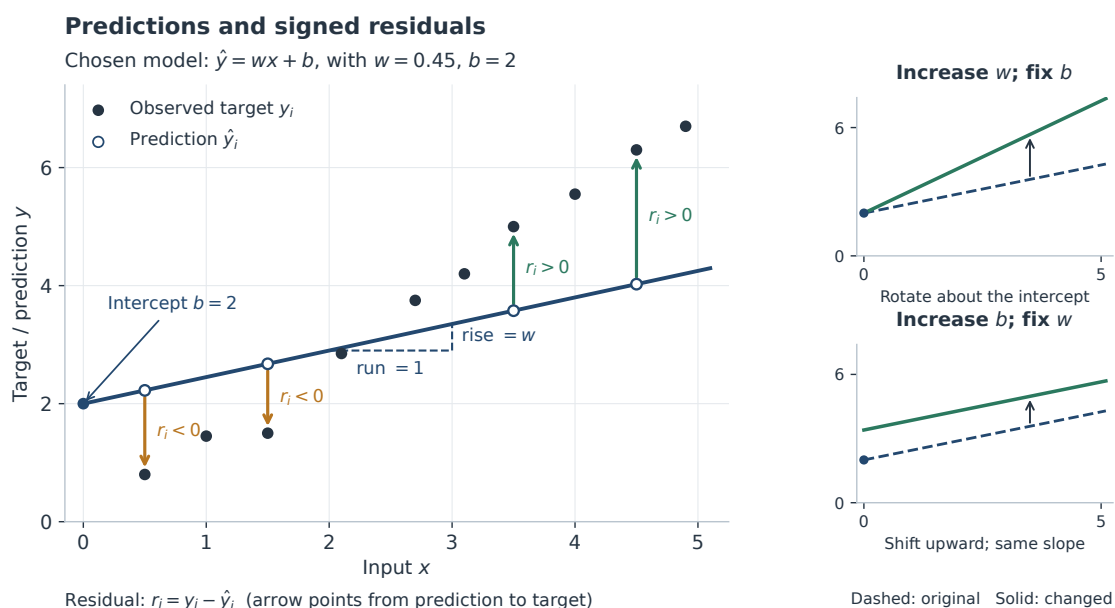


Figure 3.1: A linear model has only two parameters, so its behaviour is easy to inspect. Residuals describe individual prediction errors; a loss will combine these errors into one objective.

PyTorch: a scalar linear model

```
import torch
from torch import nn

model = nn.Linear(in_features=1, out_features=1)

# For a batch X with shape [batch, 1]:
```

```
y_hat = model(X) #  $y_{\text{hat}} = X @ W.T + b$ 

# The two learned objects are the weight and bias:
print(model.weight.shape) # [1, 1]
print(model.bias.shape) # [1]
```

PyTorch stores the weight in a matrix even when there is only one input and one output. This is a clue about where we are heading: the scalar line is one member of a family of linear transformations.

3.2 From a line to a linear layer

Real examples usually contain more than one measured quantity. Suppose an example is represented by d features,

$$\mathbf{x} = (x_1, x_2, \dots, x_d)^\top.$$

A scalar linear prediction is a weighted sum

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_dx_d + b. \quad (3.3)$$

Using vector notation,

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b. \quad (3.4)$$

The dot product therefore has a very concrete interpretation: every feature contributes to the prediction in proportion to a learned weight.

If w_j is large and positive, increasing feature x_j tends to raise the prediction. If it is large and negative, increasing the feature tends to lower it. If it is near zero, the model is comparatively insensitive to that feature, at least within this linear description.

3.2.1 Several outputs at once

Now suppose the model produces m numerical outputs. We could write m separate weighted sums, but matrix notation says the same thing compactly:

$$\hat{\mathbf{y}} = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad (3.5)$$

where

$$\mathbf{W} \in \mathbb{R}^{m \times d}, \quad \mathbf{b} \in \mathbb{R}^m, \quad \hat{\mathbf{y}} \in \mathbb{R}^m.$$

Each row of $\mathbf{W}$ contains the weights for one output. Matrix multiplication is not introduced here merely to shorten notation. It is the basic operation by which neural networks mix coordinates into new coordinates.

A linear map can stretch some directions, shrink others, rotate or reflect coordinate systems, combine several measurements, or discard information by projecting a high-dimensional vector into fewer dimensions. These geometric interpretations become increasingly important once hidden layers begin to construct internal representations.

There is one small terminological subtlety. Mathematically, $\mathbf{W}\mathbf{x}$ is a *linear* map, whereas $\mathbf{W}\mathbf{x} + \mathbf{b}$ is more precisely an *affine* map because of the translation $\mathbf{b}$. In machine-learning practice, ‘Linear’ layer usually refers to the affine form including a bias. We shall follow that convention unless the distinction matters.

3.2.2 A batch is one larger matrix multiplication

If a batch contains B examples and we store them as rows of a matrix

$$\mathbf{X} \in \mathbb{R}^{B \times d},$$

then all predictions can be computed together:

$$\hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}. \quad (3.6)$$

The bias is added to every row. This is one reason matrix multiplication is so central to deep learning: modern hardware can execute a large collection of weighted sums extremely efficiently as one tensor operation.

Check your understanding

A model has 5 input features and 3 numerical outputs. What are the shapes of $\mathbf{x}$, $\mathbf{W}$, $\mathbf{b}$ and $\hat{\mathbf{y}}$? If a batch contains 64 examples, what is the shape of $\mathbf{X}$ and of the batch of predictions?

3.3 The loss is a specification of error

The model produces predictions. Training requires one more ingredient: a scalar objective that says which predictions are preferable.

For regression, two common choices are mean squared error (MSE),

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad (3.7)$$

and mean absolute error (MAE), also called L1 error,

$$\mathcal{L}_{\text{L1}} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|. \quad (3.8)$$

Both are sensible ways to penalise wrong numerical predictions. They do not, however, express the same preference.

Squaring makes large residuals disproportionately expensive. An error of size 4 contributes sixteen times as much squared error as an error of size 1. Under L1 the same errors contribute in the ratio four to one. MSE therefore reacts strongly to extreme errors; L1 is less sensitive to their magnitude.

This difference is not an implementation detail. It changes the answer that is optimal.

3.3.1 What if the model is forced to predict a constant?

Imagine that the model receives no useful input and must predict the same scalar c for every target. Under MSE it chooses

$$c = \frac{1}{n} \sum_i y_i,$$

the sample mean. Under L1, any sample median is an optimum.

This is a remarkably useful fact because it reveals what each loss is asking the model to do. MSE says, in effect, “place the prediction at the centre that minimises squared distance.”

L1 says “place it at a centre that minimises absolute distance.” Different notions of distance imply different notions of the best central answer.

The Week 2 practical makes this visible first with scalar targets and then with images. That experiment is worth remembering because the same mathematics survives when the model becomes very complicated. If the inputs do not tell the network which of several possible targets will occur, the loss still pulls the output toward whatever summary minimises the expected penalty.

Application to the architecture

The running StoryReasoning system predicts a fifth movie frame from four preceding frame–description pairs. Roughly 85% of the five-frame windows contain a shot cut. On those windows, the exact target pixels are not determined by the preceding images. A pixelwise L1 objective nevertheless demands one deterministic image. Its safest solution is therefore drawn toward a per-pixel median of plausible futures. In the repository, a constant median image obtains an overall L1 of about 0.151, while simply copying the last real frame scores about 0.170. On shot cuts, a visually real frame can lose to the bland constant.

Nothing has gone wrong with optimisation. The optimiser is following the objective faithfully. The deeper problem is that the objective rewards a statistical summary when we wanted a plausible future. This is the Week 2 principle in its most consequential form: *loss is specification, not merely measurement.*

3.3.2 Training objective and evaluation metric need not be identical

A model is often trained with one quantity and judged with several others. We may optimise MSE but report MAE as well. A retrieval model may train with a contrastive loss but be evaluated using top-10 retrieval accuracy. A classifier may train with cross-entropy but be judged by accuracy, recall, calibration and cost-sensitive measures.

This is legitimate, but it requires discipline. The training loss determines the local signal that changes the parameters. An evaluation metric tells us whether the resulting behaviour serves our purpose. If the two disagree, improving the training loss may not improve the quantity we actually care about.

A useful question is therefore not simply

“Which loss is standard for this model?”

but rather

“What behaviour becomes optimal under this loss, given the uncertainty in the target?”

That question will return when we study classification, autoencoders, language modelling and probabilistic generation.

Optional mathematics — why MSE chooses the mean

Suppose a constant predictor outputs c for targets $y_1, \dots, y_n$. Its squared-error objective is

$$\mathcal{L}(c) = \frac{1}{n} \sum_{i=1}^n (y_i - c)^2.$$

Differentiate with respect to c , set the derivative to zero, and show that

$$c = \frac{1}{n} \sum_i y_i.$$

Then sketch why the L1 objective changes slope whenever c crosses an observed target and is minimised at a median. You do not need a formal proof of the L1 result; a sorted list of five targets is enough to see the mechanism.

3.4 A derivative is local sensitivity

We now have parameters and a loss. The next question is mechanical: if the loss is too large, which way should a parameter move?

Let $\mathcal{L}(w)$ denote the loss when all parameters except w are held fixed. The derivative

$$\frac{d\mathcal{L}}{dw}$$

measures local sensitivity: how much the loss changes for a very small change in w .

If the derivative is positive, increasing w slightly tends to increase the loss, so decreasing w is locally promising. If the derivative is negative, increasing w tends to reduce the loss. If it is close to zero, the loss is locally insensitive to small changes in that parameter.

The derivative does not reveal the entire landscape. It is a local measurement, rather like standing on a hillside and measuring the slope beneath one foot. Yet local information is enough to construct an iterative procedure: measure the slope, move downhill a little, measure again.

3.4.1 Finite differences: a slope without symbolic calculus

Before differentiating an equation, we can estimate a derivative experimentally. Choose a small ϵ and compare the loss on either side of the current value:

$$\frac{d\mathcal{L}}{dw} \approx \frac{\mathcal{L}(w + \epsilon) - \mathcal{L}(w - \epsilon)}{2\epsilon}. \quad (3.9)$$

This centred finite difference is not how large neural networks are normally trained, but it is conceptually valuable. It shows that a derivative is not an abstract symbol inserted by convention. It summarises an observable fact: perturbing the parameter in one direction changes the loss differently from perturbing it in the other.

Finite differences also provide a useful debugging tool. For a small model, an analytically or automatically computed gradient can be compared with a numerical finite-difference estimate. If they disagree badly, one of the computations is probably wrong.

3.4.2 Many parameters give a gradient

Our line has two parameters, so the loss is a function $\mathcal{L}(w, b)$. We collect the two partial derivatives into a vector:

$$\nabla \mathcal{L} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial w} \\ \frac{\partial \mathcal{L}}{\partial b} \end{bmatrix}. \quad (3.10)$$

This vector is the *gradient*. It points in the direction of steepest local increase of the loss. The opposite vector, $-\nabla \mathcal{L}$, points in the direction of steepest local decrease.

For a network with millions or billions of parameters the same definition applies. The gradient is simply much longer. Each component asks: if this parameter were nudged while the others were momentarily fixed, how would the loss respond?

Same loss surface, different learning rates

$$\mathcal{L}(w, b) = \frac{1}{n} \sum_i (wx_i + b - y_i)^2 \quad | \quad \text{Same start and 8 updates in every panel}$$

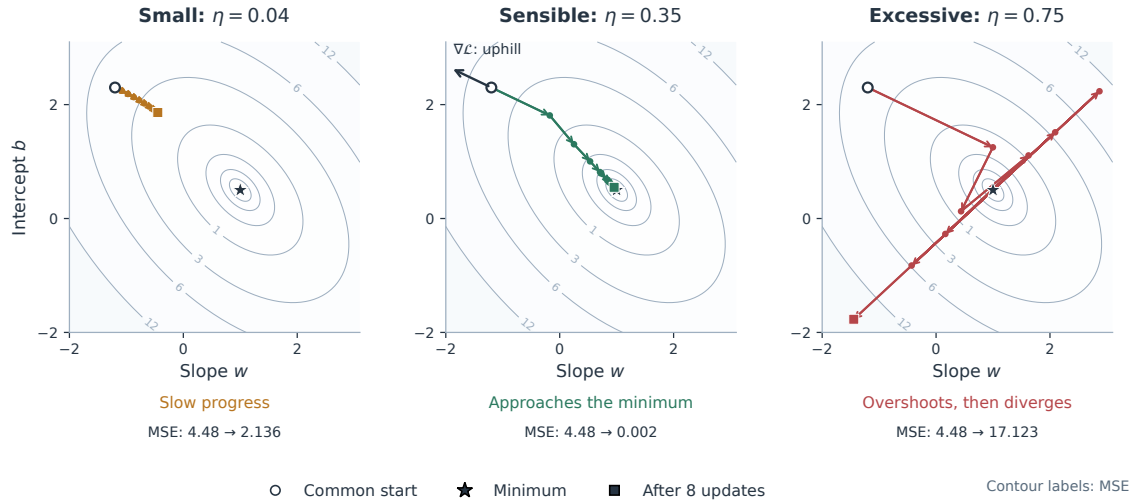


Figure 3.2: Training as motion in parameter space. The gradient describes local uphill sensitivity; gradient descent moves in the opposite direction. The learning rate determines how far each update travels.

Check your understanding

At some point in training, $\partial\mathcal{L}/\partial w = -3$ and $\partial\mathcal{L}/\partial b = 0.5$. Without calculating a new loss, what local change in w and b would gradient descent favour? Which parameter currently has the larger magnitude of local sensitivity?

3.5 Gradient descent: turning sensitivity into an update

The simplest gradient-descent update is

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}, \quad (3.11)$$

where θ denotes all model parameters and $\eta > 0$ is the *learning rate*.

For our two-parameter line this means

$$w_{t+1} = w_t - \eta \frac{\partial \mathcal{L}}{\partial w}, \quad (3.12)$$

$$b_{t+1} = b_t - \eta \frac{\partial \mathcal{L}}{\partial b}. \quad (3.13)$$

The gradient chooses a direction; the learning rate chooses a step size.

This separation matters. A perfectly correct gradient does not guarantee successful training. If η is extremely small, the learner moves in sensible directions but makes painfully little progress. If η is too large, each update can leap across a low-loss region and land higher on the other side. Repeated overshooting may cause oscillation or divergence.

3.5.1 Why training is iterative

One might ask why, having found the downhill direction, we do not simply continue in a straight line until reaching the minimum. The answer is that the gradient itself changes as the parameters move. A direction that is downhill at the current point need not remain downhill far away.

Gradient descent therefore alternates measurement and intervention:

1. compute predictions using the current parameters;
2. compute a loss;
3. compute the gradient of that loss at the current parameters;
4. update the parameters;
5. repeat from the new position.

Learning is a trajectory, not one calculation.

This distinction will become more important in neural networks. Two models can begin from different random parameters, see examples in a different order, or use different learning rates and end at different solutions even when their architecture and training data are identical.

3.5.2 The gradient of MSE for a line

For the model $\hat{y}_i = wx_i + b$, define the prediction error

$$e_i = \hat{y}_i - y_i.$$

Then

$$\mathcal{L} = \frac{1}{n} \sum_i e_i^2.$$

The derivatives are

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{2}{n} \sum_i e_i x_i, \tag{3.14}$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{2}{n} \sum_i e_i. \tag{3.15}$$

These formulas are worth reading rather than merely accepting. The bias gradient is proportional to the average signed prediction error. If predictions are systematically too high, the update tends to lower the bias; if systematically too low, it tends to raise it.

The weight gradient also multiplies each error by x_i . An error observed at a positive input pushes the slope differently from the same error observed at a negative input. The gradient therefore combines *what went wrong* with *where in the input space it went wrong*.

A neural network will use the same principle at a vastly larger scale. Backpropagation, introduced later, is an efficient method for obtaining these sensitivities through compositions of many operations.

Optional derivation — the two gradients

Starting from

$$\mathcal{L}(w, b) = \frac{1}{n} \sum_i (wx_i + b - y_i)^2,$$

use the chain rule to derive [equation \(3.15\)](#). Then evaluate both derivatives for the three points

$$(0, 1), \quad (1, 3), \quad (2, 5)$$

when $w = 0$ and $b = 0$. Before applying an update, predict whether both parameters should increase or decrease.

3.6 The practical meaning of a training loop

The equations can make training look more ceremonial than it is. A training loop is simply the repeated execution of the mechanism we have assembled.

For a batch of examples:

forward prediction $\rightarrow$ loss $\rightarrow$ gradients $\rightarrow$ update

Then the process begins again with the updated parameters.

The term *forward pass* refers to the computations from inputs to predictions, and usually onward to the loss. In our line, the forward pass is just $wx + b$. In a deep network it may involve hundreds of tensor operations. The logic is the same.

PyTorch: one regression training step

```
model = nn.Linear(4, 1)
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

optimizer.zero_grad() # discard gradients from the previous step
y_hat = model(X_batch) # forward pass
loss = loss_fn(y_hat, y_batch)
loss.backward() # ask autograd for parameter gradients
optimizer.step() # theta <- theta - lr * gradient
```

This is already recognisably the code used to train much larger networks. Two details deserve restraint at this stage. First, ‘`loss.backward()`’ does not mean that we have introduced a new learning rule; it asks PyTorch to compute the same derivatives represented by $\nabla \mathcal{L}$. How those derivatives are propagated efficiently through a multilayer computation is the subject of backpropagation in Weeks 3–4. Second, ‘`optimizer.step()`’ hides the parameter-update arithmetic, not the conceptual mechanism. For plain SGD it implements [equation \(3.11\)](#).

3.7 Why not use the entire dataset for every update?

So far the loss and gradient have been written as averages over all n training examples. This is *full-batch gradient descent*. If the dataset is small, that is perfectly reasonable. If it contains millions of examples and the model is expensive, recomputing the exact full-dataset gradient before every parameter update becomes wasteful.

The alternative is to estimate the gradient using only part of the training set.

3.7.1 Stochastic and mini-batch gradients

At one extreme, *stochastic gradient descent* may use a single randomly chosen example for an update. More commonly in deep learning, a *mini-batch* of B examples is used:

$$\hat{\nabla} \mathcal{L}_B = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i, \quad (3.16)$$

where ℓ_i is the loss contribution from one example and $\mathcal{B}$ is the selected batch.

The hat is useful conceptually: the mini-batch gradient is an estimate of the gradient we would obtain from the whole training set. Different batches contain different examples, so their estimates are not identical.

A one-example gradient can be very noisy. One unusual example may point in a direction quite different from the full-data gradient. Averaging over more examples reduces that variation. Thus batch size controls a statistical trade-off:

- small batches give cheap but noisy gradient estimates and many updates per pass through the data;
- large batches give more stable estimates but cost more per update and produce fewer updates per pass;
- the full batch gives the exact gradient of the empirical training loss but may be unnecessarily expensive.

Noise is therefore not evidence that SGD is malfunctioning. It is the expected consequence of estimating a population-of-training-examples quantity from a sample of those examples. Week 1's sampling idea has reappeared inside the optimiser.

ACTION — create this figure

Make a two-part figure. Left: in a $(\partial L / \partial w, \partial L / \partial b)$ plane, plot a fixed full-dataset gradient and clouds of gradient estimates from batch size 1, a medium mini-batch, and a large mini-batch; smaller batches should have visibly larger spread. Right: show three parameter trajectories toward the same low-loss region, with full-batch smooth, mini-batch moderately noisy, and single-example SGD jagged.

Figure 3.3: Mini-batch gradients are noisy estimates of the full-data gradient. Larger batches reduce variation, but a noisy trajectory can still make useful progress.

3.7.2 Batch, update and epoch are different counters

These terms are easy to blur together.

A *batch* is the set of examples used to compute one gradient estimate. An *update* is one parameter change. An *epoch* is one pass through the training dataset.

If a dataset contains $N = 10,000$ examples and the batch size is $B = 100$, then one epoch contains approximately

$$\frac{N}{B} = 100$$

updates. With $B = 10$, the same epoch contains about 1000 updates. Both runs have seen approximately the same number of examples during the epoch, but they have changed their parameters a very different number of times.

This is why statements such as “model A learned faster because it converged in fewer epochs” can be misleading when batch sizes differ. Computational work, examples processed and parameter updates are related quantities, but they are not identical.

3.7.3 Why shuffle?

Suppose the dataset is stored in a meaningful order: perhaps examples are sorted by target value, grouped by class, recorded chronologically or gathered from one source before another. If SGD visits them in that fixed order, successive gradients can be systematically biased for long stretches of an epoch.

Shuffling before each epoch mixes those local regularities. It does not change which examples exist in the dataset. It changes the order in which the changing model encounters them.

Order matters because learning is iterative. After the first batch, the parameters have changed; therefore the second batch is evaluated by a slightly different model. Reversing the batches does not, in general, reverse the updates and return us to the same point.

This connects directly to the Week 1 perceptron practical, where changing the order of examples could lead to a different separating boundary. The learning rule interacts with the path through the data.

Check your understanding

A dataset has 12,000 examples. Compare batch sizes 1, 32, 256 and 12,000. Approximately how many updates occur in one epoch for each? Which setting should give the noisiest individual gradient estimates? Which gives the exact empirical gradient before each update?

3.8 When decreasing loss is not enough

Gradient descent answers a precise question: how can we change parameters to reduce the chosen objective? It does not answer a different question: does the resulting predictor use the information we intended it to use?

This distinction is easy to expose with a linear model. Suppose the true data contain a relationship between x and y , but we randomly shuffle the inputs while leaving the targets in place. The set of x values is unchanged and the set of y values is unchanged. What has been destroyed is their pairing.

Under MSE, a model with a bias can still improve. Its weight may shrink toward zero while its bias approaches the mean target. The result is approximately

$$\hat{y} \approx \bar{y}.$$

The learner has found a sensible constant baseline. Its loss may be far lower than at random initialisation. Yet it has not recovered the intended input–target relationship because that information is no longer present.

This example introduces a habit that will become central later: when a model is supposed to use a particular signal, perturb or shuffle that signal and see what changes. A falling training loss shows that optimisation is doing something. It does not, by itself, establish what information the model has used.

3.8.1 Baselines return

Week 1 introduced the question “better than what?” In regression, an immediate baseline under MSE is the constant training-target mean. Under L1, the median is a natural constant baseline.

If a sophisticated predictor barely beats such a constant, one of several things may be true: the inputs are weakly informative, the model cannot exploit them, optimisation has failed, or the evaluation sample is noisy. The baseline does not diagnose the cause, but it prevents us from mistaking mere reduction of error for useful learning.

The Week 2 shuffled-input practical adds another diagnostic: *prediction spread*. If predictions barely change as inputs change, the model may have collapsed toward a constant even though the loss looks respectable.

Key idea

Optimisation answers “did the parameters move in a direction favoured by the loss?” A diagnostic asks “did the model acquire the behaviour we intended?” These are not the same question.

3.9 What stochasticity changes — and what it does not

The word *stochastic* can make SGD sound as if it has abandoned the objective and begun wandering randomly. That is the wrong picture.

Each mini-batch gradient is still computed from the same model and the same loss. Randomness enters through which examples happen to contribute to a particular update. Across many representative batches, the gradient estimates fluctuate around the full-data direction. The path is noisy, but the noise is structured by the objective.

This helps explain an initially surprising phenomenon: the full-dataset loss can increase after one perfectly valid mini-batch update. The update was chosen to reduce the loss of the current batch locally, not necessarily the loss of every training example simultaneously. Another batch may pull in a somewhat different direction.

Over many updates, useful progress can emerge from these imperfect local estimates. In large-scale deep learning this is essential. Waiting for an exact full-corpus gradient after every parameter change would be computationally prohibitive.

There is also a subtler benefit, which we will revisit in later weeks: the noise in SGD changes which optimisation trajectories are easy to follow. For now we need only the modest claim supported by the practical experiment: a gradient estimate need not be exact to be useful.

3.10 A complete linear-learning system

We can now write the entire system introduced abstractly in Week 1.

For a batch of inputs $\mathbf{X}$ and targets $\mathbf{Y}$:

1. **Model.** Compute predictions

$$\hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}.$$

2. **Residuals.** Compare predictions with targets.

3. **Loss.** Reduce those discrepancies to a scalar objective, for example MSE.

4. **Gradient.** Measure the local sensitivity of that objective to every adjustable parameter.
5. **Update.** Move the parameters a learning-rate-scaled step opposite the gradient.
6. **Repeat.** Use another batch and continue until the chosen stopping condition is reached.

There is no magic transition between regression and deep learning. Neural networks elaborate the model in Step 1 and make Step 4 computationally more demanding. They also introduce new questions about representations, non-linearities, generalisation and the interaction of many layers. But the skeleton remains.

Optional coding — write the loop without an optimiser

Create a small synthetic regression dataset with four input features and one target. Store the weights in a PyTorch tensor $\mathbf{w}$ and the bias in a scalar tensor. Compute predictions using matrix multiplication, calculate MSE, and update $\mathbf{w}$ and b using gradients you derive or obtain with ‘torch.autograd.grad’. Do not use ‘nn.Linear’ or an optimizer.

Then compare your predictions with an equivalent ‘nn.Linear(4, 1)’ model whose parameters are initialised to the same values. The two forward computations should agree. The purpose is not speed; it is to see exactly what the convenience class packages together.

3.11 What to carry into Week 3

A linear layer can mix and reweight information, but it has an important limitation. If we compose one affine map with another affine map and insert no non-linearity between them, the result is still just another affine map. Depth alone does not yet buy us a richer class of functions.

That observation sets up the next question. How can a model construct a decision boundary or transformation that one linear map cannot express? The answer begins by placing a non-linear operation after the weighted sum and then stacking such transformations.

Before moving on, the central ideas of this chapter should be secure:

- A linear model is a weighted transformation of inputs plus a bias; matrix multiplication expresses many such weighted sums at once.
- Residuals describe individual errors; a loss specifies how those errors are valued and combined.
- MSE and L1 encode different preferences. A loss is therefore part of the task specification, not merely a score recorded after training.
- A derivative measures local sensitivity. A gradient collects those sensitivities for all parameters.
- Gradient descent moves parameters opposite the gradient; the learning rate controls the step size.
- Mini-batch gradients are noisy estimates of the full-data gradient. Batch size, updates and epochs are related but distinct.
- A falling loss is evidence of optimisation, not proof that the model uses the intended input or has learned the intended behaviour.

The machinery is still small enough to draw. From next week onward, we shall begin to hide some of it inside layers. The discipline should remain the same: whenever a network behaves strangely, return to the chain from prediction to loss to gradient to update and ask which link is producing the observed behaviour.

Chapter 4

From linear rules to neural networks

In Week 2, learning altered the parameters of a linear model until its predictions reduced a chosen loss. That mechanism was already surprisingly general. It gave us parameters, a forward computation, an objective, gradients and iterative updates. Yet the model itself remained severely restricted: however long we trained it, a linear transformation could only express a linear relationship.

This week asks what must change if the structure of the problem is not linear.

The answer is not simply “add more parameters”. Two linear transformations placed one after another are still equivalent to one linear transformation. Something qualitatively different must occur between them. Neural networks obtain that difference by alternating learned linear projections with fixed nonlinear functions. Once we do this, an intermediate layer is no longer merely another calculation on the way to the output. It can become a new representation of the problem.

That shift is the important one. We shall begin with a single artificial neuron, then ask what decision it can make, why one such decision can be insufficient, how hidden layers transform a geometry, and finally how information about an error can travel backward through the resulting composition.

Key idea

A multilayer network is useful not merely because it performs more arithmetic, but because nonlinear hidden layers can transform a problem into a representation in which a simple final decision becomes possible.

4.1 From a linear model to an artificial neuron

Recall the linear prediction from Week 2. With an input vector $\mathbf{x} \in \mathbb{R}^d$, weights $\mathbf{w} \in \mathbb{R}^d$ and bias b , we compute

$$z = \mathbf{w}^\top \mathbf{x} + b. \quad (4.1)$$

For regression, z may itself be the prediction. A neuron inserts one additional operation:

$$h = \phi(z) = \phi(\mathbf{w}^\top \mathbf{x} + b), \quad (4.2)$$

where ϕ is an *activation function*.

It is useful to give the two quantities different names. The weighted sum z is the *pre-activation*. The value h after applying ϕ is the *activation*. Keeping these separate prevents

a common conceptual blur: the weights determine a learned linear projection, while the activation changes how the unit responds to that projection.

Historically, the perceptron used a threshold-like decision: one side of the threshold produced one response and the other side produced another (Rosenblatt 1958). Modern networks normally use differentiable or almost-everywhere differentiable activations instead, but the basic decomposition remains visible:

weighted evidence $\longrightarrow$ nonlinear response.

4.1.1 A score before a class

Suppose our task has two classes. A linear classifier can first compute a real-valued score

$$z = \mathbf{w}^\top \mathbf{x} + b.$$

A simple rule is then

$$\hat{y} = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

The value z is often called a *logit* when it is the raw output used by a classification model. For the moment, treat a logit simply as a score before the final class decision. Week 4 will examine how logits are connected to probabilities and classification losses.

The score contains more information than the final label. A point with $z = 8$ and a point with $z = 0.01$ are placed on the same side of the boundary, but they are not equally far from it. Neural-network training usually operates on such continuous scores rather than on a hard class decision, because continuous quantities can carry graded information about change.

4.2 Classification is geometry

Equation (4.1) has a geometric interpretation. The points satisfying

$$\mathbf{w}^\top \mathbf{x} + b = 0 \tag{4.3}$$

form a decision boundary. In two dimensions this boundary is a line. In three dimensions it is a plane. In higher dimensions it is a hyperplane.

The weight vector $\mathbf{w}$ is perpendicular to that hyperplane. Changing its direction rotates the boundary; changing the bias shifts the boundary without changing its orientation. The sign of z tells us on which side of the boundary a point lies.

This gives a precise description of what a linear classifier can do: it can divide the input space into two half-spaces with one flat boundary.

That is sometimes exactly what the problem requires. If two groups of points are well separated by a line, there is no merit in constructing a complicated nonlinear model merely because one is available. But other arrangements cannot be separated in this way, however well the linear model is optimised.

4.2.1 Optimisation failure or representational failure?

This distinction is worth making early. Suppose a classifier performs badly. There are at least two very different explanations.

First, the model class might contain a good solution, but our learning procedure has not found it. This is an *optimisation problem*. Second, the model class may simply be incapable of expressing the required rule. This is a *representational problem*.

ACTION — create this figure

Create a two-panel 2D diagram. Left: two classes that are cleanly separated by one straight decision boundary, with the normal weight vector w drawn perpendicular to the line. Right: an XOR-style four-corner arrangement with diagonally opposite corners sharing a class. Show several candidate straight lines, each necessarily leaving at least one region on the wrong side.

Figure 4.1: A linear classifier can separate two half-spaces with one flat boundary. Some class arrangements are not linearly separable, so no amount of optimisation can make one line solve them perfectly.

The XOR-style arrangement in [figure 4.1](#) supplies the cleanest example of the second case. Searching longer, reducing the learning rate or collecting the same kind of data more carefully will not make one straight boundary perform a task that requires a non-straight division of the space.

This is the phenomenon explored directly in the first Week 3 practical. The useful lesson is not the famous XOR pattern itself. It is the diagnostic question it teaches us to ask:

Is the learner failing to find a good member of its model class, or is the model class itself too restricted?

Later, with large networks, the answer will be much harder to see. Here we can draw it.

4.3 Why more linear layers do not solve the problem

A natural first attempt is to add depth. Perhaps one linear layer is too simple, but two linear layers might be richer.

Let

$$h = W_1 x + b_1$$

and then

$$y = W_2 h + b_2.$$

Substituting the first equation into the second gives

$$y = W_2(W_1 x + b_1) + b_2 \tag{4.4}$$

$$= (W_2 W_1) x + (W_2 b_1 + b_2). \tag{4.5}$$

If we define

$$W_\star = W_2 W_1, \quad b_\star = W_2 b_1 + b_2,$$

then the whole two-layer computation is simply

$$y = W_\star x + b_\star.$$

It is another affine transformation.

The same argument applies to any finite stack of affine layers with nothing nonlinear between them. Such a stack may contain many parameters, but its input–output map can still be collapsed into one affine transformation.

Key idea

Depth is not automatically nonlinearity. A sequence of linear or affine layers without nonlinear activations is still only one linear or affine transformation in disguise.

This algebra explains an empirical result students see in the practicals: if the hidden activation in a small MLP is replaced by the identity function, the network loses the representational advantage that made the hidden layer useful.

Optional mathematics — collapse three affine layers

Suppose

$$\mathbf{h}_1 = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1, \quad \mathbf{h}_2 = \mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2,$$

and

$$\mathbf{y} = \mathbf{W}_3 \mathbf{h}_2 + \mathbf{b}_3.$$

Derive a single matrix $\mathbf{W}_\star$ and bias $\mathbf{b}_\star$ such that

$$\mathbf{y} = \mathbf{W}_\star \mathbf{x} + \mathbf{b}_\star.$$

Then explain in one sentence why adding an identity activation between the layers changes nothing.

4.4 The nonlinearity

The missing ingredient is a nonlinear function between learned projections. For one hidden layer we write

$$\mathbf{h} = \phi(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \tag{4.6}$$

where ϕ is applied element by element. An output layer may then compute

$$\mathbf{s} = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2. \tag{4.7}$$

The composition can no longer, in general, be reduced to one matrix multiplication because ϕ interrupts the algebraic collapse.

Several activations recur in the history and practice of neural networks.

4.4.1 Step activation

A threshold or step activation changes abruptly at a chosen value, often zero:

$$\phi(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

It gives a clear picture of a neuron as a switch and is useful for understanding early perceptrons. Its drawback for gradient-based learning is equally clear: away from the discontinuity its ordinary derivative is zero, while at the discontinuity it is not differentiable in the ordinary sense. A small change in the weights usually causes no small change at all in the output.

4.4.2 Sigmoid

The logistic sigmoid is

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

It maps all real inputs smoothly into $(0,1)$. Near zero, changing z changes the output appreciably. Far into either tail, the curve flattens and becomes relatively insensitive to further changes.

Sigmoids were historically important in neural networks and remain useful in specific output mechanisms. Hidden layers in large modern networks usually use other activations because saturation can make gradients small over broad ranges.

4.4.3 Hyperbolic tangent

The hyperbolic tangent

$$\tanh(z)$$

is also smooth and saturating, but its range is $(-1,1)$. It is centred around zero, which often makes its hidden activations easier to interpret geometrically than a sigmoid's strictly positive outputs. Like the sigmoid, it becomes locally insensitive for sufficiently large positive or negative inputs.

4.4.4 ReLU

The rectified linear unit is

$$\text{ReLU}(z) = \max(0, z). \quad (4.8)$$

For positive inputs it is simply the identity; for negative inputs it outputs zero. Its derivative is therefore

$$\frac{d}{dz} \text{ReLU}(z) = \begin{cases} 1, & z > 0, \\ 0, & z < 0, \end{cases}$$

with a convention required exactly at zero.

ReLU is especially useful for geometric intuition. Each hidden unit divides its input space according to whether its pre-activation is positive or negative. On one side the unit is active and contributes a linear response; on the other side its contribution is zero. Several such units divide the input space into regions. Within each fixed pattern of active and inactive ReLUs, the whole network behaves linearly; crossing a ReLU boundary changes which linear rule is in force.

Thus a ReLU network can be *piecewise linear* without being globally linear.

A modern smooth activation such as GELU plays a similar architectural role: it prevents stacked layers from collapsing into one global linear transformation while providing a smooth response. We need not dwell on its formula here. The important question is what the nonlinearity permits the composition to represent.

Check your understanding

A network contains five **Linear** layers but no activation functions between them. Has it necessarily become capable of representing a curved decision boundary? Explain without appealing to the number of parameters.

ACTION — create this figure

Create a compact activation-function figure with four aligned plots for step, sigmoid, tanh and ReLU over z from roughly -5 to 5. Beneath or beside each curve show a faint derivative/sensitivity curve. Emphasise saturation for sigmoid/tanh and the two constant-slope regions of ReLU.

Figure 4.2: Common activation functions transform the same pre-activation in different ways. Their local sensitivities matter because gradients through a network are products of such local sensitivities.

4.5 From neurons to a multilayer perceptron

An *MLP*, or multilayer perceptron, alternates affine transformations and nonlinear activations. A simple one-hidden-layer network can be written

$$\mathbf{z}_1 = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1, \quad (4.9)$$

$$\mathbf{h}_1 = \phi(\mathbf{z}_1), \quad (4.10)$$

$$\mathbf{s} = \mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2. \quad (4.11)$$

Here $\mathbf{h}_1$ is the hidden representation and $\mathbf{s}$ is the output score or collection of scores.

Three words are useful.

Input layer refers to the supplied features. It does not usually perform a learned transformation by itself. A *hidden layer* produces an intermediate representation that is neither the raw input nor the final output. The *output layer* maps the final hidden representation to the quantities required by the task.

The *width* of a hidden layer is the number of units in it. The *depth* is the number of successive learned transformations, with conventions differing slightly across texts about whether to count the output layer. In these notes, what matters is the structural idea: width provides several features at one stage; depth composes transformations across stages.

4.5.1 A hidden layer is a change of coordinates

The most productive mental model is not that each hidden neuron is a miniature classifier. Instead, the hidden layer constructs new coordinates.

Suppose an input point $\mathbf{x} \in \mathbb{R}^2$ is transformed into

$$\mathbf{h} = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix}.$$

The same example can now be plotted in two spaces: its original coordinates (x_1, x_2) , and its hidden coordinates (h_1, h_2) . The class arrangement may be tangled in the first space and simple in the second.

The output layer need not produce a complicated boundary in hidden space. It may still be a linear classifier there:

$$s = \mathbf{v}^\top \mathbf{h} + c.$$

What looks nonlinear from the original input coordinates can therefore arise from a simple linear decision applied after a nonlinear transformation.

This is a central idea of deep learning: *learn a representation in which the desired computation becomes easier.*

ACTION — create this figure

Create a three-stage diagram of a small 2D classification problem. Left: two interlocking or two-moons classes in input space with no satisfactory straight separator. Middle: arrows through a nonlinear hidden layer transforming points into a new 2D hidden space. Right: the transformed classes are more nearly linearly separable, with one straight output boundary. Use the same individual points across panels so the viewer can track that the examples did not change, only their coordinates.

Figure 4.3: A hidden layer can reorganise the geometry of a problem. The output layer may remain linear in hidden space even though the corresponding decision boundary is nonlinear in the original input space.

4.5.2 Depth as repeated re-representation

Adding another hidden layer gives

$$\mathbf{h}_2 = \phi_2(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2), \quad \mathbf{h}_1 = \phi_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1).$$

The second layer does not receive the original problem directly. It receives the representation constructed by the first layer. Depth therefore permits a sequence of reorganisations.

This description is more useful than saying that deeper networks are simply “more powerful”. The practical question is what computation a representation makes easy for the next layer. In vision, later layers may eventually make object identity easier to read out; in language, representations may make syntactic or semantic distinctions easier to recover; in our tiny geometric examples, a hidden layer can make classes linearly separable.

Not every deeper network will learn a useful representation. Capacity is only possibility, not a guarantee. Optimisation, data and objective still determine which transformation is actually found.

PyTorch: the notation of a tiny MLP

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(2, 4), # z1 = W1 x + b1
    nn.ReLU(), # h1 = ReLU(z1)
    nn.Linear(4, 1), # s = W2 h1 + b2
)

# A batch with B examples has shape [B, 2].
# The raw output has shape [B, 1] and can be treated as a logit.
```

The code is compact because PyTorch stores the matrices and biases inside each `Linear` module. Conceptually, however, nothing has disappeared: the network still implements the equations above.

Optional coding — make the matrices visible

Create a `Linear(2, 3)` layer and one input vector with two features. Print the layer's weight matrix and bias. Compute the layer output once with PyTorch and once manually using matrix multiplication. Then apply `torch.relu` manually and check that it matches an equivalent `Sequential` model.

The purpose is not to train a network. It is to verify that the compact library notation is exactly the matrix notation used in the chapter.

4.6 ReLU networks divide space into regions

The hidden-representation view can be made still more concrete for ReLU networks.

For one hidden unit,

$$h_j = \max(0, \mathbf{w}_j^\top \mathbf{x} + b_j).$$

The equation

$$\mathbf{w}_j^\top \mathbf{x} + b_j = 0$$

defines the boundary at which that unit changes between inactive and active behaviour. With several hidden units, several such boundaries cross the input space. Each region corresponds to a particular activation pattern such as

(active, inactive, active, ...).

Within one region, every active ReLU behaves like the identity and every inactive ReLU behaves like zero. The remaining composition is affine there. Move into a neighbouring region and a different set of units becomes active, producing a different affine rule.

This explains how a network constructed from simple pieces can make a bent or faceted decision boundary. The network is not drawing one complicated curve directly. It is assembling different local linear behaviours.

The picture also clarifies the role of width. More hidden units can create more learned boundaries and therefore more possible activation patterns. Depth can repeatedly partition and transform regions produced by earlier layers. We will later see that the practical usefulness of this capacity depends heavily on whether optimisation can use it and whether the learned solution generalises.

Check your understanding

ReLU itself is linear for every positive input and constant for every negative input. Why can a network containing ReLUs nevertheless be nonlinear as a function of its original input?

4.7 The forward pass: one composed calculation

So far we have described what a network can represent. We now reconnect representation to learning.

Consider the smallest network that still contains a hidden nonlinearity:

$$z_1 = w_1x + b_1, \quad (4.12)$$

$$h = \text{ReLU}(z_1), \quad (4.13)$$

$$\hat{y} = w_2h + b_2, \quad (4.14)$$

$$\mathcal{L} = (\hat{y} - y)^2. \quad (4.15)$$

Given current values for x , y and the four parameters, we can evaluate these equations from top to bottom. This is the *forward pass*.

A forward pass does not learn anything by itself. No parameter changes merely because a prediction has been produced. It is the evaluation of the current function: compute intermediate values, obtain an output, then compute the loss.

Those intermediate values matter. To know how changing w_1 affects the final loss, we need to know how w_1 affected z_1 , how z_1 affected h , how h affected $\hat{y}$, and how $\hat{y}$ affected $\mathcal{L}$.

This dependency structure is naturally represented as a computational graph.

ACTION — create this figure

Draw the scalar computational graph for the sequence: first-layer weighted sum, ReLU hidden activation, second-layer output, and squared-error loss. Show the input and four parameters entering their operations, and the target entering the loss. Place small arrows under the graph pointing left-to-right labelled “forward: values” and right-to-left labelled “backward: sensitivities”.

Figure 4.4: A computational graph separates the forward flow of values from the backward flow of sensitivities. Early parameters affect the loss through chains of intermediate computations.

A large neural network is the same idea at greater scale. Matrix multiplications, activations, normalisation, attention and losses are all nodes in a larger differentiable computation. The graph tells us not only how to obtain the output, but how one quantity depends on another.

4.8 Local sensitivity and the chain rule

Week 2 introduced a derivative as local sensitivity. If

$$u = f(v),$$

then

$$\frac{du}{dv}$$

tells us how a small change in v changes u , locally.

A neural network is a composition of many such functions. Suppose

$$a = f(b), \quad b = g(c).$$

Then a change in c first changes b , which then changes a . The chain rule states

$$\frac{da}{dc} = \frac{da}{db} \frac{db}{dc}.$$

The total sensitivity is a product of local sensitivities along the path.

Apply this to the scalar network in [equation \(4.15\)](#). The first-layer weight w_1 affects the loss through the path

$$w_1 \rightarrow z_1 \rightarrow h \rightarrow \hat{y} \rightarrow \mathcal{L}.$$

Therefore

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial z_1} \frac{\partial z_1}{\partial w_1}. \quad (4.16)$$

Each factor is local:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = 2(\hat{y} - y), \quad (4.17)$$

$$\frac{\partial \hat{y}}{\partial h} = w_2, \quad (4.18)$$

$$\frac{\partial h}{\partial z_1} = \begin{cases} 1, & z_1 > 0, \\ 0, & z_1 < 0, \end{cases} \quad (4.19)$$

$$\frac{\partial z_1}{\partial w_1} = x. \quad (4.20)$$

No individual component needs to understand the entire network. Each contributes its own local derivative. The chain rule combines them into the sensitivity of the final loss to an early parameter.

4.8.1 What happens at an inactive ReLU?

[Equation \(4.16\)](#) contains an instructive special case. If $z_1 < 0$, then

$$\frac{\partial h}{\partial z_1} = 0.$$

The whole product for $\partial \mathcal{L} / \partial w_1$ becomes zero, no matter what the other factors are.

This is not yet a full discussion of “dead ReLUs”; that belongs to Week 4, where we consider training dynamics. For now, observe the mechanism. The same switch that makes the forward activation zero can also block the backward sensitivity through that path. Forward behaviour and backward behaviour are two views of the same local function.

Optional mathematics — follow one gradient

Let

$$x = 2, \quad y = 5, \quad w_1 = 1, \quad b_1 = 0, \quad w_2 = 2, \quad b_2 = 0.$$

For the scalar network in [equation \(4.15\)](#):

1. compute z_1 , h , $\hat{y}$ and $\mathcal{L}$;
2. compute the four local derivatives in [equation \(4.16\)](#);
3. multiply them to obtain $\partial \mathcal{L} / \partial w_1$;
4. change only b_1 so that $z_1 < 0$ and repeat.

Explain why the first-layer gradient changes so dramatically.

4.9 Backward pass and backpropagation

If we can apply the chain rule, why give the procedure a special name?

Because a network may contain millions or billions of parameters. Estimating the effect of each parameter independently would repeat an enormous amount of work. Many parameters share the same later computations. Backpropagation exploits that reuse.

Start at the loss. We know trivially that

$$\frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1.$$

From there, move backward through the graph. At each operation, combine the sensitivity arriving from later computations with that operation's local derivative. Once we have computed

$$\frac{\partial \mathcal{L}}{\partial h},$$

for example, that quantity can be reused when calculating gradients for every parameter that influenced h .

This reverse accumulation of derivatives is the *backward pass*. In neural-network language, the efficient application of this reverse-mode chain rule is called *backpropagation*. The classic modern account by Rumelhart, Hinton and Williams emphasised exactly this use of error derivatives to learn internal representations (Rumelhart et al. 1986).

Key idea

Backpropagation is not a mysterious neural-network-specific law of learning. It is an efficient algorithm for applying the chain rule through a composed computation, reusing derivatives that many parameters have in common.

4.9.1 Backpropagation is not gradient descent

Two procedures are often collapsed in casual speech:

1. **Backpropagation computes gradients.** It answers questions such as $\partial \mathcal{L} / \partial w$.
2. **An optimiser uses gradients to change parameters.** Ordinary gradient descent then performs, for example,

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}.$$

Calling `backward()` does not, by itself, make a PyTorch model learn. It populates gradient information. A separate update step changes the parameters.

This distinction will matter greatly in Week 4, where different optimisers will use the same backpropagated gradients in different ways.

4.9.2 Why the forward pass must remember things

Backpropagation requires information produced during the forward pass. The derivative of ReLU depends on whether its pre-activation was positive. The derivative of a matrix multiplication depends on the values that entered it. More generally, local derivatives often require intermediate activations.

Automatic-differentiation systems therefore record enough structure during the forward computation to reconstruct the required local derivative operations later. The resulting graph is not mystical bookkeeping hidden inside PyTorch; it is a mechanised form of the dependencies we have drawn by hand.

PyTorch: autograd computes gradients; it does not update parameters

```
import torch

x = torch.tensor(2.0)
y = torch.tensor(5.0)
w1 = torch.tensor(1.0, requires_grad=True)
b1 = torch.tensor(0.0, requires_grad=True)
w2 = torch.tensor(2.0, requires_grad=True)
b2 = torch.tensor(0.0, requires_grad=True)

z1 = w1 * x + b1
h = torch.relu(z1)
y_hat = w2 * h + b2
loss = (y_hat - y) ** 2

loss.backward()
print(w1.grad, b1.grad, w2.grad, b2.grad)

# No parameter has changed yet.
```

The fourth Week 3 practical follows essentially this computation in slow motion: first by perturbing parameters, then by multiplying local derivatives manually, and finally by comparing those results with autograd. The point is to remove any sense that `backward()` performs an unexplained trick.

4.10 Representation learning and optimisation meet

We can now state what changed between Week 2 and Week 3.

In linear regression, learning adjusted a rule whose representation was fixed: the model always acted directly on the supplied input coordinates. In an MLP, the parameters of earlier layers determine intermediate coordinates themselves. Gradient descent therefore changes not only a final predictor but also the representation on which later predictors operate.

A hidden unit can rotate, shift and threshold one projection of the input. A hidden layer combines several such responses. Training alters the boundaries at which those units become active, and therefore alters the geometry presented to the next layer. In a two-dimensional teaching example, we can literally watch the points move in hidden space as the parameters change.

The output layer is learning and the hidden layer is learning, but they are not performing identical jobs. The output layer tries to read useful information from its input representation. Earlier layers try to construct a representation from which useful information can be read.

That is why a network's internal activations deserve to be studied in their own right. Final accuracy can tell us whether a task was solved; it does not tell us how the problem was reorganised internally.

Application to the architecture

The StoryReasoning system later uses a learned visual representation: a frame encoder compresses an image into a latent vector before the rest of the system reasons about the sequence. An early version used a very narrow 16-dimensional bottleneck followed by ReLU. Many latent coordinates became identically or almost identically zero, and

the representation became nearly constant across different frames.

Week 3 gives us the mechanism needed to understand why this is dangerous. A ReLU coordinate that remains on its negative side outputs zero, and its local derivative through that path is also zero. If many coordinates contribute no variation, the next part of the system is not receiving a rich re-representation of the frame at all.

Do not treat this yet as a complete diagnosis of training failure. In Week 4 we will ask why units become inactive, how to measure collapse, and what changes in optimisation or normalisation can prevent it. The Week 3 lesson is simpler: *the hidden representation is part of what the network learns, and a failure of that representation can make later computation impossible no matter how sophisticated the later modules are.*

4.11 What a neural network is — and is not

At this point we can give a working definition without appealing to biological metaphor.

A feed-forward neural network is a parameterised composition of simple differentiable or almost-everywhere differentiable transformations. Linear layers mix coordinates. Nonlinear activations prevent the composition from collapsing into one global linear map. Hidden layers create intermediate representations. A forward pass evaluates the composition; a backward pass efficiently computes how the final loss depends on its parameters.

Nothing in this definition says that a network automatically understands a task. Nothing says that a deeper network must train better than a shallow one. Nothing says that a hidden unit corresponds to one human concept. Those are empirical questions about learned solutions, not consequences of the word “neural”.

The power of the construction is more modest and more precise: by composing trainable projections with nonlinear transformations, we obtain a flexible family of functions whose internal representations can themselves be adjusted using gradient information.

4.12 Connections to the Week 3 practicals

The practical sequence approaches the chapter’s argument from four complementary directions.

The first experiment begins with a failure: one straight line cannot solve the four-corner arrangement. Two thresholded hidden units can. The important observation is that the successful network changes the representation before making the final linear decision.

The second holds weights fixed and changes only activation functions. This separates the learned projection from the nonlinear response and makes local sensitivity visible before backpropagation is introduced.

The third watches a two-unit hidden representation evolve while an MLP learns a two-moons dataset. Because the hidden layer has exactly two units, its representation can be plotted directly. The experiment makes the phrase “representation learning” literal rather than metaphorical.

The fourth reduces backpropagation to one scalar computational graph. Finite differences, manual chain-rule derivatives and PyTorch autograd are compared on the same numbers, so the abstraction never outruns the calculation.

The lecture and practical therefore divide the labour deliberately. The notes provide the common language and the mathematical relationships; the notebook supplies the evidence that these relationships produce the claimed behaviour.

4.13 A compact map of the computation

The ideas of the chapter can be summarised in one chain:

$$\mathbf{x} \xrightarrow{\mathbf{W}_1, \mathbf{b}_1} \mathbf{z}_1 \xrightarrow{\phi} \mathbf{h}_1 \xrightarrow{\mathbf{W}_2, \mathbf{b}_2} \mathbf{s} \xrightarrow{\text{task/loss}} \mathcal{L}.$$

Read left to right, this is a forward computation. The hidden activation $\mathbf{h}_1$ is a learned representation of the input. Nonlinearity makes that representation capable of changing the geometry of the problem.

Read right to left, local derivatives transmit sensitivity:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{s}} \rightarrow \frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} \rightarrow \frac{\partial \mathcal{L}}{\partial \mathbf{z}_1} \rightarrow \frac{\partial \mathcal{L}}{\partial \mathbf{W}_1}.$$

Backpropagation performs this reverse calculation efficiently. An optimiser can then use the resulting gradients to update the parameters, after which a new forward pass begins.

This cycle joins the previous two weeks:

$$\text{examples} \rightarrow \text{objective} \rightarrow \text{optimisation} \rightarrow \text{representation}.$$

The new link is representation. The learner is no longer changing only how strongly it weights the original inputs. It is changing the coordinates in which the problem will be solved.

Check your understanding

Explain, without using the phrase “because neural networks are nonlinear”, why an MLP with a ReLU hidden layer can express a decision rule that a single linear classifier cannot. Your answer should mention the hidden representation and the output boundary.

4.14 Where we go next

We have deliberately studied networks in a forgiving setting: tiny datasets, shallow models and computations that fit on a page. This lets us understand what each operation is supposed to do.

Real training is less polite. Gradients can become very small or very large. ReLUs can remain inactive. Initial parameter scales matter. Different optimisers follow different trajectories. A model can fit the training data and still fail on new examples. A decreasing loss can coexist with the wrong learned behaviour. Several objectives can compete through one shared representation.

Those are not reasons to discard the simple picture developed here. They are reasons to preserve it. Week 4 asks what happens when this same forward–backward mechanism is placed inside a realistic learning system and begins to misbehave.

Chapter 5

Training, generalisation and diagnosis

A neural network can be perfectly legal mathematics and still be a poor learning system. The forward pass may produce finite numbers. The backward pass may produce gradients. The optimiser may update every parameter without error. The training loss may even fall smoothly. None of these facts, by itself, establishes that the network is learning the behaviour we intended.

This is the point at which neural-network work becomes engineering rather than algebra alone.

In Week 3 we asked what a multilayer network can represent. We saw that nonlinear hidden layers can reorganise a problem and that backpropagation can carry local sensitivity information through a computational graph. This week asks a harder question: what evidence tells us that this machinery is working well? A model can fail because the objective rewards the wrong behaviour, because useful gradients do not reach important parameters, because the representation collapses, because the model exploits an undesirable signal, or because a rule that fits the training sample does not survive outside it. These are different failures. They call for different interventions.

Key idea

A disappointing model score is a symptom, not a diagnosis. Before changing the optimiser, the architecture or the dataset, ask which part of the learning system is actually failing and what measurement would distinguish that explanation from its competitors.

The chapter therefore has two strands. The first completes the mechanics of neural-network classification and training: logits, cross-entropy, gradients through several layers, optimisation, initialisation and normalisation. The second develops a discipline for interpreting the resulting behaviour: train/validation comparisons, baselines, ablations, permutation controls, calibration and checkpoint selection.

5.1 Backpropagation in a complete network

Consider a two-layer network

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \quad (5.1)$$

$$\mathbf{h} = \phi(\mathbf{z}^{(1)}), \quad (5.2)$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}, \quad (5.3)$$

$$\mathcal{L} = \ell(\mathbf{z}^{(2)}, y). \quad (5.4)$$

The forward pass computes these quantities in order. During the backward pass we ask how a small change in each intermediate value would alter the final loss.

For the second layer the dependency is direct. The loss depends on $\mathbf{W}^{(2)}$ through $\mathbf{z}^{(2)}$. For the first layer the dependency is indirect:

$$\mathbf{W}^{(1)} \longrightarrow \mathbf{z}^{(1)} \longrightarrow \mathbf{h} \longrightarrow \mathbf{z}^{(2)} \longrightarrow \mathcal{L}.$$

The chain rule combines local sensitivities along this path. In scalar notation we wrote such products explicitly in Week 3. With vectors and matrices the bookkeeping becomes larger but not conceptually different.

It is useful to imagine a backward message attached to each node of the graph. At the output we know how the loss changes with the output score. That message is transformed as it passes backward through each operation. A linear layer sends sensitivity to both its inputs and its parameters. An activation multiplies the incoming sensitivity by its local derivative. When two later computations depend on the same earlier value, their contributions add.

This last point is why backpropagation is efficient. The sensitivity of the loss with respect to an intermediate activation is computed once and reused for every earlier parameter that influences that activation. We do not perturb millions of parameters one by one and rerun the network millions of times.

ACTION — create this figure

Draw a compact computational graph for a two-layer MLP: $\mathbf{x} \rightarrow$ affine layer $\rightarrow \mathbf{z}^{(1)} \rightarrow \text{ReLU} \rightarrow \mathbf{h} \rightarrow$ affine layer $\rightarrow \mathbf{z}^{(2)} \rightarrow$ logits $\rightarrow$ loss. Above the arrows show the forward values. Beneath the same arrows show backward sensitivities $d\mathcal{L}/d(\cdot)$ travelling in the opposite direction. At the two affine layers show separate branches to parameter gradients $d\mathcal{L}/d\mathbf{W}$ and $d\mathcal{L}/d\mathbf{b}$.

Figure 5.1: Forward computation and backward sensitivity are two passes over the same computational graph. Backpropagation reuses intermediate sensitivities rather than estimating each parameter derivative independently.

5.1.1 Automatic differentiation is bookkeeping, not an optimiser

Modern libraries record the differentiable operations that produced a loss and then apply the chain rule automatically. In PyTorch, calling `loss.backward()` fills gradient fields such as `weight.grad`. Nothing has yet been optimised. A separate step decides how those gradients change the parameters.

This separation matters experimentally. If a model fails, we may ask three different questions:

1. Was the forward computation capable of representing a useful solution?
2. Did the backward computation deliver informative gradients to the relevant parameters?
3. Did the update rule use those gradients in a productive way?

A single training-loss curve does not answer all three.

PyTorch: one forward pass, one backward pass

```
model = nn.Sequential(
    nn.Linear(20, 32),
    nn.ReLU(),
    nn.Linear(32, 1),
)

logits = model(x_batch).squeeze(-1) # forward pass
loss = criterion(logits, y_batch.float())

optimizer.zero_grad()
loss.backward() # compute gradients
optimizer.step() # update parameters
```

The three lines at the end should remain mentally distinct. Clearing stale gradients, computing new gradients and updating parameters are separate operations.

5.2 Classification: from scores to objectives

In Week 3 a classifier produced a real-valued score, or *logit*. The sign of a binary logit could be used to choose one of two classes. To train such a classifier, however, we want a differentiable objective that does more than say “right” or “wrong”. It should distinguish a barely correct score from a confidently correct one, and a barely incorrect score from a wildly incorrect one.

5.2.1 Binary classification

Let $z \in \mathbb{R}$ be the logit for one example. The sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (5.5)$$

maps it to a number between zero and one. We can interpret

$$p = \sigma(z)$$

as the model’s probability assigned to class 1.

For a target $y \in \{0, 1\}$, binary cross-entropy is

$$\ell_{\text{BCE}}(p, y) = -y \log p - (1 - y) \log(1 - p). \quad (5.6)$$

If $y = 1$, the loss is small when p is near one and large when p is near zero. If $y = 0$, the roles reverse. A confidently wrong prediction is therefore penalised much more strongly than an uncertain one.

The usual implementation does not require us to compute a sigmoid ourselves before the loss. Libraries combine the logit and cross-entropy calculation in a numerically stable expression. In PyTorch this is the role of `BCEWithLogitsLoss`.

Key idea

The network should normally output logits during training. The loss function can combine the appropriate probability transformation with cross-entropy in a numerically stable way. Convert logits to probabilities separately when you need to inspect or threshold them.

A probability is still not a class decision. At deployment we might predict class 1 when $p \geq 0.5$, but 0.5 is a decision threshold, not a law of nature. If missing a positive example is costly, a lower threshold may be preferable; if false alarms are costly, a higher one may be appropriate. Precision and recall change as the threshold moves even though the network parameters do not.

5.2.2 Multi-class classification

Suppose there are K mutually exclusive classes. The network produces a vector of logits

$$\mathbf{z} = (z_1, \dots, z_K).$$

The softmax function converts these scores into probabilities that sum to one:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}. \quad (5.7)$$

If the true class is c , the cross-entropy loss is

$$\ell_{\text{CE}} = -\log p_c. \quad (5.8)$$

Increasing the true-class logit helps, but because all softmax probabilities share the same denominator, the class scores compete with one another.

Sigmoid and softmax therefore express different assumptions. A sigmoid treats each output as an independent yes/no probability and is appropriate for binary or multi-label problems. Softmax treats the outputs as mutually exclusive alternatives. A photograph may simultaneously contain a dog, a bicycle and a person, so independent sigmoids make sense for object presence. A handwritten digit is normally assigned one class among ten, so softmax is natural.

PyTorch: binary and multi-class logits

```
# Binary: one logit per example
binary_head = nn.Linear(hidden_dim, 1)
binary_loss = nn.BCEWithLogitsLoss()
loss = binary_loss(binary_head(h).squeeze(-1), y.float())

# Multi-class: K logits per example
multiclass_head = nn.Linear(hidden_dim, num_classes)
multiclass_loss = nn.CrossEntropyLoss()
loss = multiclass_loss(multiclass_head(h), class_index)
```

Optional mathematics — the clean gradient hidden inside BCE

For binary cross-entropy with $p = \sigma(z)$, show that

$$\frac{\partial \ell_{\text{BCE}}}{\partial z} = p - y.$$

The result is strikingly simple: the sensitivity of the loss to the logit is exactly the difference between predicted probability and target. Work through the chain rule using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Then interpret the sign of $p - y$ separately for $y = 0$ and $y = 1$.

5.3 A loss can decrease while learning goes wrong

Once classification loss is available, it is tempting to reduce neural-network engineering to a single question: did the loss go down? That question is necessary but weak.

A falling loss establishes that the optimiser found parameter changes that improve the chosen objective on the examples being measured. It does not establish that the model uses the intended input, that useful information survives in hidden layers, that validation behaviour improves, or that the objective itself rewards what we care about.

We need to look at the dynamics of learning, not merely its final number.

5.3.1 Plateaus, saddles and local minima

The loss surface of a neural network is a function over all parameter values. With millions of parameters it cannot be drawn directly, but the geometric language from Week 2 still helps.

A *local minimum* is a point lower than nearby alternatives. A *saddle* descends in some directions and rises in others. A *plateau* is a region where gradients are small and progress is slow. Neural-network loss surfaces contain many such structures, but it is misleading to imagine training primarily as a little ball repeatedly getting trapped in bad local bowls. In high dimensions there are many directions, and practical failures often come from scale, gradient flow, data, objective choice or generalisation rather than from one simple local minimum.

The useful engineering question is not “does the surface contain local minima?” It is “what evidence explains the trajectory I am observing?”

5.3.2 Vanishing and exploding gradients

Backpropagation repeatedly multiplies local derivatives. If many factors have magnitude substantially below one, a sensitivity can shrink as it travels backward. Earlier layers then receive very small gradients. If many factors are large, gradients can grow instead, producing unstable updates.

These effects are called *vanishing* and *exploding* gradients. Their severity depends on the architecture, activation functions, parameter scales and sequence depth. We will meet architectures later that were partly invented to create easier paths for information and gradients.

A gradient norm is a simple diagnostic. For a parameter tensor θ , we can monitor

$$\|\nabla_{\theta} \mathcal{L}\|_2.$$

A number close to zero is not automatically bad; a well-trained model near a stationary point may have small gradients. The informative question is whether gradients vanish in layers that still need to learn while the loss remains poor.

5.3.3 Dead ReLUs

ReLU gives

$$\phi(z) = \max(0, z), \quad \phi'(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases}$$

If one unit receives negative pre-activations for every relevant example, its activation is always zero. The local derivative is also zero, so ordinary gradient information does not pass through that unit to the parameters that determine its pre-activation. The unit is effectively silent.

One zero ReLU output on one example is normal. ReLU is designed to be inactive in part of the input space. The warning sign is persistent inactivity across nearly all examples, especially when accompanied by near-zero gradient norms and little variation in the hidden representation.

This illustrates a general principle: a representation can fail without producing a software error. The tensor may have the expected shape and dtype while carrying almost no useful information.

ACTION — create this figure

Create three aligned training-diagnostic panels for a healthy and a failed tiny ReLU network. Panel 1: train/validation loss. Panel 2: hidden activation spread and fraction of zero activations. Panel 3: first-layer gradient norm. In the failed run show loss stagnation, near-zero activation spread, many zero activations and collapsed early-layer gradients.

Figure 5.2: Training dynamics contain more information than a final loss. Activation spread and gradient flow can distinguish an optimisation/representation failure from an ordinary slow trajectory.

5.4 Initialisation: beginning in a learnable regime

Before the first update, the network already has a geometry determined by its initial parameters. If all weights in a layer were exactly equal, many units would receive identical gradients and remain redundant. If weights are too large, activations and gradients can grow through depth; if too small, signals can shrink toward zero.

Initialisation schemes therefore try to preserve a sensible scale as information moves through a network. The precise formulas are less important here than the principle.

For a linear layer with n_{in} inputs, Xavier-style initialisation chooses a weight scale based on both input and output width and is well matched to roughly symmetric activations such as tanh. Kaiming/He-style initialisation accounts for the fact that ReLU discards the negative half of its pre-activations and scales weights accordingly.

These schemes do not magically place the model near the best solution. Their purpose is more modest and more important: begin in a regime where activations vary and gradients can propagate.

Biases matter too. A sufficiently negative bias before a ReLU can push almost every pre-activation below zero, immediately silencing a layer. The Week 4 practical makes this

pathology deliberately visible. The important lesson is not that one should memorise “never use negative bias”. It is that initial conditions can determine whether the model begins in a region from which gradient-based learning has useful information.

5.5 Optimisers change the trajectory

The simplest update from Week 2 was stochastic gradient descent:

$$\theta_{t+1} = \theta_t - \eta g_t, \quad (5.9)$$

where g_t is a gradient estimate and η is the learning rate.

Different optimisers transform this gradient information before applying an update. They can make training substantially easier, but they should not be treated as universal repairs for any disappointing model.

5.5.1 Momentum

SGD with momentum keeps a running velocity. One common form is

$$\mathbf{v}_{t+1} = \mu \mathbf{v}_t + \mathbf{g}_t, \quad (5.10)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_{t+1}. \quad (5.11)$$

The parameter update therefore depends not only on the current gradient but also on recent gradients. Directions that persist across batches accumulate; rapid oscillations can partly cancel. Geometrically, momentum gives the optimiser inertia.

5.5.2 Adam and AdamW

Adam keeps moving estimates of the first and second moments of the gradient. Informally, it combines a momentum-like direction estimate with parameter-wise scaling based on recent gradient magnitude. Parameters with very different gradient scales can therefore receive updates of more comparable effective size.

AdamW is commonly preferred when weight decay is desired because it separates the decay operation cleanly from the adaptive gradient transformation.

The important conceptual distinction is between *gradient* and *update*. Backpropagation tells us local sensitivity. The optimiser decides how that sensitivity, its recent history and possibly regularisation should produce a parameter change.

5.5.3 Learning-rate schedules

A useful learning rate early in training need not remain useful later. Schedules change η over time. A step schedule reduces it at chosen milestones. A cosine schedule decreases it smoothly. One-cycle strategies first increase and later decrease the rate.

The details vary, but the shared purpose is to control the scale of movement through parameter space. A larger rate can make rapid progress when the current solution is crude; a smaller rate can help refine a solution without continually overshooting.

Again, a schedule cannot repair a target that contains no useful signal, a metric that rewards the wrong behaviour, or a hidden representation that has collapsed completely. Optimisation choices act on the trajectory available to them.

5.5.4 Curriculum learning

Most optimisation methods change how parameters respond to examples. Curriculum learning changes which examples are presented, or their difficulty, over time. A learner may first encounter easier or cleaner cases and later face harder ones.

This is a useful reminder that the optimisation trajectory depends on more than the optimiser object. Batch order, sampling, data augmentation and curriculum all alter the sequence of gradient information received by the model.

Check your understanding

A model has almost-zero first-layer gradient norm from the first batch onward, and every hidden ReLU is zero for every example. Would switching from SGD to Adam be your first diagnostic intervention? What evidence would you inspect before changing the optimiser?

5.6 Keeping scales under control: normalisation

Even with sensible initialisation, activation distributions change during training because the parameters feeding each layer change. Normalisation layers explicitly control some statistics of intermediate representations.

The three common names in this module differ mainly in *which axes are normalised*.

Batch Normalisation. BatchNorm estimates statistics using examples in the mini-batch, separately for channels or features. It has been particularly influential in convolutional networks. Because it depends on batch statistics during training, behaviour changes with batch size and the layer maintains running statistics for evaluation.

Layer Normalisation. LayerNorm normalises across the features of each individual example. It does not require other examples in the batch and is therefore natural in sequence models and many modern architectures.

Group Normalisation. GroupNorm divides channels into groups and normalises within each group for each example. It is useful when batch statistics are noisy or batch sizes are small.

After normalising, these layers usually apply learned scale and shift parameters, so normalisation does not force every representation to remain permanently standardised. It provides a more controlled coordinate system from which the network can learn.

The engineering question is not “which normalisation is best?” in the abstract. It is which statistics are meaningful and stable for the architecture and data at hand.

5.7 Generalisation: fitting the sample is not the final task

Optimisation concerns performance on the objective we compute during training. Generalisation concerns behaviour on examples that did not determine the fitted parameters.

A model can reach nearly zero training loss by learning genuine structure, by memorising peculiar examples, by exploiting a shortcut that only exists in the training distribution, or by some mixture of these. Training performance alone cannot tell us which occurred.

5.7.1 Underfitting and overfitting

A model *underfits* when it cannot obtain good performance even on the training data under the present training setup. The cause might be insufficient capacity, poor optimisation, an inappropriate representation or simply an impossible/noisy target.

A model *overfits* when training performance continues to improve while performance on held-out data stagnates or degrades. The model is becoming better adapted to the finite training sample without becoming better at the intended population-level task.

The familiar picture is two curves: training loss falls steadily; validation loss falls at first and then turns upward. Useful as that picture is, overfitting need not always appear in exactly that form. Accuracy may plateau while confidence becomes more extreme. Different subgroups can degrade at different times. A shortcut feature can produce excellent training accuracy almost immediately while validation performance never becomes competitive.

ACTION — create this figure

Draw three stylised train/validation learning-curve pairs. A: both high and flat (underfitting or optimisation failure). B: both improve and remain close (healthy generalisation). C: training keeps improving while validation worsens after a minimum (overfitting). Beneath the figure add the warning: “curves are symptoms; inspect mechanism before assigning a cause”.

Figure 5.3: Train and validation trajectories help separate fitting from generalisation, but the same curve shape can have several causes. They are evidence to combine with other diagnostics, not diagnoses by themselves.

5.7.2 Capacity and the limits of the simple bias–variance cartoon

Width, depth and parameter count influence how many functions a model can express and how easily it can adapt to training examples. Increasing capacity can reduce underfitting, but it also makes it possible to fit increasingly idiosyncratic structure.

A classical story therefore predicts a U-shaped test-error curve: too little capacity underfits; too much overfits. Modern overparameterised networks complicate this picture. In some regimes test error can rise near the point where the model first becomes capable of interpolating the training set and then fall again as capacity increases further, a phenomenon called *double descent*.

The practical lesson is not that “bigger is always better”. It is that capacity and generalisation do not obey one universal monotonic rule. They interact with optimisation, regularisation, data volume, architecture and implicit biases of the training procedure. Validation evidence remains indispensable.

5.7.3 Regularisation

Regularisation changes which fitted solutions are easy or attractive.

Weight decay. Weight decay discourages large parameter magnitudes. In simple settings it corresponds closely to adding an L_2 penalty to the objective. In adaptive optimisers, implementations such as AdamW decouple this decay from the gradient scaling.

Dropout. During training, dropout randomly sets a fraction of activations to zero and rescales the survivors. The network cannot rely on exactly the same set of units on every pass, which can discourage brittle co-adaptation. At evaluation time dropout is disabled.

Data augmentation. Augmentation modifies inputs while preserving the target: small image translations, crops, flips where appropriate, noise, or domain-specific transformations. It encodes an assumption that certain changes should not alter the answer. Unlike a generic penalty on parameters, augmentation regularises by changing the observed training distribution.

Early stopping. If validation performance peaks and then deteriorates, stopping at the best validation checkpoint prevents later training from replacing a more general solution with a more sample-specific one. Early stopping therefore links regularisation directly to checkpoint selection.

Regularisation is not a substitute for adequate signal. If the labels are wrong, the task is ill-defined or the metric rewards an undesirable output, making the model smaller or adding dropout does not solve the specification problem.

5.8 Shortcuts: learning can succeed for the wrong reason

Imagine a digit classifier in which every training image of class A receives a tiny bright patch in one corner, while class B does not. The patch is highly predictive, much simpler than the digit shape, and available to the learner. A network that relies on it may achieve excellent training performance.

Has learning failed? In one sense, no. The network has found a real statistical regularity in its training data and used it successfully. The failure lies in the relationship between that regularity and the deployment problem we care about.

If clean validation images do not contain the patch, performance may collapse. More revealingly, we can intervene. Add the patch to clean images. Remove it. Reverse its association with the class. If predictions change dramatically while the digit itself is untouched, we have causal evidence that the model depends on the shortcut.

This is stronger than merely observing a suspicious correlation between training and validation scores. It changes one candidate signal while holding the rest of the input approximately fixed.

Key idea

A feature can be genuinely predictive and still be undesirable. Generalisation is not only about whether the model learned a pattern, but whether it learned a pattern that remains valid under the conditions in which the model will be used.

The Week 4 shortcut practical makes this distinction deliberately artificial so it is easy to see. Real shortcuts are often less obvious: scanner marks in medical images, background context in object recognition, collection-site artefacts, author identity in text, or leakage from how labels were created.

5.9 Calibration: confidence is another behaviour to evaluate

Accuracy asks whether the predicted class was correct. Calibration asks whether stated probabilities correspond to empirical frequencies.

Suppose a classifier assigns probability about 0.8 to one thousand predictions. If it is well calibrated, roughly 80% of those events should be correct. A model can have high accuracy and still be overconfident or underconfident.

A simple reliability diagram groups predictions into probability bins and compares mean confidence with observed accuracy in each bin. Perfect calibration lies near the diagonal.

Calibration matters whenever a probability will be used as a degree of confidence rather than merely as an intermediate score. Threshold decisions, triage systems and risk-sensitive applications all depend on it.

Calibration is also another example of the module's recurring theme: one metric does not answer every question. Accuracy, ranking quality, F1 and calibration describe different aspects of behaviour.

Optional coding — build a reliability diagram

Take a trained binary classifier and its validation probabilities. Divide the interval $[0, 1]$ into ten bins. For each non-empty bin, compute the mean predicted probability and the empirical fraction of positive outcomes. Plot one against the other together with the line $y = x$.

Then answer: can two classifiers have similar accuracy but visibly different calibration? What decision would make that difference matter?

5.10 Experimental discipline: ask one diagnostic question at a time

A neural-network experiment should be designed to discriminate between explanations. This sounds obvious, yet a common debugging pattern is to change the learning rate, optimiser, width, dropout, data preprocessing and loss simultaneously. If performance improves, the cause remains unknown.

A more useful sequence begins with cheap, interpretable tests.

5.10.1 Better than what?

Compare against a trivial baseline or floor before interpreting a learned score. For classification this may be the majority class, a constant probability or a simple heuristic. For temporal prediction it may be “same as the last step”. For image prediction it may be a constant image or copying an input.

A baseline provides scale. An accuracy of 82% can be impressive if chance is 50% and embarrassing if a one-line heuristic obtains 90%.

5.10.2 Does the output depend on the intended input?

Prediction spread is one crude test: do outputs vary across examples? A stronger test deliberately destroys or removes a candidate signal.

In a *permutation control*, one feature or condition is shuffled across examples while targets remain fixed. If performance barely changes, the model may not be using that signal. If performance collapses, the signal mattered to the fitted predictor.

In an *ablation*, we remove or neutralise a component or input source and compare behaviour. The ablation should be narrow enough that its interpretation is clear.

These interventions are especially important in multimodal systems. A model given image and text can obtain good output while silently ignoring the text if the image alone

is easier. A shuffled-condition test directly asks whether pairing the correct text with the correct image matters.

5.10.3 What is happening inside the representation?

When predictions are nearly constant, inspect hidden variation. Useful quantities include activation standard deviation across examples, fraction of zero activations, pairwise similarity between representations, and gradient norms by layer.

A representation with 256 coordinates is not automatically rich. If every example maps to almost the same vector, the effective information content is tiny. Conversely, large activation variance is not automatically useful; variation must relate to the task. Internal diagnostics narrow the explanation but must be connected back to behaviour.

5.10.4 Train and validation answer different questions

Training loss answers: *how well are we fitting the objective on examples used for optimisation?*

Validation metrics answer: *how well does the current fitted model behave on held-out development data?*

The distinction matters for checkpoint selection. If the deployment goal is clean validation accuracy, choosing the epoch with lowest training loss can favour a later, more overfit model. The criterion used to select a checkpoint becomes part of the learning procedure.

The test set must remain outside this loop. If we repeatedly choose architectures, thresholds or checkpoints based on test performance, the test set gradually becomes development data and loses its role as an independent final estimate.

5.10.5 Multi-task learning: one representation, several objectives

Modern systems often optimise several losses at once. Suppose one shared representation h feeds an image head, a text head and a character-prediction head:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{img}} + \lambda_2 \mathcal{L}_{\text{text}} + \lambda_3 \mathcal{L}_{\text{char}}.$$

Each head sends a gradient into the shared representation. Those gradients need not point in compatible directions. One objective may dominate simply because its scale is larger or because it is easier to reduce.

This is *multi-task competition*. A shared vector is not an unlimited neutral container. If one head can improve by collapsing or reshaping the representation in a way that harms another, the total objective may still decrease.

The first response should again be diagnostic: log each loss separately, evaluate each task separately, inspect gradients or representation spread, and compare shared versus separate projections where possible. Merely increasing the width of the shared vector may hide rather than explain the conflict.

Application to the architecture — a diagnostic cascade

The StoryReasoning system offers several examples of why diagnosis must precede redesign.

1. The metric can reward the wrong answer. On held-out next-frame windows, a constant per-pixel median image obtains image L1 around 0.151, while copying the last real frame is around 0.170. On shot cuts, a bland constant can therefore outscore a sharp frame from the same story. This is an objective/metric problem before it is an optimisation problem.

2. A representation can collapse while the program still runs. An early 16-dimensional visual bottleneck with ReLU produced many inactive units and an almost constant latent representation. Increasing the latent width and using LayerNorm produced a much healthier representation. The relevant evidence was not merely final loss: dead-unit counts and latent spread showed that the hidden state had stopped carrying useful variation.

3. A decoder can ignore a condition. In a later multimodal stage, text cross-entropy was almost unchanged when the conditioning signal was shuffled. The generator could model the corpus style without using the intended context. A shuffled-condition control exposed this signal-use failure immediately.

4. Pretraining can be destroyed during fine-tuning. The visual autoencoder reconstructed frames at about 0.040 L1 before sequence training. Merely lowering its learning rate did not prevent reconstruction from drifting to roughly 0.080. Keeping a reconstruction objective active during fine-tuning held the monitor near 0.041. The reconstruction score acted as a health monitor for a pretrained capability.

5. The best checkpoint depends on the question. Pixel loss began overfitting before other heads had finished learning. Selecting the checkpoint with the lowest image L1 therefore chose the wrong stage of training for the system as a whole. Validation retrieval, which better reflected the desired conditional representation, was used instead.

The common pattern is more important than the specific mechanisms: establish a floor, perturb the claimed signal, inspect representation health, monitor pretrained capabilities, and select checkpoints using the metric that answers the actual question.

5.11 A compact diagnostic workflow

When a model behaves suspiciously, the order of questions matters. Expensive interventions should come after cheap evidence.

1. **Define the desired behaviour.** Which metric actually represents success? Could the objective reward an undesirable shortcut or average?
2. **Establish floors and baselines.** Does the learned model beat a constant, chance predictor or simple heuristic?
3. **Check input dependence.** Do predictions vary with the input? What happens if the intended signal is shuffled, permuted or ablated?
4. **Inspect training dynamics.** Are losses improving? Do gradients reach early layers? Are activations varied or collapsed?
5. **Compare train and validation.** Is the model failing to fit, or fitting without generalising?
6. **Inspect checkpoint choice.** Was the model selected using a metric aligned with deployment behaviour?
7. **Only then modify the mechanism suggested by the evidence.** Change one factor at a time when possible.

ACTION — create this figure

Create a compact decision-flow diagram beginning with “Model behaviour is poor or suspicious”. Branch through: metric/objective aligned? -> beats baseline? -> output depends on intended signal under permutation/ablation? -> activations and gradients healthy? -> train vs validation gap? -> checkpoint aligned? End branches with candidate diagnoses: objective mismatch, insufficient signal/use, optimisation/representation failure, generalisation failure, checkpoint/selection failure. Include a note: categories can overlap.

Figure 5.4: A diagnostic workflow for neural-network experiments. The purpose is not to assign a label mechanically, but to order measurements so that competing explanations can be separated before redesign.

The categories can overlap. A representation may collapse because optimisation drove ReLUs into an inactive region. A model may generalise poorly because it successfully learned a shortcut. A multi-task objective may both distort a representation and select a poor checkpoint. Diagnosis is therefore not taxonomy for its own sake. It is a way of asking which intervention is justified by evidence.

5.12 What to carry forward

By this point a neural network should no longer look like a single object called “the model”. It is a coupled system:

data $\rightarrow$ representation $\rightarrow$ prediction $\rightarrow$ objective $\rightarrow$ gradients $\rightarrow$ updates,

observed through finite training and validation samples.

Every arrow can fail in a different way. The data can contain a shortcut. The representation can collapse. The prediction head can ignore one condition. The objective can reward the wrong behaviour. Gradients can vanish. Updates can be too large. A model can fit the training set and fail on validation. A useful intermediate capability can drift during fine-tuning. A checkpoint can be selected by the wrong metric.

This is why neural-network engineering is empirical in a deeper sense than “try some hyperparameters”. Good experiments are designed to make explanations compete. A baseline asks whether complexity helped. A permutation asks whether a signal mattered. An activation monitor asks whether a representation remained alive. A validation curve asks whether fitting transferred beyond the training sample. An ablation asks what behaviour survives when one mechanism is removed.

Week 5 will turn from these general learning-engineering questions to a particular architectural assumption: images contain local spatial structure. Convolutional networks build that assumption directly into the computation. The same discipline developed here will remain essential. An inductive bias is useful only if we can show what behaviour it changes and whether that change survives beyond the training examples.

Check your understanding

A classifier reaches 99% training accuracy and 71% validation accuracy. Replacing one small corner patch in every validation image with zeros changes 40% of its predictions; zeroing an unrelated patch changes 2%. Its hidden activations and gradients look

healthy. Which failure explanation is most strongly supported? Which tempting explanation is now less plausible?

Chapter 6

Convolution and inductive bias

In the previous chapters, we gave the learner increasingly powerful tools. A multilayer network can represent nonlinear rules; backpropagation can train many parameters; modern optimisation and regularisation can make that training practical. Yet one question has so far been left deliberately unresolved: *which computations should the architecture make easy?*

An image is not an arbitrary list of numbers. Nearby pixels are often related. An edge can appear near the top of an image or near the bottom and still be the same kind of edge. Small visual structures combine into larger ones. If a model must discover all of these facts from scratch, then much of its data and parameter budget is spent relearning regularities that we already know are common in images.

A convolutional neural network builds some of those assumptions into the computation. It does not guarantee that the assumptions are correct, and it does not make a model immune to bad data or bad objectives. It changes which solutions are easy to represent and easy to learn.

This is an example of an *inductive bias*: a preference built into the learner before it sees the particular training set.

Key idea

An architecture is not a neutral container for parameters. It expresses assumptions about the structure of the problem. Convolution says, roughly, that useful local patterns may recur at different spatial locations and should be processed by the same detector.

That statement is more important than the convolution formula itself. The purpose of this chapter is to unpack what it means, where it helps, what information it discards, and how it leads naturally to visual encoders that can be reused by later systems.

6.1 From prediction to representation

Until now, many of our examples could be drawn as

input $\longrightarrow$ network $\longrightarrow$ prediction.

For modern visual systems, it is often more useful to split the network conceptually into two parts:

image $\longrightarrow$ $\underbrace{\text{feature extractor / encoder}}_{\text{shared representation}} \longrightarrow \underbrace{\text{task head}}_{\text{specific output}} .$

The encoder turns a raw image into a representation. The head interprets that representation for a particular task.

A classifier head might produce class logits. A detector head might produce bounding boxes and object scores. A segmentation head might produce one class distribution per pixel. A retrieval system might compare the representation with vectors from other images. A decoder might attempt to reconstruct an image from the representation.

This separation matters because the representation can be useful beyond the task that originally created it. If an encoder has already learned edges, textures, shapes and higher-level visual structure, a later task need not begin again from individual pixels. This is the basic logic behind *pretraining* and *transfer learning*.

The split is conceptual rather than absolute. The head can influence the features during end-to-end training, and different tasks favour different representations. But it gives us a useful question to ask of every visual model:

What information has the encoder made easy for the head to use?

Week 4 taught us not to infer this from accuracy alone. In this chapter we begin by asking what architecture makes likely even before training has produced a particular representation.

6.2 Why not flatten the image?

Suppose an RGB image has height H , width W , and three colour channels. Flattening it produces a vector in

$$\mathbb{R}^{3HW}.$$

A fully connected layer from this vector to m hidden units contains approximately

$$3HWm$$

weights.

For a 224×224 RGB image and only $m = 1000$ hidden units, this is already about

$$224 \times 224 \times 3 \times 1000 \approx 150 \text{ million}$$

weights before the model has computed a second layer.

The number is not the deepest problem. A more important issue is what those weights *mean*. In a flattened network, the weight connecting the pixel at location (10,10) to one hidden unit is independent of the weight connecting an otherwise identical local pattern at (100,100). Nothing in the architecture says that the same visual structure should be treated similarly after it moves.

This creates two related difficulties.

First, the model must learn local structure from a representation in which every location is initially unrelated to every other location. An edge detector learned in one corner does not automatically become an edge detector elsewhere.

Second, the model spends separate parameters on rules that could reasonably be shared. More parameters are not always harmful, but unnecessary freedom means that more examples may be required to discover the intended regularity.

A fully connected network can, in principle, learn many image tasks. The point is not impossibility. The point is that its default assumptions do not match the spatial structure we usually expect in images.

Check your understanding

Imagine a dataset in which a small symbol determines the class, but during training the symbol appears only on the left side of the image. Which architecture would require explicit evidence that the same symbol on the right should mean the same thing: a flattened MLP or a convolutional network? Why?

This is the distinction explored experimentally in the Week 5 practical: both models can fit the observed training positions, but the architectures differ in how naturally the rule transfers to held-out positions. The useful conclusion is not “CNNs are always better.” It is that architecture changes which generalisations are cheap to learn.

6.3 The convolutional idea: one local detector, reused

Consider a single-channel image X and a small $k \times k$ matrix of weights K , which we shall call a *kernel* or *filter*. At one spatial location, the kernel looks only at a local image patch. It multiplies corresponding values and adds them:

$$Z_{i,j} = b + \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{u,v} X_{i+u,j+v}. \quad (6.1)$$

The important step is then to move the same kernel across the image and repeat the calculation.

The output Z is a *feature map*. One output number says how strongly this learned local pattern responds at one location. The next output number applies the same learned detector to a neighbouring patch.

ACTION — create this figure

Draw a small grayscale image with a bright L-shaped motif. Overlay a 3x3 receptive-field window and show one kernel-dot-patch calculation producing one cell of a feature map. Then show the same kernel sliding to several positions. Emphasise that the nine kernel weights are reused, not copied as independently learned weights.

Figure 6.1: Convolution applies one local detector repeatedly across space. Each output value is computed from a local receptive field, but the kernel parameters are shared across all spatial positions.

Two assumptions have appeared at once.

6.3.1 Locality

The unit at (i, j) does not initially inspect the entire image. It sees a local *receptive field*. This says that nearby pixels are likely to contain useful jointly interpretable structure.

For many visual patterns this is sensible. Edges, corners, texture fragments and small parts are local phenomena. A model need not compare every pixel directly with every other pixel in its first operation.

Locality is still an assumption. If a task depends immediately on an arbitrary long-range relationship between distant pixels, a very local architecture may require depth before the relevant information can interact.

6.3.2 Weight sharing

The same kernel is used at every spatial position. The model therefore does not learn one edge detector for the upper-left corner and another unrelated detector for the lower-right corner. It learns a detector and asks where that detector responds.

If a kernel has size 3×3 , one input channel and one output channel, it needs only nine weights plus a bias, regardless of whether the input image is 20×20 or 2000×2000 . The number of *applications* grows with image size, but the number of learned detector parameters does not.

This is parameter efficiency, but the stronger consequence is statistical. Evidence for the detector at one location updates the same parameters that will be used at other locations. The learner can therefore pool evidence across positions.

Key idea

Weight sharing is a statement about what should count as the same feature. It says that a local pattern detected here and the same local pattern detected there should use the same learned rule.

6.4 Translation equivariance is not invariance

Suppose an object moves five pixels to the right. What should happen inside the network?

For an ordinary convolution, the corresponding activation pattern also moves approximately five pixels to the right. The output changes, but it changes in a predictable way with the input. This property is called *translation equivariance*.

Writing a translation operation as T_Δ , ideal equivariance has the form

$$f(T_\Delta X) = T_\Delta f(X).$$

Translate the input, and the representation translates with it.

This is different from *translation invariance*, for which

$$f(T_\Delta X) = f(X).$$

An invariant representation ignores the transformation entirely.

The distinction is essential. A feature map should often preserve *where* a feature occurred. If an edge moves, it is useful for the edge response to move. A final image classifier, by contrast, may care that a dog is present while caring less about whether the dog is 20 pixels left or right of centre.

Real CNNs are not perfectly translation equivariant in every circumstance. Stride, padding, boundaries, pooling and finite sampling can disturb the ideal relationship. It is therefore safer to treat convolution as a strong architectural tendency toward translation-consistent feature extraction rather than as a magical guarantee of complete shift robustness.

6.5 Images have channels; convolutions create channels

An RGB image is already a stack of three two-dimensional arrays. Convolution generalises naturally to this case.

ACTION — create this figure

Create three small diagrams. First row: an input motif moves right and its convolutional feature-map peak moves right: label this equivariance. Second row: after global pooling, both moved inputs produce the same one-number summary: label this invariance. Third row: show a localisation task where preserving the moved position is necessary.

Figure 6.2: Equivariance preserves the structure of a transformation in the representation; invariance deliberately removes sensitivity to that transformation. Which is desirable depends on the task.

Let the input have C_{in} channels. One output kernel must have one slice for each input channel, so its shape is

$$C_{\text{in}} \times k_h \times k_w.$$

At each spatial location, the kernel combines values across both space and input channels to produce one output number.

To produce C_{out} output feature maps, we learn C_{out} such kernels. The full weight tensor therefore has shape

$$C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w.$$

The parameter count, including one bias per output channel, is

$$C_{\text{out}} (C_{\text{in}} k_h k_w + 1). \quad (6.2)$$

Notice what is absent from this expression: image height and image width.

An output channel can be thought of as a learned kind of detector. The first layer might learn several oriented edges or colour contrasts. Later layers do not operate directly on RGB pixels; they operate on feature maps produced by earlier layers, so their kernels combine already learned visual features.

This creates a hierarchy. A later unit can respond to a configuration of earlier features, and a still later unit can respond to larger configurations. The hierarchy is not guaranteed to correspond neatly to human concepts, but the computational structure makes compositional feature building possible.

PyTorch: one convolutional layer

```
conv = nn.Conv2d(
    in_channels=3,
    out_channels=32,
    kernel_size=3,
    stride=1,
    padding=1,
)

x = torch.randn(16, 3, 60, 125) # batch, channels, height, width
h = torch.relu(conv(x))
print(h.shape) # (16, 32, 60, 125)
```

In this example, padding preserves the spatial height and width while changing the channel dimension from three raw colour channels to 32 learned feature channels.

6.6 Stride, padding and output size

A convolutional layer makes several design decisions beyond the kernel values.

6.6.1 Padding

Without padding, a kernel cannot be centred near the image boundary without extending beyond the available pixels. A 3×3 valid convolution therefore shrinks the output.

Padding adds values around the boundary, commonly zeros. With a 3×3 kernel, stride one and one pixel of padding on each side, the output can preserve the original spatial size.

Padding is not merely formatting. Boundary values have less real visual context than interior values, and the chosen padding rule affects what the model sees near edges.

6.6.2 Stride

A stride of one evaluates neighbouring receptive fields one pixel apart. A stride of two skips alternate positions, reducing spatial resolution. Strided convolution therefore performs feature extraction and downsampling in one learned operation.

For one spatial dimension with input size n , kernel size k , padding p and stride s , the output size is

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1. \quad (6.3)$$

The same formula is applied independently to height and width.

Optional mathematics — shapes and parameter counts

An input batch has shape $32 \times 3 \times 64 \times 64$. A convolution uses 48 output channels, a 5×5 kernel, padding 2 and stride 2.

1. Compute the output height and width.
2. Compute the shape of the output tensor.
3. Compute the number of learned weights and biases.
4. Explain why doubling the image width would change the number of convolution operations but not the number of learned parameters.

6.7 Receptive fields grow through depth

A first-layer 3×3 convolution sees only 3×3 pixels. Does that mean a CNN can never use distant information? No. Receptive fields compose.

Suppose a second 3×3 convolution reads the first feature map. One second-layer unit depends on a 3×3 region of first-layer units, and each of those units already depends on a 3×3 region of the original image. With stride one, the second-layer unit has an effective receptive field of 5×5 pixels. A third such layer reaches 7×7 , and so on.

Stride and pooling make receptive fields grow faster because one step in a later feature map corresponds to several pixels in the original image.

This gives depth a second meaning beyond Week 3’s nonlinear representational power. In a CNN, depth also controls the spatial scale at which information can interact.

Early layers can respond to local detail. Deeper layers can combine information over larger portions of the image. Eventually a representation can summarise the entire image while having been built through repeated local operations.

ACTION — create this figure

Draw three stacked convolutional layers with 3x3 kernels. Highlight one unit in layer 1 and its 3x3 input receptive field, one unit in layer 2 and its 5x5 effective receptive field, and one unit in layer 3 and its 7x7 effective receptive field. Add a second mini-example showing how stride-2 downsampling makes the effective receptive field grow faster.

Figure 6.3: Local computation does not imply globally local information. Receptive fields expand as convolutional layers are composed, allowing deeper features to integrate evidence over progressively larger spatial regions.

6.8 Pooling: useful forgetting

CNNs often reduce spatial resolution as features become more abstract. Historically, a common mechanism is *pooling*.

Max pooling replaces a local region by its largest activation. Average pooling replaces it by the mean. Both operations summarise several nearby positions with fewer numbers.

This can be useful. If a feature means “a vertical edge is present somewhere in this small region,” then retaining the exact pixel at which the strongest response occurred may be unnecessary. Pooling reduces computation, increases the effective receptive field of later units, and makes the representation less sensitive to small translations.

But compression is information loss.

Imagine two feature maps. Each contains one strong activation of value 10, but one is on the left and the other is on the right. Global max pooling maps both feature maps to the scalar 10. A later classifier cannot infer which side contained the activation because that distinction no longer exists in its input.

No optimiser can recover information that the representation has made identical.

This is the same representational logic we encountered in Week 3 and Week 4. If two inputs become the same hidden representation but require different outputs, a downstream head cannot separate them. The failure is upstream of the head.

Key idea

Invariance is obtained by discarding distinctions. That is beneficial only when the discarded distinctions are irrelevant to the task.

Global pooling is therefore natural for some classification tasks and disastrous for tasks requiring precise localisation. Detection, segmentation and image reconstruction need more spatial information than a single whole-image vector can provide at every stage.

Modern CNNs often use strided convolutions as an alternative or complement to explicit pooling. The trade-off remains the same: spatial compression makes representation cheaper and often more robust, but each reduction asks what detail can safely be forgotten.

6.9 The architecture chooses what must be relearned

We can now state the inductive bias more precisely.

A flattened MLP gives each input location its own set of weights. If a motif appears only on the left during training, the weights connected to right-side pixels may receive no evidence about that motif. Generalisation to the right must be inferred indirectly or learned from additional examples.

A CNN ties those positions together through shared kernels. Learning the motif on the left immediately changes the detector used on the right. The architecture has converted a generalisation question into a parameter-sharing rule.

This is powerful when the assumption is appropriate. It can also be wrong.

Absolute position sometimes matters. In radiology, anatomy has a meaningful spatial arrangement. In documents, something printed near the top may play a different role from the same pattern near the bottom. In traffic scenes, sky-like texture at the bottom of an image may signal something unusual. Convolution does not erase position automatically, especially after padding and later layers, but pure location independence is not universally desirable.

This is why the correct lesson is not “encode as much invariance as possible.” It is “encode biases that reflect useful regularities while preserving distinctions the task may need.”

Data and inductive bias can also compensate for one another. If a weakly biased model sees the motif at every possible location, it can learn location-robust behaviour explicitly. If data are scarce or unevenly distributed, a well-matched architectural prior becomes more valuable.

6.10 A short history of convolutional architecture ideas

CNN history is useful when it is read as a sequence of design problems rather than a list of blueprints.

6.10.1 LeNet: local features and shared weights

Early convolutional systems for handwritten character recognition already contained the pattern that still matters today: convolutional feature extraction, spatial reduction and a final classifier. LeNet-5 demonstrated that learned local filters and shared weights could be trained end to end for document recognition (LeCun, Bottou, et al. 1998).

The important lesson is not the exact number of layers. It is the separation between spatial feature extraction and final prediction.

6.10.2 AlexNet: scale, compute and deep visual learning

AlexNet’s ImageNet result made large convolutional networks central to modern computer vision (Krizhevsky et al. 2012). The change was not one isolated theoretical invention. Larger labelled datasets, GPUs, ReLU activations, data augmentation and dropout made deeper convolutional training effective at a scale that transformed the field.

The lesson for this module is that architecture, optimisation, regularisation, data and hardware interact. A good inductive bias does not remove the need for engineering.

6.10.3 VGG: repeated small convolutions

VGG explored the idea that depth could be built from a very regular recipe of small convolutions (Simonyan and Zisserman 2015). Repeated 3×3 layers build larger effective receptive fields while inserting nonlinearities between stages.

This reinforces a theme from Week 3: composing simple transformations can be more expressive than replacing them with one large operation, especially when nonlinearities intervene.

6.10.4 ResNet: make depth easier to optimise

As networks became deeper, adding layers did not automatically improve trainable models. Residual networks introduced skip connections of the form

$$\mathbf{h}_{\ell+1} = \mathbf{h}_{\ell} + F(\mathbf{h}_{\ell}), \quad (6.4)$$

so a block learns a change to its input rather than an entirely new representation from scratch (He et al. 2016).

The identity path gives information and gradients a direct route through the block. This connects directly to Week 4: architecture can influence optimisation by changing how easily signals propagate, not only by changing what functions are representable.

Architecture	Design idea	What to remember
LeNet	convolution + subsampling + head	local learned features and shared spatial weights
AlexNet	deeper CNN at ImageNet scale	architecture succeeds together with data, compute and training engineering
VGG	repeated small convolutions	simple local operations compose into larger receptive fields
ResNet	residual/skip connections	deeper computation is easier when information and gradients have short paths

Table 6.1: Canonical CNNs are most useful as examples of architectural ideas, not as layer configurations to memorise.

Modern visual systems include residual CNNs, vision transformers and many hybrids. The enduring question is not whether a model is called a CNN. It is which symmetries, spatial interactions and parameter-sharing patterns its architecture assumes.

6.11 Pretraining and transfer: reusing the encoder

Suppose we have only a few thousand labelled examples for a new visual task. Training a large encoder from random initialisation may be wasteful. A common alternative is to start from an encoder trained on a larger source task.

There are two basic strategies.

In *feature extraction*, the pretrained encoder is frozen. We compute representations and train only a new head. This is cheap and protects the source representation from drifting.

In *fine-tuning*, some or all encoder parameters are updated on the new task. This can adapt the representation to the target domain, but Week 4's lesson applies immediately: a useful pretrained component can be damaged if the new objective or learning dynamics pull it away from the behaviour that made it useful.

Transfer learning works when the source representation contains regularities that the target task can reuse. Early visual filters are often broadly useful because edges and textures recur across many image domains, while later features can be more task-specific.

The practical question is therefore not simply “pretrained or not?” It is:

Which representation should be inherited, which layers should adapt, and what diagnostic tells us whether useful information has been preserved?

This question will recur in later weeks when pretrained language and image components become part of a larger system.

6.12 Autoencoders: learning a visual bottleneck without labels

So far, our encoder has usually been trained because a supervised task required a prediction. There is another route: train the representation by asking it to preserve enough information to reconstruct its own input.

An *autoencoder* contains an encoder E_θ and decoder D_ϕ :

$$\mathbf{z} = E_\theta(X), \quad (6.5)$$

$$\hat{X} = D_\phi(\mathbf{z}). \quad (6.6)$$

The latent vector $\mathbf{z}$ is a bottleneck between the input and reconstruction.

The training objective might be

$$\mathcal{L}_{\text{recon}} = \frac{1}{N} \sum_{i=1}^N \|D_\phi(E_\theta(X_i)) - X_i\|_1. \quad (6.7)$$

The model receives no class label. The input itself supplies the target.

If the bottleneck is genuinely restrictive, the encoder cannot merely copy every pixel independently. It must construct a compact code from which the decoder can recover important visual detail.

This does not mean that every latent dimension becomes a human-interpretable concept. Nor does low reconstruction error guarantee that distances in latent space are semantically meaningful. Reconstruction rewards information useful for reconstructing pixels, which may include colour, background and low-level texture that a later semantic task considers irrelevant.

The value of reconstruction is more modest and very concrete: it tests whether the latent remains input-dependent and contains enough information for the decoder to recover the frame.

PyTorch: a small convolutional encoder

```
class Encoder(nn.Module):
    def __init__(self, latent_dim=128):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, stride=2, padding=1), nn.ReLU(),
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=2, padding=1), nn.ReLU(),
        )
        self.to_latent = nn.Linear(flat_dim, latent_dim)
```

```

self.norm = nn.LayerNorm(latent_dim)

def forward(self, x):
    h = self.features(x)
    z = self.to_latent(h.flatten(1))
    return self.norm(z)

```

The decoder is essential during autoencoder training because it turns “does the latent preserve the image?” into a measurable reconstruction problem. Once a useful encoder has been learned, later systems may discard the decoder and operate directly on z .

6.12.1 The bottleneck is a design decision

A bottleneck can be too small. If the latent cannot preserve sufficient information, reconstruction will remain poor no matter how long the optimiser runs.

It can also fail dynamically. Week 4 showed how inactive ReLUs can collapse variation. A nominally 256-dimensional vector is not useful if most dimensions are constant across all images.

Useful diagnostics therefore include both the reconstruction error and the variation of latent coordinates across examples. If

$$\text{std}(z_j) \approx 0$$

for many coordinates j , the effective representation may be much smaller than its declared dimensionality.

Latent interpolation provides a different kind of probe. Given two encoded images z_a and z_b , consider

$$z(\alpha) = (1 - \alpha)z_a + \alpha z_b, \quad 0 \leq \alpha \leq 1.$$

Decoding intermediate points can reveal whether nearby latent positions produce gradual or abrupt visual changes. This is descriptive evidence about the representation’s geometry, not proof that Euclidean distance corresponds to semantic similarity.

Optional coding — what survives the bottleneck?

Train a small convolutional autoencoder on a simple image dataset. After training, choose several held-out images and zero 10%, 25% and 50% of latent coordinates before decoding.

Compare reconstruction error and visible changes. Repeat with different randomly selected coordinate subsets. Does damage increase smoothly? Do some subsets matter more than others? What evidence would support the claim that information is distributed rather than stored in one dedicated unit?

6.13 Application to the architecture: a visual currency for StoryReasoning

Application to the architecture

The StoryReasoning system cannot conveniently reason over all $60 \times 125 \times 3$ pixel values of every frame at every later stage. Its first learned visual component is therefore a convolutional autoencoder.

The architecture narrative uses a four-block strided CNN encoder. A 60×125 frame is progressively compressed to a spatial tensor of roughly $4 \times 8 \times 256$, projected to a 256-dimensional latent vector, and normalised with LayerNorm. A mirrored decoder maps the latent back to pixels.

This component is pretrained using frame reconstruction alone. Its validation L1 reaches about 0.040, compared with roughly 0.152 for the constant median-image reconstruction floor. That difference establishes something important before any sequence model exists: the encoder's latent contains enough input-specific information for the decoder to recover the frame far better than a generic image.

The earlier 16-dimensional ReLU bottleneck is an equally important result. Many units became inactive and the representation nearly died. Increasing the latent width to 256 and using LayerNorm produced stable latent spread instead. This is a concrete meeting point between Week 4 and Week 5: an architectural bottleneck is useful only if training preserves variation through it.

The autoencoder does *not* solve next-frame prediction. It solves a representation problem. Once frames have become vectors, the system can compare them, retrieve visually similar frames and later pass sequences of latents to temporal models. The practical Week 5 retrieval experiment deliberately freezes the encoder and shows the limit clearly: visual similarity works well for near-copy continuations but cannot reason through a camera cut merely because the representation reconstructs well.

The number 0.040 is also a useful ceiling for later decoder-based image prediction. Even if a future model predicted the exact target latent, the pretrained decoder would still reconstruct the target with about this error. Prediction error and reconstruction error are therefore different sources of imperfection and should be measured separately.

This is a recurring engineering pattern: separate a difficult system into measurable components. Before asking whether temporal reasoning predicts the future correctly, establish whether the visual encoder can represent the present correctly.

6.14 Representation is not reasoning

Once an image is represented by a vector, many operations become easy. We can compute distances, nearest neighbours, clusters or linear probes. A retrieval system can rank candidate images by latent similarity.

But a good representation does not automatically contain the computation required by a new task.

Imagine four movie frames followed by a shot cut. A latent encoder may represent each frame faithfully. The true fifth frame may nevertheless be visually distant from every context frame. Choosing the nearest latent then fails for a principled reason: the rule uses visual similarity when the problem requires temporal or narrative information.

This distinction protects us from a common diagnostic mistake. If latent retrieval performs poorly, we should not immediately conclude that the encoder is poor. We should first

ask whether the retrieval rule asks the representation to solve something it was never trained or structured to infer.

Conversely, excellent reconstruction does not prove semantic usefulness. An autoencoder can preserve low-level detail while organising latent space in a way that is awkward for classification or retrieval. The representation must always be judged against the downstream question.

The Week 5 practical therefore ends at a deliberate boundary. Students now possess a visual currency: frame $\rightarrow$ latent vector. They do not yet possess a mechanism that combines four such vectors over time. That missing operation belongs to later sequence-model chapters.

6.15 What architecture has bought us

Convolution has changed the learning problem in several ways at once.

- **Locality** restricts early computation to nearby values, matching common image structure.
- **Weight sharing** lets evidence for one detector accumulate across spatial positions.
- **Equivariance** makes translated patterns produce correspondingly translated feature responses.
- **Downsampling** trades spatial precision for cheaper, broader representations.
- **Depth** combines local features across progressively larger receptive fields.
- **Encoder/head separation** makes representations reusable across tasks.
- **Reconstruction pretraining** provides a target even when task labels are absent and gives a direct diagnostic of information preservation.

None of these is automatically good. Locality can delay long-range interactions. Weight sharing can be inappropriate when absolute position is essential. Pooling can destroy localisation. Reconstruction can preserve the wrong details for a semantic task. A pretrained encoder can drift during fine-tuning.

The common theme is that every design choice changes which distinctions survive and which solutions are easy to discover.

Check your understanding

For each of the following tasks, decide whether strong translation invariance is desirable, undesirable, or somewhere in between: image-level species classification; locating a tumour in a scan; recognising a traffic sign; predicting the pixel coordinates of a face; retrieving visually similar movie frames. Justify the answer in terms of information the representation must preserve.

6.16 A practical reading of a CNN

When you encounter a convolutional architecture, resist the temptation to read it as a sequence of API calls. Ask the same mechanistic questions we have used throughout the module.

What is the receptive field at each stage? Where is spatial resolution reduced? How many feature channels are created? Which weights are shared? At what point is position information discarded? What representation reaches the head? What loss shapes that representation? Which baseline tells us whether the learned features are useful? If the encoder was pretrained, what measurement tells us that fine-tuning has not destroyed it?

A diagram of boxes is not yet an explanation. The architecture becomes meaningful when each operation can be related to an assumption about information.

Optional design exercise — choose the bias before the model

You have three datasets:

1. microscopic texture crops where class depends on a repeated local pattern;
2. chessboard images where the task is to identify the exact square containing a piece;
3. satellite images where a long river may span most of the frame.

For each dataset, propose an encoder design at the level of locality, downsampling and spatial aggregation. Which assumptions of a standard image-classification CNN are helpful? Which could destroy information needed by the task? Do not specify a large named architecture; justify the inductive biases.

6.17 Summary

Week 4 taught us that successful optimisation is not the same as successful learning. Week 5 adds another layer: even before optimisation begins, the architecture determines which regularities are easy to learn.

A convolutional layer uses local receptive fields and shared weights. This makes recurring spatial patterns economical to represent and naturally produces translation-equivariant feature maps. Multiple channels learn multiple kinds of local features; depth grows the effective receptive field and composes those features into larger structures. Stride and pooling compress space, often increasing robustness while deliberately discarding localisation information.

These choices are examples of inductive bias. They can improve generalisation when they match the data and hurt when they remove distinctions the task requires.

The encoder/head view then lets us separate visual representation from downstream prediction. A convolutional autoencoder can learn a compact latent through reconstruction, and reconstruction error plus latent spread provides direct evidence that information survived the bottleneck. In StoryReasoning, this creates the first reusable learned representation of a movie frame.

The next question is no longer merely whether the representation contains information. It is how we obtain evidence about *which* information a model uses and whether our explanation of that use is faithful.

Chapter 7

Explanation as evidence

A neural network can make a correct prediction for a reason we dislike, an incorrect prediction for a reason we understand, or a useful prediction for a reason we cannot easily describe. Once models become large enough that their internal computations are not readable line by line, a new problem appears alongside prediction itself: *what evidence would justify a claim about why the model behaved as it did?*

This question is more difficult than it first appears. A colourful heatmap can look persuasive. A hidden unit can correlate with a concept. A linear probe can decode an attribute. An attention-like weight can become large. None of these observations, by itself, proves that the corresponding feature caused the prediction.

Week 4 introduced a diagnostic habit: a symptom is not a diagnosis. This week extends the same discipline to explanation. We will distinguish several questions that are often collapsed into the word *importance*: Is the output locally sensitive to a feature? Does changing the feature alter the prediction? Can information about the feature be decoded from a hidden representation? Is that information actually used by the downstream computation? Can a simpler model approximate the network near one example? What change would have altered the decision?

Different methods answer different questions.

Key idea

An explanation is not a picture attached to a prediction. It is a claim about a model, and the strength of the explanation depends on whether the evidence actually supports that claim. Whenever possible, move from inspection to a matched control or intervention.

The aim of this chapter is therefore not to find one universally correct explanation method. It is to learn how to state explanation claims precisely enough that they can be tested.

7.1 What are we trying to explain?

Before choosing a method, we should name the object of explanation. Several distinctions are useful because they prevent methods from being compared as if they were interchangeable.

7.1.1 A prediction, a representation, or a mechanism

A *prediction explanation* asks why the model produced a particular output for a particular input. Why was this image classified as an eight? Why did this frame receive a high retrieval score?

A *representation explanation* asks what information is present in an internal state. Does a visual embedding preserve whether a person is present? Does a hidden vector retain colour, position, identity or some nuisance variable?

A *mechanistic explanation* asks how internal computations transform information into behaviour. Which path carries the relevant signal? Which intermediate variables matter? Which components are necessary for the output?

These are progressively stronger questions. Knowing that character presence is linearly decodable from an image embedding does not yet tell us whether a later predictor uses that information. Knowing that the final prediction changes when a hidden direction is ablated is stronger evidence of use, but still may not reveal the complete mechanism.

7.1.2 Local and global explanations

A *local* explanation concerns one input or a small neighbourhood around it. A saliency map for one image is local. A counterfactual saying that one patient’s prediction would change if one measurement changed is local.

A *global* explanation concerns behaviour across a population of inputs. Permutation importance measured over a whole test set is global. A statement such as “the model generally relies on background texture” is global and requires evidence from many examples.

A model can use different evidence on different examples. A local explanation should therefore not be silently promoted into a global claim.

7.1.3 Model-specific and model-agnostic methods

Some methods require access to the model’s internal derivatives or activations. Gradient saliency is model-specific in this sense: we must differentiate through the model.

Other methods treat the predictor as a function that can be queried. We can perturb inputs, observe outputs and fit a surrogate without knowing the internal architecture. These methods are more model-agnostic, although they still depend on choices about how inputs are perturbed and which outputs are measured.

7.1.4 Post-hoc and intrinsically interpretable models

A *post-hoc* explanation is produced after a model has been trained. Most attribution maps, local surrogates and probes belong here.

An *intrinsically interpretable* model is designed so that parts of the prediction process have an intended human-readable meaning. A small sparse linear model is a simple example. Some architectures introduce explicit concepts, prototypes, rules or sparse features for similar reasons.

Intrinsic structure can make inspection easier, but interpretability is not automatic. A named component may not behave as intended, and a sparse feature may still combine several concepts. Inspectability remains a hypothesis until behaviour supports it.

7.2 Sensitivity is not the same as importance

Consider a scalar model output $F(\mathbf{x})$, perhaps a class logit. A natural first question is how that output changes when one input coordinate changes slightly. The derivative

$$\frac{\partial F}{\partial x_j}$$

measures precisely this local sensitivity.

ACTION — create this figure

Create a compact 2-by-2 map of explanation questions. Horizontal axis: local to global. Vertical axis: output behaviour to internal representation/mechanism. Place examples such as saliency/IG, counterfactual, permutation importance, linear probe, ablation/intervention and global concept statistics in the appropriate regions. Add a note that methods overlap and answer different questions.

Figure 7.1: Interpretability is not one task. Explanation methods differ in whether they concern individual predictions or population behaviour, and whether they describe outputs, representations or internal mechanisms.

For an image, the gradient with respect to every pixel gives a saliency map:

$$\nabla_{\mathbf{x}} F(\mathbf{x}).$$

Large magnitudes indicate coordinates where a small local perturbation would change the chosen scalar strongly. This is useful information. But the phrase *small local perturbation* matters.

A feature can have been essential in moving the model into its present state while having a tiny derivative at the endpoint. Saturating nonlinearities give a simple example. If a sigmoid probability is already extremely close to one, changing the logit slightly may barely change the probability. The probability gradient can therefore be near zero even when one input feature was decisive in making the logit large.

This is why an explanation should identify the scalar being differentiated. The gradient of a logit and the gradient of the corresponding probability are not the same explanation.

PyTorch: gradient saliency explains a chosen scalar

```
x = image.clone().requires_grad_(True)
logits = model(x)
score = logits[0, target_class] # state what is explained
score.backward()
saliency = x.grad.abs()
```

The code is short; the interpretation is not. The gradient says what would happen under an infinitesimal change at this particular input. It does not directly tell us what caused the model to arrive at the current output, what would happen under a large meaningful change, or whether the highlighted pixels form a coherent concept.

Check your understanding

Suppose a pixel has a gradient of zero for the class probability. Give two different explanations for why that might happen. Which additional measurement would distinguish them?

7.3 Integrated Gradients: explaining a change from a reference

One response to the locality problem is to accumulate gradients along a path from a reference input to the observed input. Integrated Gradients (IG) does exactly this (Sundararajan et al. 2017).

Let $\mathbf{x}'$ be a baseline and $\mathbf{x}$ the input being explained. Along the straight-line path

$$\mathbf{x}(\alpha) = \mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'), \quad 0 \leq \alpha \leq 1,$$

the attribution to coordinate j is

$$\text{IG}_j(\mathbf{x}) = (x_j - x'_j) \int_0^1 \frac{\partial F(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial x_j} d\alpha. \quad (7.1)$$

In practice the integral is approximated with a finite number of steps e.g., via a Riemann approximation.

IG changes the question. Vanilla saliency asks: *what is the local sensitivity here?* IG asks: *which coordinates account for the change in the chosen scalar between this reference and this input along this path?*

Under appropriate conditions, the attributions approximately satisfy a completeness relation:

$$\sum_j \text{IG}_j(\mathbf{x}) \approx F(\mathbf{x}) - F(\mathbf{x}').$$

This gives us a useful numerical check.

But a new modelling choice has appeared: the baseline. An all-black image may be a natural absence-of-signal reference in one domain and an unrealistic image far outside the data distribution in another. A zero vector may mean “no feature” after one preprocessing pipeline and “an average feature value” after standardisation. Different defensible baselines can produce different attributions because they explain different changes.

This disagreement need not mean that one computation is wrong. It may mean that two different reference questions were asked.

7.3.1 Expected Gradients

Expected Gradients broadens the reference idea by averaging attribution paths over a distribution of baseline examples rather than relying on one fixed reference (Erion et al. 2021). Conceptually, instead of asking “how did the model move from this one baseline to this input?”, we ask how the input differs from many plausible reference points drawn from a background distribution.

This can reduce sensitivity to one arbitrary baseline, but it does not remove the need for judgement. We must still decide what reference distribution is relevant. A background sample from the wrong population can answer the wrong question very consistently.

Optional mathematics — completeness for a one-dimensional path

For a differentiable scalar function $F(x)$, show that

$$(x - x') \int_0^1 F'(x' + \alpha(x - x')) d\alpha = F(x) - F(x').$$

Use the substitution $u = x' + \alpha(x - x')$. Then explain why the multidimensional IG completeness property can be viewed as the same fundamental-theorem-of-calculus idea applied along a path.

7.4 Perturbation turns an explanation into a test

Attribution maps are descriptive until we ask whether changing the highlighted evidence changes behaviour.

Suppose a method assigns large attribution to a patch of an image. We can occlude that patch and measure the change in the model’s logit. This is stronger evidence than merely observing that the patch is bright on the heatmap.

Yet even perturbation requires controls. Removing any sufficiently large region of an image may damage a classifier. If we erase the top 10% of attributed pixels and the prediction changes, we should compare that result with removing the same number of randomly selected pixels or low-attribution pixels.

A matched intervention therefore has the form

targeted change versus comparable control change.

The comparison asks whether the explanation identifies features whose removal matters *more than an equally destructive generic perturbation*.

7.4.1 Occlusion and feature perturbation

For images, we may mask pixels, patches or semantic regions. For tabular data, we may replace a feature with a reference value. For language, we might remove, replace or mask tokens, although the replacement itself can produce unnatural inputs.

This exposes a central limitation of perturbation methods: the altered example may leave the data distribution. A black square placed over a face is not a naturally occurring face. Setting one correlated laboratory measurement to an impossible value while holding all others fixed can create an invalid patient record.

The behavioural change is still real evidence about the model under that intervention, but we should not automatically interpret it as evidence about a plausible real-world change.

7.4.2 Permutation importance

Permutation importance is a useful global version of the same idea. For feature x_j , we shuffle that feature across examples while leaving the rest of each row unchanged. The feature’s values remain distributed as before, but their association with each example is broken. If test performance falls substantially, the model depended on information carried by that feature.

The method asks a global question:

How much performance is lost when this feature–example pairing is destroyed?

Correlation complicates the answer. If two features carry nearly redundant information, permuting one may have little effect because the other still supplies the signal. Conversely, independently shuffling one feature can create combinations that were rare or impossible in the original data. A small permutation importance is therefore not proof of irrelevance.

7.5 Local surrogate models: approximate the black box nearby

Another strategy is not to inspect derivatives at all. Instead, query the model near an input and fit a simpler model that approximates its behaviour in that neighbourhood.

LIME is a canonical example (Ribeiro et al. 2016). Suppose the black-box predictor is F , the input of interest is $\mathbf{x}_0$, and we create perturbed points $\mathbf{x}^{(i)}$. We assign larger weights to

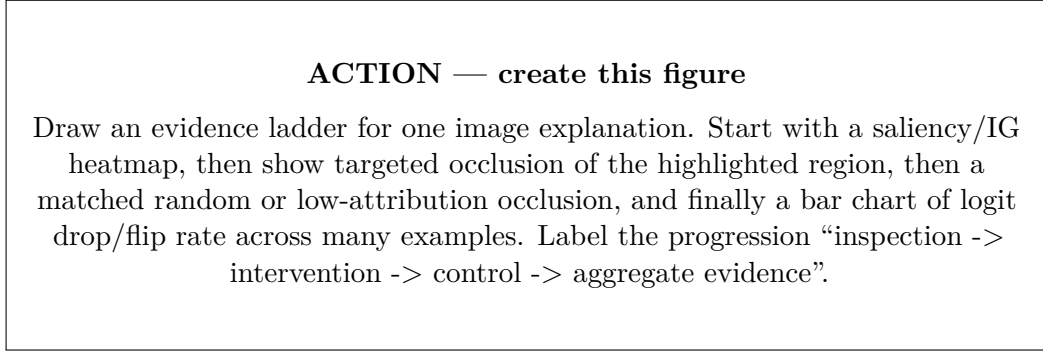


Figure 7.2: A visually plausible attribution becomes more convincing when it predicts a behavioural consequence that exceeds matched control perturbations across multiple examples.

perturbations close to $\mathbf{x}_0$, then fit an interpretable surrogate g , often a sparse linear model, by approximately solving

$$\min_g \sum_i \pi_{\mathbf{x}_0}(\mathbf{x}^{(i)}) \left(F(\mathbf{x}^{(i)}) - g(\mathbf{x}^{(i)}) \right)^2 + \Omega(g),$$

where $\pi_{\mathbf{x}_0}$ gives locality weights and $\Omega(g)$ favours a simple surrogate.

The coefficients of g then describe the black box *as approximated locally*.

Two cautions follow immediately.

First, the surrogate explains the trained model, not necessarily the true data-generating process. If the network has learned a shortcut, a faithful surrogate should reproduce that shortcut.

Second, locality is a modelling choice. A very narrow neighbourhood may yield an accurate but fragile explanation. A broad neighbourhood may smooth over important nonlinear behaviour and become a poor description of the specific example.

For this reason, local surrogate explanations should report their local fidelity. An explanation with a simple coefficient table but a poor fit to the model is not a successful approximation.

7.6 SHAP: contributions through coalitions

Shapley values originate from cooperative game theory. The basic idea is to assign each feature a contribution by asking how much it changes the prediction when added to many possible subsets, or *coalitions*, of the other features. SHAP adapts this idea to model explanation (Lundberg and Lee 2017).

For a feature j , the Shapley value is conceptually an average marginal contribution

$$\phi_j = \mathbb{E}_S [F(S \cup \{j\}) - F(S)],$$

with carefully chosen weights over subsets S .

This provides an appealing additive decomposition:

$$F(\mathbf{x}) \approx \phi_0 + \sum_j \phi_j,$$

where ϕ_0 is a reference prediction.

The difficult phrase is hidden in the notation $F(S)$: what does it mean for a feature to be “missing”? Implementations must define a background distribution or conditional

expectation. With correlated features, different choices about missingness can yield different contributions. Exact Shapley computation also grows exponentially with the number of features, so practical methods use approximations or exploit model structure.

SHAP is therefore not a machine for revealing a unique metaphysical importance. It is a principled answer to a particular additive attribution question under a particular treatment of missing features.

Check your understanding

Two highly correlated features encode nearly the same signal. Why might permutation importance, a local LIME surrogate and SHAP assign them different apparent importance? The answer should refer to the question each method asks, not merely say that the methods are “different algorithms”.

7.7 Counterfactual explanations

Attribution asks how credit is distributed. Counterfactual explanation asks a different question:

What change to this input would have changed the model’s decision?

For an input $\mathbf{x}$, one may seek

$$\mathbf{x}^* = \arg \min_{\tilde{\mathbf{x}}} d(\tilde{\mathbf{x}}, \mathbf{x})$$

subject to the model producing a desired alternative output.

This is attractive because it describes a behavioural boundary in concrete terms. But “smallest change” is only meaningful after defining a distance and constraints. A mathematically tiny counterfactual may be impossible in the real domain. Age cannot decrease. A medical variable cannot be changed independently of every correlated variable. An image perturbation that fools the network may not correspond to any plausible object.

Good counterfactual design therefore includes feasibility and plausibility constraints, not merely optimisation against the classifier.

Counterfactuals also explain the decision boundary, not necessarily the causal structure of the real world. If changing a postcode changes a loan model’s decision, that demonstrates model dependence on postcode; it does not establish that changing a person’s postcode would causally change their economic risk.

7.8 What is inside a representation? Probes

Week 5 separated an encoder from a task head. Once we have a hidden representation $\mathbf{h} = E(\mathbf{x})$, we can ask what information is available in $\mathbf{h}$.

A *probe* is a deliberately restricted predictor trained on frozen representations. A linear probe takes the form

$$\hat{y}_{\text{probe}} = \mathbf{W}\mathbf{h} + \mathbf{b}.$$

If a linear probe predicts an attribute well on held-out data, then that attribute is linearly decodable from the representation.

This is useful because it separates representation learning from the probe. We do not alter the encoder; we test what a simple readout can recover.

But decodability is not use.

A representation may retain information because it was useful for another task, because it is correlated with useful information, or simply because the encoder had enough capacity not to discard it. The downstream classifier may ignore it completely.

Key idea

A successful probe supports the claim “this information is recoverable from the representation by this class of probe.” It does not by itself support the stronger claim “the original predictor uses this information to make its decision.”

Probe design also requires controls. A high-capacity probe can sometimes learn structure even from weak representations. Shuffled-label controls test whether the probe pipeline is producing spurious performance. Comparisons with random or untrained encoders help determine how much decodability comes from learned representation structure rather than from architecture alone.

7.8.1 From probe to intervention

To test use, we need to change the relevant information and observe the original predictor.

One option is input intervention: change the suspected feature while preserving other variables as carefully as possible, then measure the prediction change.

Another is activation intervention: alter the hidden representation directly. We might zero selected coordinates, replace a direction, patch in an activation from another example, or remove a subspace identified by a probe.

If the original output changes systematically under a targeted intervention, we have stronger evidence that the intervened information participates in the computation.

Even then, care is required. Information can be distributed over many coordinates. Zeroing one neuron may do little because other units encode redundant information. Conversely, a broad ablation may damage the representation so severely that many behaviours fail at once. The best interventions change as little as possible while targeting the claim of interest.

PyTorch: a simple hidden-state intervention

```
h = encoder(x)
logits = head(h)

h_ablated = h.clone()
h_ablated[:, dims_to_test] = 0.0
logits_ablated = head(h_ablated)

effect = (logits_ablated - logits).abs().mean()
```

The code measures a behavioural consequence. The difficult part is choosing `dims_to_test` and a control intervention that make the result interpretable.

7.9 Faithfulness: an explanation should predict behaviour

The word *faithful* is useful because it asks whether an explanation tracks the model’s actual computation rather than merely appearing plausible to a human.

Suppose a heatmap consistently highlights the object in an image. That looks reassuring, but a sanity check might reveal that similar heatmaps appear after model parameters are randomised. In that case the visual structure may come partly from the input or explanation

ACTION — create this figure

Create a three-column diagram labelled “present”, “decodable”, and “used”.

Column 1 shows a hidden representation containing several mixed signals.

Column 2 shows a linear probe successfully reading one signal, with a shuffled-label control failing. Column 3 shows a targeted intervention on that signal and the original model output either changing or remaining stable.

Emphasise that each column supports a stronger claim.

Figure 7.3: Representation analysis should distinguish information being present, information being decodable by a chosen probe, and information being causally relevant to the original predictor.

algorithm rather than from the learned decision function. Explanation methods therefore need controls just as predictive models do (Adebayo et al. 2018).

Useful controls include:

- randomising labels or model parameters where appropriate;
- comparing targeted perturbations with matched random perturbations;
- comparing learned representations with random-weight representations;
- checking whether a local surrogate actually matches model outputs locally;
- repeating explanations across multiple examples rather than selecting one attractive case;
- measuring stability under small irrelevant changes;
- testing whether an explanation predicts the effect of an intervention.

The aim is not to demand impossible certainty. It is to calibrate the claim to the evidence.

7.10 Attention, gates and other inspectable quantities

Later chapters will introduce mechanisms that explicitly assign weights to pieces of information. Such weights are tempting to read as explanations. If one frame receives weight 0.7 and another receives 0.1, it is natural to say that the first frame “mattered more.”

That conclusion may be wrong.

A weight can reflect a positional prior, a normalisation constraint, or one part of a computation whose downstream transformations change the final effect. Alternative weight patterns may sometimes produce similar outputs. For these reasons, attention weights should not automatically be equated with explanations of model behaviour (Jain and Wallace 2019).

The same warning applies to gates, similarity scores and retrieval ranks. Inspectability is valuable: it gives us hypotheses about where to intervene. But a hypothesis becomes stronger when changing the supposedly important component changes the behaviour in the predicted direction.

Application to the architecture

The StoryReasoning system eventually exposes several quantities that look explanatory: frame-selection weights, gates, latent similarities, retrieval ranks and per-character predictions. Week 6 gives us a rule for interpreting them: *read them as hypotheses first*.

A particularly instructive later case is frame selection. A content-dependent selector was intended to choose which of four input frames was useful for the next-frame prediction. The learned weights initially favoured a positional pattern: frame 3 was often largest. On near-copy windows, however, the selector chose the actually closest input only about 29% of the time, barely above the 25% four-way chance level. The weights were inspectable, but they did not yet justify the claim that the model was selecting the relevant frame.

When explicit supervision was added, closest-input accuracy rose to about 52%. Yet coupling that selector to the shared sequence state damaged text retrieval severely. The lesson is not that attention or selection weights are useless. It is that an interpretable-looking internal variable must be checked against the behaviour it is supposed to explain, and against side effects elsewhere in the system.

The same logic applies to the visual representation introduced in Week 5. A probe may show that character presence is decodable from a frozen frame embedding. That is evidence about representation content. Only a later intervention on a predictor can establish whether the predictor actually uses that character information.

7.11 Intrinsic interpretability and sparse representations

Post-hoc methods start with a complex representation and attempt to explain it afterward. Another strategy is to build representations that are easier to inspect from the beginning.

Sparsity is one recurring idea. If each example activates only a small subset of features, those features may be easier to associate with recurring concepts than a dense vector in which every coordinate changes slightly for every input.

A sparse autoencoder takes an existing representation $\mathbf{h}$, maps it to a usually larger feature vector $\mathbf{a}$, encourages most components of $\mathbf{a}$ to be zero or small, and reconstructs $\mathbf{h}$ from those features:

$$\mathbf{a} = E_s(\mathbf{h}), \quad \hat{\mathbf{h}} = D_s(\mathbf{a}).$$

The loss combines reconstruction with a sparsity penalty.

The hope is that sparse features may align more cleanly with recurring directions of variation in the original dense representation. But sparsity does not guarantee semantic meaning. A sparse feature can still mix several unrelated causes, and different training runs can discover different decompositions. Week 10 will return to the geometric reason such decompositions are interesting when we study distributed representations and superposition.

For now, sparse representation methods illustrate an important design principle: interpretability can influence the representation objective itself, not only the analysis performed afterward.

7.12 Transfer learning as a question about reusable information

Week 5 introduced pretrained encoders as a practical way to reuse visual features. Interpretability gives us a sharper question: *what information has pretraining made easily accessible?*

Suppose a frozen ImageNet encoder maps StoryReasoning frames to vectors. We can train a small linear probe to predict whether a frame contains one or several annotated characters. If the probe performs well on held-out stories, then character-related information is linearly accessible in the pretrained representation.

Several comparisons improve the claim:

- a majority-class baseline asks whether the probe beats a trivial rule;
- a random-weight encoder asks whether learned pretraining contributed beyond the architecture;
- shuffled labels ask whether the probe pipeline can manufacture performance from noise;
- held-out stories test whether decodability generalises beyond near-duplicate frames;
- nearest neighbours reveal aspects of representation geometry but do not prove mechanism.

This is transfer learning viewed through an interpretability lens. A representation transfers when useful information for a new task is already organised in a form a simple head can exploit.

Yet the same caution remains: transferability does not prove future use. A later StoryReasoning predictor may have access to character information and still ignore it because another signal is easier to optimise.

Optional coding — explanation claims as tests

Take any trained classifier with at least five input features. Choose one explanation method you can implement simply: gradient magnitude, permutation importance, or a local linear surrogate.

Write down the exact claim you think the method supports. Then design one matched intervention that could falsify that claim. For example, compare removal of the most highly ranked features with removal of the same number of randomly selected features across many test examples.

The exercise is complete only when the explanation has been converted into a behavioural prediction.

7.13 A hierarchy of claims

We can now arrange several common interpretability statements by strength.

1. **The model is locally sensitive to this feature.** Supported by derivatives or small perturbations around the input.
2. **This feature receives attribution relative to this reference/background.** Supported by methods such as IG or SHAP, under their assumptions.

3. **This simple surrogate approximates the model in this neighbourhood.** Supported by local fit quality.
4. **This information is decodable from the representation.** Supported by a probe with appropriate controls.
5. **Changing this feature or internal signal changes the model's output.** Supported by a targeted intervention relative to controls.
6. **The model generally relies on this feature across the data distribution.** Requires aggregate intervention evidence over a relevant sample.
7. **This feature causally matters in the real world.** This is a claim about the data-generating process, not merely about the model, and usually requires evidence beyond model interpretability methods.

Moving down the list requires stronger evidence. The categories are not perfectly ordered in every setting, but the exercise exposes a common error: using evidence from an earlier statement to assert a later one.

Key idea

Interpretability becomes scientifically useful when explanations make risky predictions about model behaviour: if this feature matters, changing it in a controlled way should produce a measurable consequence. A good explanation method therefore earns trust through tests, not aesthetics.

7.14 What should you ask when shown an explanation?

When presented with an attribution map, feature ranking, probe result or internal weight, ask:

- What exactly is being explained: a logit, probability, loss, class decision, hidden representation or some other scalar?
- Is the claim local or global?
- What baseline, background distribution or neighbourhood defines the comparison?
- Does the method measure sensitivity, attribution, approximation, decodability or intervention effect?
- What matched control would reveal whether the apparent pattern is special?
- Could correlation or redundancy hide alternative routes for the same information?
- Does the intervention keep the example within a plausible data distribution?
- If the method describes a representation, has downstream use actually been tested?

These questions will remain useful long after the particular methods in this chapter have been replaced by newer ones.

7.15 Chapter summary

Week 5 asked what assumptions an architecture builds into visual learning. This week asked what information the resulting representations contain and what evidence allows us to say that the model uses it.

The central distinctions are:

- local sensitivity is not the same as global importance;
- attribution depends on the chosen output, reference and method;
- permutation and occlusion are interventions, but still require controls and plausibility checks;
- local surrogates approximate model behaviour near an example rather than reveal the true data-generating rule;
- counterfactuals describe decision-changing alternatives, not necessarily causal real-world changes;
- a probe establishes decodability, not downstream use;
- ablation and intervention strengthen claims by testing behavioural consequences;
- inspectable internal quantities such as attention weights or gates are hypotheses, not explanations by default;
- sparse or intrinsically interpretable representations can make analysis easier, but still require validation;
- transfer learning can be understood as reusable information becoming accessible to a new head.

The habit to carry forward is simple: whenever an explanation looks convincing, ask what observable consequence should follow if it is true.

Chapter 8

Language, sequence and memory

An image arrives all at once. A sentence does not. Its meaning depends not only on which symbols appear, but on the order in which they appear and on relationships that may stretch across many positions. The sentences “the dog chased the cat” and “the cat chased the dog” contain almost the same words, yet they describe different events. A representation that remembers only which words occurred has already lost something essential.

This makes sequence modelling a problem of *memory*. As each element arrives, what should be preserved from the past? What may safely be forgotten? How should the model combine what it sees now with what it remembers from before? And when the task is to generate a sequence, how does the model turn a learned distribution over possible next elements into one actual output?

These questions connect directly to the previous weeks. Week 5 asked what information an architecture makes easy to represent. Week 6 asked what evidence shows that information is present or used. This week adds time: information can be present at one position, transformed repeatedly, forgotten, or retrieved much later.

Key idea

A sequence model is not merely a network applied many times. It is a mechanism for deciding what information from earlier positions survives long enough to influence later predictions.

We will begin with language because it makes the problem concrete, but the same principles apply to audio, video, sensor streams, financial series and any task in which order matters.

8.1 Turning language into numbers

A neural network consumes numbers, while language begins as a stream of symbols. The first step is therefore representational: turn text into discrete units and then turn those units into vectors.

8.1.1 Words, tokens and subwords

A *token* is one unit presented to the model. A token need not correspond exactly to a written word. A simple word-level system might map

“the dog runs” $\longrightarrow$ [the, dog, runs],

but a subword tokenizer may divide an uncommon word into several reusable pieces. This reduces the need for a vocabulary containing every possible word form and gives rare or novel words representations assembled from familiar components.

Tokenisation therefore already introduces an inductive choice. A word-level vocabulary gives common words a single identity but needs an unknown-token mechanism for unseen words. Character-level tokenisation can represent any spelling but produces long sequences. Subword methods occupy a useful middle ground: a frequent word may remain whole while an uncommon one is decomposed into pieces.

The tokenizer assigns each token an integer identifier. The integer itself has no useful geometry. Token 412 is not intrinsically “closer” to token 413 than to token 900. The ID is an address into a representation table.

8.1.2 One-hot vectors and learned embeddings

Suppose a vocabulary contains V tokens. A one-hot representation for token i is a vector in $\mathbb{R}^V$ whose i -th coordinate is one and all others are zero. Distinct tokens are equally far apart and share no structure by construction.

An embedding layer replaces this sparse vector with a dense vector

$$\mathbf{e}_i \in \mathbb{R}^d,$$

where normally $d \ll V$. Equivalently, an embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d}$ stores one learned row per token. Looking up token i returns row $\mathbf{E}_{i,:}$.

When the embedding table is learned as part of a language task, tokens that are useful in similar contexts can acquire related vectors. This is one of the central ideas behind distributed word representations (Bengio et al. 2003; Mikolov et al. 2013). The geometry is learned because it helps the objective; semantic structure is an emergent property of that training, not something guaranteed by the word *embedding*.

PyTorch: token embeddings

```
embedding = nn.Embedding(num_embeddings=vocab_size,
                          embedding_dim=64)

# token_ids: [batch, time]
x = embedding(token_ids) # [batch, time, 64]
```

Notice the new axis: time. An image tensor had spatial dimensions. A token sequence has a position dimension. The vector at one position says which token occurred there, but the model must still combine information across positions.

8.1.3 A bag of words forgets order

Consider token embeddings $\mathbf{e}_1, \dots, \mathbf{e}_T$. If we represent a sentence by their mean,

$$\bar{\mathbf{e}} = \frac{1}{T} \sum_{t=1}^T \mathbf{e}_t,$$

then any permutation of those same token vectors produces exactly the same result. The representation is invariant to order.

That may be useful for some tasks. If the question is simply which topics or words appear, a pooled representation can work surprisingly well. But it cannot distinguish two sequences that contain the same multiset of tokens in different orders.

This gives us the first memory question:

How can a representation depend on what came before, not merely on what is present?

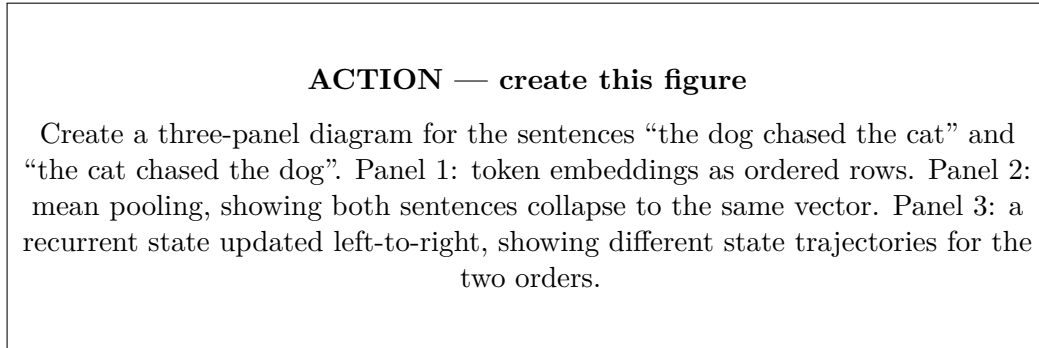


Figure 8.1: Token identity alone does not encode a sequence. An order-invariant pooling operation can destroy distinctions that a sequential update preserves.

8.2 Recurrent neural networks: a state that moves through time

A recurrent neural network (RNN) introduces a hidden state $\mathbf{h}_t$ that is updated as each input arrives. A simple recurrent update is

$$\mathbf{h}_t = \phi(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}), \quad (8.1)$$

where $\mathbf{x}_t$ is the representation of the current token and ϕ is typically a nonlinear activation such as $\tanh$.

The same parameters $\mathbf{W}_x$, $\mathbf{W}_h$ and $\mathbf{b}$ are reused at every time step. This is weight sharing through time. Instead of learning a different transformation for “position 1”, “position 2” and so on, the model learns one state-update rule and applies it repeatedly (Elman 1990).

The hidden state is therefore a learned summary of the prefix:

$$\mathbf{h}_t = f_\theta(\mathbf{x}_1, \dots, \mathbf{x}_t).$$

It is common to call this state the model’s *memory*. That language is useful, provided we do not imagine a literal storage box containing an intact copy of every previous token. The state is a fixed-size vector whose contents are continually transformed. Information can be strengthened, entangled, compressed or lost.

8.2.1 The same rule, unrolled

Although Equation (8.1) describes one update, the whole sequence can be visualised by unrolling the recurrence:

$$\mathbf{h}_0 \xrightarrow{\mathbf{x}_1} \mathbf{h}_1 \xrightarrow{\mathbf{x}_2} \mathbf{h}_2 \xrightarrow{\mathbf{x}_3} \dots \xrightarrow{\mathbf{x}_T} \mathbf{h}_T.$$

The parameters are shared, but the intermediate states differ because the input history differs.

This makes order matter automatically. If two tokens swap positions, the second token is processed from a different preceding state, and all later states may change.

For a sequence-classification problem, the final state $\mathbf{h}_T$ might feed a classifier. For next-token prediction, a head can be applied at every position. For a sequence-to-sequence task, one recurrent network may encode an input sequence and another may decode an output sequence.

PyTorch: a recurrent sequence encoder

```
gru = nn.GRU(input_size=embedding_dim,
             hidden_size=hidden_dim,
             batch_first=True)

states, h_final = gru(x)
# states: [batch, time, hidden_dim]
# h_final: [1, batch, hidden_dim]
```

The distinction between `states` and `h_final` will matter later. The full sequence of states keeps one representation per position. The final state compresses the whole history into one vector.

8.2.2 Backpropagation through time

Training an RNN still uses backpropagation. The difference is that the same transformation appears repeatedly in the computational graph. To understand how an early token or parameter influenced a later loss, the backward pass follows the chain through many recurrent steps. This is called *backpropagation through time* (BPTT).

If a loss at time T depends on an early state $\mathbf{h}_1$, the derivative contains a product of local Jacobians along the path from time 1 to time T . The idea is the same one we encountered with deep feed-forward networks, except that depth now grows with sequence length.

This immediately revives a Week 4 problem. Repeated multiplication by derivatives smaller than one can make an early influence vanish; repeated multiplication by large factors can make it explode. A recurrent network can therefore be mathematically capable of remembering a fact while being difficult to train to preserve it over a long delay.

Key idea

In a recurrent network, sequence length creates computational depth. A dependency across 100 positions creates a much longer information and gradient path than a dependency across five positions.

Optional mathematics — why repeated derivatives matter

Consider the scalar recurrence

$$h_t = \tanh(wh_{t-1}).$$

Ignoring other inputs, write $\partial h_T / \partial h_0$ as a product of local derivatives. What happens when the magnitude of most factors is below one? What happens if they are consistently greater than one?

The purpose is not to prove that every RNN must fail, but to see why a long temporal path creates the same gradient-scale problem as a very deep network.

8.3 Gating: learning what to keep and what to change

The long short-term memory network (LSTM) and gated recurrent unit (GRU) were designed to make persistent information easier to maintain across recurrent updates (Hochreiter and Schmidhuber 1997; Cho et al. 2014). Their exact gate equations differ, but the important idea is shared: the network learns how strongly to preserve old state, incorporate new evidence and discard information that is no longer useful.

For a GRU, one part of the update can be viewed schematically as

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t, \quad (8.2)$$

where $\mathbf{z}_t$ is a learned update gate between zero and one, $\tilde{\mathbf{h}}_t$ is a candidate new state, and $\odot$ denotes element-wise multiplication.

The equation is useful because it exposes a qualitatively different path. If one coordinate of $\mathbf{z}_t$ is close to zero, the corresponding old-state coordinate can pass forward with little change. If the gate is close to one, the candidate replaces it more strongly. The model can therefore learn a spectrum between remembering and updating rather than being forced to overwrite the state in the same way at every step.

An LSTM makes this persistence even more explicit through a separate cell state and learned input, output and forget gates. Students do not need to memorise every formula to understand its purpose: create controlled routes along which information and gradients can survive.

Gating is not perfect memory. A GRU trained on sequences up to length 30 may still fail badly at length 300. The architecture changes the path through which information can survive; it does not remove distribution shift, capacity limits or optimisation difficulty.

Check your understanding

A GRU performs well on a delayed-memory task up to 40 steps but degrades at 100. Which claim is justified?

1. “GRUs cannot model long-term dependencies.”
2. “This trained GRU preserved the tested information over tens of steps, but the evidence does not support arbitrary-length memory.”

The second statement is the defensible one. Memory claims need an explicit tested range.

8.4 Language modelling: predicting one token from its history

Recurrence gives us a way to summarise a prefix. A language model uses that summary to assign probabilities to what might come next.

For a token sequence $x_1, \dots, x_T$, the chain rule of probability gives the autoregressive factorisation

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{<t}), \quad (8.3)$$

where $x_{<t}$ denotes all earlier tokens.

The model’s job is therefore repeated next-token prediction. At time t , the hidden state summarises the prefix and a linear output layer produces one logit per vocabulary token. Softmax turns those logits into a categorical distribution over the next token.

This framing is powerful because no sentence-level label is needed. Every sequence supplies many training targets: after each prefix, the next observed token is the target. This is a form of self-supervision.

8.4.1 Teacher forcing

During training, we normally know the true sequence. To predict token x_t , the decoder is given the true preceding tokens $x_{<t}$. This is called *teacher forcing*.

For the sentence

[<bos>, the, dog, runs, <eos>],

a training pair can be written as

input:	<bos>	the	dog	runs
target:	the	dog	runs	<eos>

The target is simply shifted by one position.

Teacher forcing makes optimisation easier because the model is trained from correct prefixes rather than from its own early mistakes. But it also creates a discrepancy. At generation time the true future tokens are unavailable. The model must feed back tokens produced by itself, so an early mistake changes the context for all later predictions.

This is not a contradiction. Training and inference are solving related but not identical computational situations.

8.4.2 Cross-entropy and perplexity

At each position the model predicts a probability for the observed next token. Cross-entropy penalises low probability on that token. If the average token cross-entropy is L , a common summary is the *perplexity*

$$\text{PPL} = e^L. \tag{8.4}$$

Lower perplexity means that, on average, the model assigns more probability to the observed next tokens.

Perplexity is useful because it evaluates the predictive distribution rather than one arbitrary generated string. But comparisons should use compatible tokenisation and data: changing from words to subwords changes what counts as one prediction step.

8.5 The model learns a distribution; decoding chooses an output

After training, the model does not contain one predetermined continuation. At every step it contains a probability distribution over possible next tokens. A decoding rule turns that distribution into an actual sequence.

8.5.1 Greedy decoding

Greedy decoding chooses

$$x_t = \arg \max_k p(x_t = k \mid x_{<t})$$

at every step. It is deterministic and easy to inspect. It can also over-emphasise common local choices. In a corpus with repeated stylistic openings, many distinct prompts can fall into the same high-probability phrase and then continue along similar trajectories.

A fluent greedy output is therefore weak evidence about the full distribution the model has learned.

8.5.2 Sampling and temperature

Sampling instead draws a token from the predicted categorical distribution. Temperature modifies how peaked that distribution is. Given logits z_k , one common definition is

$$p_\tau(k) = \frac{\exp(z_k/\tau)}{\sum_j \exp(z_j/\tau)}. \quad (8.5)$$

When $\tau < 1$, high-logit tokens become relatively more dominant. When $\tau > 1$, probability mass is spread more broadly. Top- k or nucleus sampling can additionally restrict sampling to a plausible subset of candidates.

These rules change inference, not the trained parameters. If higher temperature produces varied but incoherent text, the model has not been retrained into a worse state. We are exploring lower-probability regions of the same learned distribution.

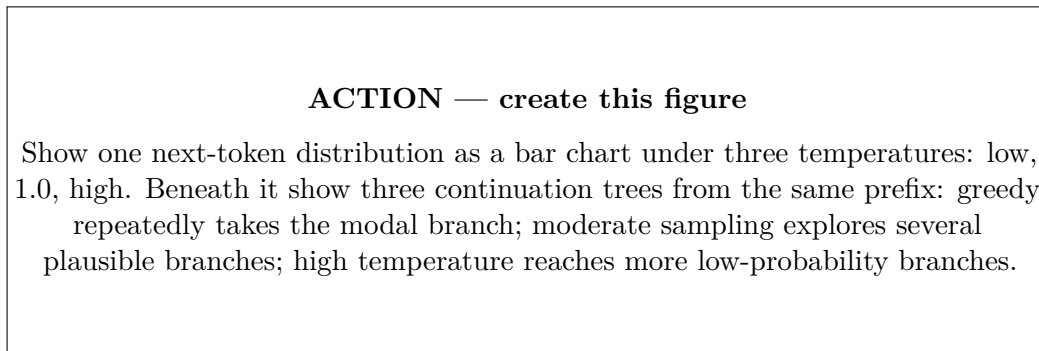


Figure 8.2: Training learns a next-token distribution. Decoding determines how one path is selected from that distribution.

8.6 When a decoder ignores its condition

Many generation problems are conditional. We may want to generate a caption from an image, a reply from a question, or the next StoryReasoning description from earlier story context.

Let $\mathbf{c}$ denote a conditioning representation. A conditional autoregressive model aims to learn

$$p(x_1, \dots, x_T \mid \mathbf{c}) = \prod_t p(x_t \mid x_{<t}, \mathbf{c}). \quad (8.6)$$

The condition may initialise the recurrent state, be concatenated with every token embedding, or enter through a richer memory mechanism.

But connecting $\mathbf{c}$ to the decoder in code does not prove that the decoder uses it.

8.6.1 Why teacher forcing can make the condition easy to ignore

Suppose descriptions in a dataset have highly predictable local syntax. Once the decoder sees the true previous token, that token may already provide most of the information needed for the next prediction. The explicit condition can become a weak secondary signal.

A model can therefore achieve low teacher-forced cross-entropy while producing almost the same distribution when the condition is replaced by a wrong one. The optimisation has not necessarily failed. The model found an easier predictive route.

This is exactly the kind of claim Week 6 taught us to test behaviourally.

A simple control is the *shuffled-condition test*:

1. evaluate the trained decoder with the correct condition;
2. randomly pair conditions with the wrong target sequences;
3. keep the token histories unchanged;
4. measure how much the loss or output distribution changes.

If performance barely changes, the condition has weak causal influence on the predictions under that evaluation regime.

Key idea

“Conditional architecture” is a statement about connectivity. “Condition-dependent behaviour” is an empirical claim and needs an intervention or control.

8.6.2 Making the easy signal less reliable

One way to encourage stronger conditioning is to make the teacher-forced history less perfectly informative. *Token dropout* occasionally replaces an earlier ground-truth token with a mask or unknown token during training. The decoder must then rely more heavily on other information, including the condition.

This should be interpreted carefully. Token dropout does not somehow tell the network what the condition means. It changes the information structure of the training problem: one route becomes less reliable, increasing the value of another.

The same principle appears throughout deep learning. If two signals both solve a task, the model has no obligation to prefer the one we intended unless the architecture, data or objective makes that route advantageous.

8.7 Sequence-to-sequence models: encoding one sequence, decoding another

Some tasks map a sequence to another sequence rather than merely continuing the same one. Machine translation is the classic example. An encoder reads an input sequence and produces a representation; a decoder generates an output sequence conditioned on that representation (Sutskever et al. 2014).

In the simplest recurrent sequence-to-sequence design,

$$\text{input sequence} \longrightarrow \mathbf{h}_{\text{final}} \longrightarrow \text{output decoder}.$$

The final encoder state acts as a fixed-size interface between the two sequences.

This architecture makes the division of labour clear. The encoder answers, “what should be remembered about the input?” The decoder answers, “how should that remembered information shape each next output token?”

But it also exposes a bottleneck. Every distinction in a long input sequence must be compressed into one vector before decoding begins. Different output steps may need different parts of the source, yet all must recover those details from the same summary.

Application to the architecture

The StoryReasoning system separates two jobs that are easy to confuse.

Input descriptions must be encoded. Each frame description is converted into a fixed vector so that a sequence model can combine four descriptions. In the architecture narrative, a frozen MiniLM encoder supplies a 384-dimensional sentence representation. Because the encoder is pretrained and frozen, the later model begins with a useful language representation rather than learning sentence semantics from the relatively small StoryReasoning corpus.

The fifth description must be generated. A separate LSTM language model learns the local statistics of StoryReasoning descriptions. Pretrained unconditionally, it reaches held-out perplexity about 17.9. This is a different job: predicting tokens one after another rather than mapping a whole sentence to one vector.

That distinction matters when the two pieces are joined. A conditional decoder can still ignore the story representation if teacher-forced previous words already predict the next word well. In the architecture experiments, comparing text cross-entropy under the true versus a shuffled condition became a direct test of whether the context actually mattered. Word dropout increased the incentive to use the condition.

The lesson is architectural and diagnostic: *encoding language, modelling language and conditioning language generation are three different problems*. Good performance on one does not prove success on the others.

This also explains why a single generated sentence is a poor diagnostic. The corpus itself has strong recurring style. Greedy decoding may select a modal phrase even when the conditional distribution differs meaningfully across inputs. Distributional metrics, sampled outputs and shuffled-condition controls answer different questions.

8.8 A fixed hidden state is an information bottleneck

A recurrent state has an attractive property: computation grows sequentially, and the same update rule can process sequences of different lengths. But every early detail that matters at time T must survive all intermediate updates.

Consider a four-description StoryReasoning window. A GRU can process the four sentence embeddings and use $\mathbf{h}_4$ to predict an embedding for description five. If retrieval improves over copy-last and mean-embedding baselines, the recurrent model has learned something useful about context. Yet this still does not prove that every position matters equally or that order is essential.

The controls from earlier weeks remain valuable:

- replace the entire context with another story window;
- reverse or permute the four positions;
- blank one input position at a time;
- compare against copy-last and mean-context baselines.

These interventions distinguish *using the window's content* from *using its order*. A recurrent architecture is order-sensitive in principle, but a trained model can still learn a nearly order-insensitive solution if that is sufficient for the data.

The bottleneck becomes more severe as context grows. If twenty earlier elements may matter for different reasons, asking one fixed-size state to preserve every useful distinction becomes increasingly demanding.

ACTION — create this figure

Draw a recurrent encoder processing six source states $h_1 \dots h_6$. Left: a fixed-summary decoder can access only h_6 , with all earlier information forced through the chain. Right: an attention-style decoder query connects directly to all six stored states and assigns different weights to them. Do not show multi-head attention or transformer block details.

Figure 8.3: Recurrence compresses the past into a moving state. Attention introduces a shorter route: retain multiple earlier states and retrieve a task-relevant combination when needed.

8.9 Attention: retrieve instead of remembering everything in one vector

The fixed-state bottleneck motivated attention mechanisms in sequence-to-sequence models (Bahdanau et al. 2015). Instead of requiring the encoder's final state to contain everything, keep a collection of representations

$$h_1, \dots, h_T.$$

For a current decoding step, construct a *query* describing what information is needed. Compare that query with a *key* derived from each stored position, turn the resulting scores into weights, and use those weights to combine the corresponding *values*.

At a high level:

$$\begin{aligned} s_i &= \text{score}(\mathbf{q}, \mathbf{k}_i), \\ \alpha_i &= \text{softmax}(s_i), \\ \mathbf{c} &= \sum_i \alpha_i \mathbf{v}_i. \end{aligned}$$

The context vector $\mathbf{c}$ is therefore not one permanent summary. It is a summary created for the current query.

This changes the information path. To use an early token, a decoder no longer has to rely only on that token's influence surviving every recurrent update. The current query can connect more directly to an earlier stored representation.

For now, three ideas are enough:

1. **query**: what information am I looking for now?
2. **key**: how should each stored item advertise what it contains?
3. **value**: what information should be retrieved if that item is relevant?

Week 9 will derive scaled dot-product attention, masking, multi-head attention, self-attention and cross-attention in detail.

And Week 6's warning still applies: the weights α_i are inspectable, but an inspectable weight is not automatically a faithful causal explanation of the model's prediction.

8.10 The first transformer view

Once we allow representations to interact through attention, recurrence is no longer the only way to combine sequence positions. A transformer applies attention so that positions can exchange information directly, then processes each resulting representation through a feed-forward network, with residual connections and normalisation around these sublayers (Vaswani et al. 2017).

A high-level encoder block can be read as

representations $\longrightarrow$ self-attention $\longrightarrow$ MLP $\longrightarrow$ updated representations,

with residual and normalisation paths stabilising the computation.

The crucial contrast with recurrence is the information path. In an RNN, token 1 influences token 100 through a chain of recurrent transitions. In self-attention, a representation at position 100 can interact directly with position 1 within one attention layer.

There is also a computational consequence. Recurrent states must normally be computed in order: $\mathbf{h}_3$ needs $\mathbf{h}_2$. During transformer training, many positions can be processed in parallel because attention operates over a whole matrix of token representations at once. This parallelism helped make transformers especially effective at scale.

Attention alone, however, does not know token order. If the same set of vectors is permuted, a bare self-attention operation permutes its outputs correspondingly. Transformers therefore need positional information added or encoded in some form. The details return in Week 9.

For autoregressive generation, one limitation remains: even a transformer normally generates output tokens one step at a time because token $t + 1$ depends on the sampled or selected token at t . Parallel training does not imply fully parallel free-running generation.

Check your understanding

Match each problem to the mechanism that addresses it most directly.

1. Same tokens in a different order collapse under mean pooling.
2. A vanilla RNN struggles to preserve one early bit across many distractors.
3. A conditional decoder scores almost identically after its condition is shuffled.
4. Different decoder steps need different details from a long source sequence.

Possible mechanisms: an order-sensitive sequence computation; gating; stronger conditioning tests/training such as token dropout; attention over stored states. None of these mechanisms is a universal substitute for the others.

8.11 What to carry into generative modelling

This week introduced sequence generation, but its deepest lesson is broader than language.

A model can represent uncertainty about what comes next through a probability distribution. Greedy decoding hides most of that distribution; sampling reveals alternatives. A conditional model can be wired to context yet fail to use it. A recurrent architecture can be order-sensitive yet learn a solution that scarcely depends on order. A hidden state can function as memory over short ranges while losing information over longer ones.

These are all versions of the same principle:

architecture defines possible information paths; training determines which paths the model actually uses.

The next step is to ask a different question. So far, language generation samples among discrete next tokens, but our visual StoryReasoning target may have many plausible future images. Week 8 will move from one deterministic point prediction to explicit generative modelling: latent distributions, sampling, VAEs, adversarial generation and diffusion.

Before moving on, the core Week 7 vocabulary should feel connected rather than memorised:

- **tokenisation** chooses the discrete units;
- **embeddings** turn those units into learned vectors;
- **recurrence** updates a state in sequence order;
- **gating** creates learnable routes for preserving and updating information;
- **autoregressive modelling** factorises a sequence into next-step distributions;
- **teacher forcing** trains from true prefixes;
- **decoding** chooses how to turn distributions into actual outputs;
- **conditioning** adds external information, whose use must be tested;
- **attention** retrieves different stored information for different queries;
- **transformers** build sequence computation primarily from attention rather than recurrence.

A useful diagnostic habit closes the chapter. Whenever a sequence model appears to remember or condition successfully, ask: *what intervention would make the claim false if the model were not actually using that information?* The answer may be a shuffled condition, a reversed sequence, a blanked position, a longer delay, or a competing baseline. Memory, like explanation, should be demonstrated rather than inferred from architecture names.

Chapter 9

From one prediction to a distribution

Week 7 ended with an apparently modest observation: a language model does not contain one next word. It assigns a distribution over possible next tokens, and a decoding rule turns that distribution into an actual continuation. Sampling twice from the same model can therefore produce two different, defensible answers.

For images and other high-dimensional continuous targets, the same idea is harder to see. A regression network usually emits one vector. An image predictor emits one array of pixels. If the future is uncertain, where should a second possible answer come from? More importantly, what objective would persuade a model to preserve several alternatives instead of averaging them into one compromise?

This is the problem of generative modelling. The change is conceptual before it is architectural. We stop asking only for a function

$$\hat{\mathbf{y}} = f_{\theta}(\mathbf{x})$$

and ask instead for a distribution

$$p_{\theta}(\mathbf{y} \mid \mathbf{x})$$

from which different plausible outputs can be sampled.

That distinction matters whenever the observations do not determine one unique target. A short story can plausibly cut to several camera views. A text prompt can correspond to innumerable valid images. A partially observed scene may have several reasonable completions. In such settings, uncertainty is not merely measurement noise around one hidden answer. It can be part of the structure of the task.

Key idea

A point predictor must commit to one output. A generative model tries to represent how probability is distributed over many possible outputs. When the target is multi-modal, the difference between these two jobs can be the difference between an average that is numerically safe and a sample that is actually plausible.

This chapter develops that idea through four mechanisms. We begin with deterministic autoencoders, which learn useful latent representations but do not by themselves define a sampling rule. We then introduce variational autoencoders, where a latent variable turns reconstruction into a probabilistic model. GANs approach distribution matching through an adversarial game. Diffusion models approach it through repeated corruption and denoising. Finally, we ask how these models can be conditioned on labels, text or other modalities, and how one should evaluate a model whose purpose is to produce more than one answer.

9.1 One recorded target does not imply one possible future

A supervised dataset normally records one target $\mathbf{y}_i$ for each input $\mathbf{x}_i$. It is tempting to conclude that the learning problem therefore has one correct output per input. This is not necessarily true.

Imagine repeatedly observing almost the same four-frame dialogue context in different films. Sometimes the next shot cuts to the speaker. Sometimes it cuts to the listener. Sometimes it remains on the current shot. The dataset records only the continuation that happened in one particular story, but the conditional process

$$p(\mathbf{y} \mid \mathbf{x})$$

may place substantial probability on several distinct outcomes.

A distribution with several separated high-probability regions is called *multimodal*. The word “mode” here means a region of high probability, not an input modality such as image or text. A multimodal future might contain several sharp image families separated by outputs that almost never occur.

This creates a difficulty for ordinary regression. A deterministic predictor has one output location. Training losses decide where that single output should sit relative to the distribution of possible targets.

9.1.1 Squared error predicts a conditional mean

Suppose, for the moment, that the target is a scalar random variable Y and the input $X = x$ is fixed. We choose one deterministic prediction a and minimise expected squared error:

$$\mathbb{E} \left[(Y - a)^2 \mid X = x \right].$$

Let

$$\mu(x) = \mathbb{E}[Y \mid X = x].$$

Then

$$\mathbb{E} \left[(Y - a)^2 \mid x \right] = \mathbb{E} \left[(Y - \mu + \mu - a)^2 \mid x \right] \quad (9.1)$$

$$= \text{Var}(Y \mid x) + (\mu - a)^2. \quad (9.2)$$

The variance term does not depend on a . The expected loss is therefore minimised by

$$a = \mu(x).$$

Mean-squared error is not “making the model blurry” by accident. It is doing exactly what its specification asks: among all single point predictions, choose the conditional mean.

For absolute error, the corresponding optimal point is a conditional median. This can be more robust to outliers, but it does not remove the fundamental restriction that only one point is emitted.

Now consider an equal mixture with targets near -1 and $+1$, but almost no data near zero. The mean is near zero. The MSE-optimal point can therefore fall in a region that the data-generating process itself rarely produces.

The same effect in pixel space is visually striking. If two plausible futures contain an object in different locations, a pixelwise average combines their intensities. Edges can double, colours mix, and the result may resemble neither future. With many possible images, the compromise can become a smooth generic picture.

ACTION — create this figure

Create a two-part figure. Left: for one fixed condition x , show a one-dimensional target density with two separated modes and mark the conditional mean in the low-density gap. Right: show two sharp but different plausible image futures A and B, with a simple pixelwise average between them that looks visibly less plausible than either endpoint. Label the average as a point-estimation optimum rather than a third mode.

Figure 9.1: When several targets are plausible, a point loss can place the optimal deterministic prediction between modes. The numerical optimum need not resemble a likely sample from the data distribution.

Optional mathematics — why the mean appears

For fixed x , let Y have any distribution with finite variance. Starting from

$$R(a) = \mathbb{E}[(Y - a)^2 \mid x],$$

differentiate with respect to a and show that the stationary point is $a = \mathbb{E}[Y \mid x]$. Check that it is a minimum.

Then consider a distribution that places probability 0.8 at $Y = -1$ and probability 0.2 at $Y = +1$. Compute the MSE-optimal point. Is that point itself one of the possible observations?

9.1.2 The irreducible part of point prediction

The decomposition above also tells us something deeper. Even the optimal point predictor retains an expected error equal to the conditional variance:

$$\min_a \mathbb{E}[(Y - a)^2 \mid x] = \text{Var}(Y \mid x).$$

If the future is genuinely uncertain, no better optimiser can remove that uncertainty from a single deterministic prediction. A larger network may estimate the conditional mean more accurately, but it is still estimating a mean.

This is an important diagnostic distinction. Poor point predictions can arise because the model class is inadequate, because optimisation failed, because the representation discarded information, or because the task itself admits several futures. Generative modelling addresses the last case by changing what the model is asked to represent.

9.2 Autoencoders: reconstruction is not yet generation

Week 5 introduced the autoencoder as a representation-learning device. An encoder maps an observation to a latent code,

$$\mathbf{z} = E_\phi(\mathbf{x}),$$

and a decoder maps the code back to a reconstruction,

$$\hat{\mathbf{x}} = D_\theta(\mathbf{z}).$$

Training minimises a reconstruction objective such as

$$\mathcal{L}_{\text{recon}} = \|D_{\theta}(E_{\phi}(\mathbf{x})) - \mathbf{x}\|_1.$$

If reconstruction is good and the bottleneck is restrictive, the latent can become a compact description of the input.

It is easy to look at this encoder–latent–decoder structure and conclude that the model is generative. After all, the decoder can turn a vector into an image. But a decoder is only useful for generation if we know how to choose vectors that correspond to plausible data.

A deterministic autoencoder does not usually provide such a rule.

9.2.1 The latent space may have holes

The encoder visits a particular set of latent locations while training. Nothing in ordinary reconstruction loss requires these codes to fill a simple region, to be centred around zero, or to resemble a standard Gaussian distribution. Different parts of latent space may be occupied by complicated curved clusters with empty regions between them.

Suppose the latent has eight dimensions. We may be tempted to draw

$$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

and feed the result to the decoder. But unless the training objective explicitly made the encoded data compatible with that distribution, this sampling rule is arbitrary. The sampled vector may land far from every code the decoder saw during training.

A neural decoder will still return numbers. That is not evidence that the numbers form a plausible sample.

Latent interpolation illustrates the same distinction. If we encode two real observations and move along the line between their codes, the decoder may produce a smooth visual transition. This tells us something about the local geometry between those two known points. It does not tell us the probability with which different latent regions should be sampled.

Key idea

An autoencoder answers “can this observation be compressed and reconstructed?”
A generative model must additionally answer “from which latent values should new observations be produced, and with what probabilities?”

Check your understanding

A deterministic autoencoder reconstructs MNIST digits almost perfectly. You sample 100 vectors from $\mathcal{N}(\mathbf{0}, \mathbf{I})$, decode them, and most outputs are malformed. Which conclusion is best supported?

1. The decoder is necessarily too weak.
2. Reconstruction training succeeded, but the chosen normal distribution was not guaranteed to match the region occupied by encoded data.
3. The optimiser failed because generation and reconstruction are the same objective.

The second conclusion separates representation learning from distribution modelling.

9.3 Latent variables: put the alternatives inside the model

A natural way to represent several possible outputs is to introduce a random latent variable $\mathbf{z}$. Instead of mapping one condition directly to one output, we describe a two-stage process:

$$\mathbf{z} \sim p(\mathbf{z}), \quad \mathbf{x} \sim p_\theta(\mathbf{x} | \mathbf{z}).$$

Different draws of $\mathbf{z}$ can lead to different observations.

The joint model is

$$p_\theta(\mathbf{x}, \mathbf{z}) = p(\mathbf{z})p_\theta(\mathbf{x} | \mathbf{z}),$$

and the probability assigned to an observation is obtained by marginalising the latent:

$$p_\theta(\mathbf{x}) = \int p(\mathbf{z})p_\theta(\mathbf{x} | \mathbf{z}) d\mathbf{z}. \quad (9.3)$$

This simple equation captures the generative idea. The observed image does not need to identify one unique hidden cause. Many latent values may contribute probability to similar observations, and the prior tells us how likely different latent values are before seeing the observation.

We now have two complementary inference questions.

- **Generation:** draw $\mathbf{z}$ from the prior and decode it into an observation.
- **Inference:** after observing $\mathbf{x}$, infer which latent values could plausibly have produced it, that is, reason about $p_\theta(\mathbf{z} | \mathbf{x})$.

The second question creates the central computational problem. For a flexible neural decoder, the exact posterior

$$p_\theta(\mathbf{z} | \mathbf{x}) = \frac{p(\mathbf{z})p_\theta(\mathbf{x} | \mathbf{z})}{p_\theta(\mathbf{x})}$$

contains the marginal likelihood from [equation \(9.3\)](#), an integral that is generally expensive or intractable to evaluate exactly.

The variational autoencoder is a practical answer to that difficulty.

9.4 Variational autoencoders: make the latent sampleable

A variational autoencoder (VAE) combines a probabilistic decoder with a learned approximation to the posterior ([Kingma and Welling 2014](#)). The architecture resembles an autoencoder, but the latent interface has changed fundamentally.

Instead of producing one code $\mathbf{z}$, the encoder produces parameters of a distribution. In the common diagonal-Gaussian form,

$$q_\phi(\mathbf{z} | \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x}))).$$

This distribution is called the *approximate posterior*. The prior is often chosen to be the simple standard normal

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

The decoder specifies $p_\theta(\mathbf{x} | \mathbf{z})$.

Training now has two jobs. The latent sampled from the posterior must contain enough information to reconstruct the observation, but the collection of posteriors should also remain compatible with the prior from which we intend to sample later.

9.4.1 Reconstruction and KL play different roles

The standard VAE objective can be written as the negative evidence lower bound,

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x} | \mathbf{z})] + D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z})). \quad (9.4)$$

The first term is a reconstruction or negative log-likelihood term. It rewards latents that allow the decoder to account for the particular observation.

Mathematical Derivation

For some observation x , consider that we wish to maximise its marginal likelihood $p(x)$. Directly computing the marginal likelihood is generally intractable, as it requires integrating over all possible values of the latent variable z . We therefore begin with:

$$p(x) = \int p(x | z)p(z)dz.$$

Taking the natural logarithm, we have:

$$\log p(x) = \log \int p(x | z)p(z), dz.$$

We then introduce the approximate posterior $q(z | x)$, which provides a tractable approximation to the true posterior $p(z | x)$:

$$\begin{aligned} \log p(x) &= \log \int q(z | x) \frac{p(x | z)p(z)}{q(z | x)} dz \\ &\geq \int q(z | x) \log \left(\frac{p(x | z)p(z)}{q(z | x)} \right) dz \\ &= \mathbb{E}_{z \sim q(z | x)} [\log p(x | z)] + \int q(z | x) \log \left(\frac{p(z)}{q(z | x)} \right) dz \\ &= \mathbb{E}_{z \sim q(z | x)} [\log p(x | z)] - D_{\text{KL}}(q(z | x) \| p(z)) \\ &= \text{Equation 9.4.} \end{aligned}$$

The second term is the Kullback–Leibler divergence. It penalises a posterior that departs too strongly from the chosen prior. Informally, it asks the encoder not to hide each training example in an arbitrary isolated region that the prior will almost never visit.

The two pressures compete. Perfect reconstruction would be easiest if every observation could occupy a highly specific code with no concern for global organisation. Easy prior sampling would be simplest if every posterior looked exactly like the prior, but then the latent might contain little information about the input.

A VAE therefore turns the bottleneck into a negotiated interface between information about one observation and a shared sampling distribution.

9.4.2 The reparameterisation trick

There is a technical obstacle. We want gradients to change μ_ϕ and σ_ϕ , but a random sampling operation appears between those parameters and the decoder.

For a Gaussian posterior we can move the randomness into a parameter-free variable:

$$\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathbf{z} = \mu_\phi(\mathbf{x}) + \sigma_\phi(\mathbf{x}) \odot \epsilon. \quad (9.5)$$

Now a sampled ϵ is treated as an input to an ordinary differentiable computation. Gradients can flow through multiplication and addition into the encoder parameters. This is the *reparameterisation trick*.

ACTION — create this figure

Draw a VAE diagram with an input x entering an encoder that outputs μ and log-variance. Show ϵ sampled from $N(0, I)$ entering a reparameterisation node $z = \mu + \sigma * \epsilon$, then a decoder producing $\hat{x}$. Draw a separate standard-normal prior beside z . Label the two training pressures: reconstruction from $\hat{x}$ to x , and KL from $q(z|x)$ toward $p(z)$. Add a second small inference panel showing generation: sample z from $p(z)$ and decode without an input x .

Figure 9.2: A VAE trains an approximate posterior for reconstruction while regularising it toward a prior that can be sampled at generation time. The encoder is needed during training/inference on observed data; unconditional generation can begin directly from the prior.

Implementations usually predict $\log \sigma^2$, often called **logvar**, rather than σ directly. Exponentiating half the log-variance gives the standard deviation while guaranteeing positivity:

$$\sigma = \exp\left(\frac{1}{2}\logvar\right).$$

PyTorch: the visible VAE mechanism

```
mu, logvar = encoder(x)
std = torch.exp(0.5 * logvar)
eps = torch.randn_like(std)
z = mu + std * eps
x_hat = decoder(z)

recon = reconstruction_loss(x_hat, x)
kl = -0.5 * (1 + logvar - mu.pow(2) - logvar.exp()).sum(dim=1)
loss = recon.mean() + beta * kl.mean()
```

The code is short because much of the idea lives in the objective. The same decoder architecture can behave very differently depending on how strongly the latent is regularised.

9.4.3 How much KL?

It is common to use a coefficient β ,

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \beta D_{\text{KL}}(q||p), \quad (9.6)$$

so that the balance can be controlled explicitly.

If β is too small, the posterior can become extremely informative about each training target while drifting far from the prior. Reconstruction may look excellent because the encoder knows exactly which target it saw, yet samples drawn from the prior at test time may land in irrelevant regions. This is *prior-posterior mismatch*.

If the pressure toward the prior is too strong, the approximate posterior may become nearly independent of the input. In an extreme case,

$$q_{\phi}(z | x) \approx p(z)$$

for every $\mathbf{x}$, so the latent conveys little useful information. When a powerful decoder learns to ignore $\mathbf{z}$, this is called *posterior collapse*.

A small KL value is therefore not automatically good. Neither is a large KL value. The number must be interpreted alongside reconstruction, prior samples, posterior samples and sample diversity.

KL warm-up is one practical response: begin training with a small KL weight and increase it gradually, allowing the reconstruction path to become useful before applying the full regularising pressure. Other mechanisms, such as free bits, prevent small amounts of information in latent dimensions from being penalised too aggressively. The details vary, but the underlying problem is always the same negotiation between information and sampleability.

Optional mathematics — diagonal Gaussian KL

For

$$q(\mathbf{z}) = \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2)), \quad p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

the KL divergence is

$$D_{\text{KL}}(q||p) = \frac{1}{2} \sum_j \left(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2 \right).$$

Inspect one term at a time. What values of μ_j and σ_j minimise it? Why does a KL-only objective erase input-specific information? Then explain why adding reconstruction prevents the trivial solution from being automatically optimal for the full VAE objective.

9.5 Conditional generation: a distribution that depends on context

Unconditional generation asks for plausible data in general. Many useful tasks are conditional. We may want an image matching a class label, a picture matching a text description, or a next frame compatible with the preceding story.

Let $\mathbf{c}$ denote the condition. The desired object is

$$p_{\theta}(\mathbf{y} \mid \mathbf{c}).$$

A conditional VAE can make the prior itself depend on context:

$$p_{\psi}(\mathbf{z} \mid \mathbf{c}), \quad q_{\phi}(\mathbf{z} \mid \mathbf{y}, \mathbf{c}), \quad p_{\theta}(\mathbf{y} \mid \mathbf{z}, \mathbf{c}). \quad (9.7)$$

The posterior sees both the condition and the realised target during training. The prior sees only the condition.

At first this can look like target leakage. Why is one part of the network allowed to look at the answer?

Because the posterior is a training-time inference mechanism. Its job is to say which latent alternatives explain the target that actually occurred. The KL term then pressures the condition-only prior to place probability in similar regions. At test time the target is unavailable, the posterior is discarded, and samples come from

$$\mathbf{z} \sim p_{\psi}(\mathbf{z} \mid \mathbf{c}).$$

The privileged target information is useful only if the prior learns to imitate the relevant posterior distribution well enough to generate without it.

This gives us a direct diagnostic. Compare posterior-sample quality with prior-sample quality. If posterior reconstructions are excellent while prior samples are poor, the target-aware inference network learned information that the test-time prior did not successfully capture.

9.5.1 Conditioning can still be ignored

Week 7 showed that a language decoder can be connected to a condition and nevertheless ignore it. The same danger exists in visual generation.

A model might produce diverse, realistic-looking samples that depend mainly on the unconditional training distribution. Diversity alone does not prove conditional generation. If the task is “generate the next frame given this story”, the samples must change appropriately when the story changes.

A shuffled-condition control remains powerful:

1. evaluate samples with the correct condition;
2. pair each target with an unrelated condition;
3. keep the trained model unchanged;
4. compare conditional metrics, retrieval and sample behaviour.

If shuffling the condition changes little, the generator may have learned a useful unconditional model while failing the conditional task.

Application to the architecture

The StoryReasoning image branch makes the reason for a conditional distribution unusually concrete. The fifth frame is a different shot in roughly 85% of windows, so a deterministic pixel-L1 predictor is rewarded for a smooth compromise. The visual encoder and decoder from Week 5 can already represent individual frames; Week 8 changes only the uncertain part of the prediction.

A recurrent context state $\mathbf{h}$ summarises the four observed steps. A conditional prior $p(\mathbf{z} \mid \mathbf{h})$ predicts a Gaussian residual distribution. During training, a posterior $q(\mathbf{z} \mid \mathbf{h}, \mathbf{z}_{\text{target}})$ is additionally allowed to inspect the frozen target-frame latent. A sampled residual and the context produce a predicted visual latent, which the inherited decoder can map back to pixels.

The KL weight becomes an empirical knob. With too little pressure, the posterior can encode the target while the prior fails to follow it. In the architecture experiments, $\beta = 10^{-3}$ left a KL of about 124 nats and prior samples were diverse but poorly tied to the context. Around $\beta = 10^{-2}$, the KL fell to about 10.8 nats; the average prior-sample L1 reached about 0.146, and the **best of** $K = 5$ reached about 0.125, below the roughly 0.13 deterministic ceiling observed for the task.

The important mechanism is not the exact number. The posterior shows what latent alternatives explain the realised target; the KL teaches the context-only prior to cover those alternatives; sampling exposes several futures instead of compressing them into one average. As always, K must be reported with a best-of- K result.

9.6 GANs: learn through a critic rather than reconstruction

Variational autoencoders define a latent-variable model and train it through a likelihood-related objective. Generative adversarial networks (GANs) take a different route (Goodfellow

et al. 2014).

A *generator* G_θ maps random noise to samples,

$$\mathbf{z} \sim p(\mathbf{z}), \quad \tilde{\mathbf{x}} = G_\theta(\mathbf{z}).$$

A *discriminator* D_ϕ is trained to distinguish real data from generated samples. In the original formulation, the two players optimise an adversarial objective of the form

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] \quad (9.8)$$

The discriminator becomes a learned signal that says, in effect, which differences between generated and real samples still make them distinguishable.

The appealing idea is that we no longer have to specify a pixelwise distance that says how one generated image should resemble one particular target. The discriminator can reward distribution-level properties that make samples look like the data.

The price is a coupled optimisation problem. The generator's objective changes as the discriminator changes, and vice versa. If the discriminator becomes too strong, the generator can receive unhelpful gradients. If the generator discovers a small family of outputs that repeatedly fools the discriminator, it may neglect other legitimate regions of the data distribution.

This latter failure is called *mode collapse*. A generator might produce sharp, convincing faces while producing only a narrow subset of identities or poses. Fidelity can be high while diversity is poor.

GANs therefore reinforce an important Week 8 lesson: sample quality is not one-dimensional. A model can generate excellent individual examples and still represent the distribution badly.

Conditional GANs introduce a condition to both generator and discriminator. The generator then tries to create samples that are realistic *and* compatible with the condition. The same diagnostic rule applies: changing or shuffling the condition should change the generated distribution in a measurable way.

9.7 Diffusion: generation as repeated denoising

Diffusion models take yet another approach. Instead of learning one direct jump from random noise to a data sample, they construct generation as a sequence of many small denoising steps (Ho et al. 2020).

The method begins with a simple corruption process. Starting from clean data $\mathbf{x}_0$, progressively add Gaussian noise to obtain

$$\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T.$$

With a suitable noise schedule, later states become increasingly close to an easy reference distribution such as a standard Gaussian.

A useful closed-form expression lets us sample a noisy version at time t directly:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I), \quad (9.9)$$

where $\bar{\alpha}_t$ records how much clean signal remains after the scheduled corruption.

The forward process is easy because we choose it. The learning problem is to reverse it.

9.7.1 Predict the corruption

A common denoising model receives a noisy sample $\mathbf{x}_t$ and the timestep t , and predicts the noise that was added:

$$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t).$$

A simple training objective is

$$\mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}, t} \left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|_2^2 \right]. \quad (9.10)$$

The timestep matters because the same visible perturbation has a different meaning when the sample is almost clean than when nearly all original structure has been destroyed.

At generation time, begin with noise and repeatedly apply the learned reverse update. Structure emerges gradually as the model removes predicted noise across a sequence of timesteps.

ACTION — create this figure

Create a horizontal diffusion diagram. Top row, left-to-right: a clean sample gradually becomes noisier across four labelled timesteps until it resembles Gaussian noise; label this fixed forward corruption q . Bottom row, right-to-left: start from noise and show a learned denoiser making several reverse steps toward a structured sample; label the model $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$. Include the condition c entering the denoiser as an optional side arrow, but do not show transformer internals.

Figure 9.3: Diffusion makes generation an iterative reverse process. Training observes clean data corrupted to many noise levels; sampling begins from noise and repeatedly applies learned denoising steps.

The mechanism differs sharply from a VAE. A VAE normally samples one latent and decodes in one pass. A diffusion sampler performs many network evaluations. This makes sampling computationally heavier, although modern samplers can reduce the number of required steps substantially.

The iterative path also provides a different modelling advantage. Rather than asking one network call to place a random point directly onto a complicated data manifold, the model learns a family of local denoising directions across corruption levels.

9.7.2 Conditioning a denoiser

A denoiser can receive extra information $\mathbf{c}$: a class label, text embedding, low-resolution image, depth map or some other context. The prediction becomes

$$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{c}).$$

The reverse trajectory is then steered toward samples that are plausible under the condition.

Classifier-free guidance is one widely used method for strengthening this steering. During training, the condition is sometimes omitted so the same model learns conditional and unconditional denoising. During sampling, one can combine the two predictions schematically as

$$\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_{\text{uncond}} + s(\boldsymbol{\epsilon}_{\text{cond}} - \boldsymbol{\epsilon}_{\text{uncond}}),$$

where s controls guidance strength. Larger s pushes more strongly toward features associated with the condition, but excessive guidance can reduce diversity or introduce artefacts. Guidance is therefore another trade-off, not a free quality knob.

9.8 CLIP: align modalities without generating either one

Conditional generation often depends on a representation that lets two modalities communicate. A text-conditioned image model needs some numerical form of the text. An image-text retrieval system needs image and text representations that can be compared.

CLIP is a useful example of this representational role (Radford et al. 2021). It contains an image encoder and a text encoder trained on paired image-text data. The learning objective encourages the embedding of a matched image and caption to have high similarity while mismatched pairs in the batch have lower similarity.

The result is an approximately aligned representation space:

$$\mathbf{v}_{\text{image}}, \mathbf{v}_{\text{text}} \in \mathbb{R}^d,$$

where cosine similarity can be used to rank cross-modal matches.

CLIP is not, by itself, an image generator. It maps observations into a shared comparison space. That space can nevertheless serve several roles around generation:

- a text embedding can condition another generative model;
- image and text embeddings can support cross-modal retrieval;
- similarity can provide one evaluation signal for whether an image reflects a prompt;
- pretrained embeddings can provide a useful latent space when pixel generation would obscure the mechanism under study.

The final point is especially important pedagogically. Working in a frozen visual embedding space can let us study whether a stochastic predictor covers several plausible futures without also asking students to build a high-quality image decoder.

Aligned embedding spaces also have limits. High CLIP similarity is not a universal definition of image quality. It can miss fine visual errors, inherit biases from pretraining data, and reward semantic compatibility while ignoring other requirements. A representation-based metric is still a specification, and Week 2's lesson has not gone away.

The contrastive mechanics that create such shared spaces will return in Week 9 under self-supervised objectives. For now, the important idea is the separation of roles: representation alignment can make conditioning or retrieval easier without itself solving generation.

9.9 How should a generative model be evaluated?

A deterministic regressor can often be summarised by one average error. A generator creates a set or distribution of outputs, so evaluation must ask several questions at once.

9.9.1 Fidelity and diversity

Two broad properties are useful.

Fidelity asks whether individual samples resemble plausible data or satisfy the condition. A nearest-neighbour distance in a sensible representation, a reconstruction error in a matched setting, or a human judgement may provide evidence here.

Diversity asks whether the model covers the range of plausible outcomes rather than repeating a narrow subset. Pairwise sample distances, mode coverage on synthetic data or category coverage can provide evidence.

Neither property subsumes the other. A model that memorises one perfect training image can have high fidelity and almost no diversity. A model that produces random noise can have enormous diversity and terrible fidelity.

This two-axis view explains why GAN mode collapse is serious and why VAE posterior collapse is different. Mode collapse produces too little variety in observed samples. Posterior collapse means a latent variable carries too little input-specific information. Both can reduce useful diversity, but the mechanisms are not the same.

9.9.2 Average sample quality versus best-of-K

Suppose a conditional generator draws K samples

$$\tilde{\mathbf{y}}^{(1)}, \dots, \tilde{\mathbf{y}}^{(K)}$$

for one target $\mathbf{y}$. The average single-sample error estimates the quality of a typical draw. A best-of- K metric instead reports

$$\min_{1 \leq k \leq K} d(\tilde{\mathbf{y}}^{(k)}, \mathbf{y}). \quad (9.11)$$

This asks whether at least one of the sampled possibilities lands near the realised target.

On a genuinely multimodal problem, best-of- K can reveal something a point estimate cannot. If the model samples several distinct plausible futures, only one may resemble the particular future that happened to be recorded. Increasing K gives the model more chances to cover that mode.

But the score is inseparable from K . Best-of-20 is expected to be better than best-of-5 even when the underlying model is identical. Reporting “best error = 0.12” without reporting K is therefore incomplete.

Best-of- K can also be gamed by excessive or unrealistic sampling. A model that produces thousands of wild guesses may eventually land near many targets. The metric is most meaningful when K matches a plausible use case and when fidelity/diversity diagnostics show that the extra samples are not merely noise.

9.9.3 Use the condition test again

For conditional generation, evaluation should include an intervention on the condition. If a text prompt is replaced by an unrelated prompt, should the sample distribution change? If four story frames are shuffled across examples, should retrieval of the actual fifth frame become worse?

This is the same empirical discipline developed in Weeks 4, 6 and 7. Architecture names do not prove signal use. A “conditional VAE” is conditional by construction, but useful conditional behaviour is demonstrated by what happens when the condition is perturbed.

9.9.4 Nearest neighbours and memorisation

When generated examples look impressive, compare them with nearby training examples. A nearest-neighbour display can reveal whether the generator is reproducing almost exact observations or combining learned structure in new ways.

This diagnostic must also be interpreted carefully. In a dataset with many near-duplicate frames, a close neighbour may be expected. A distant neighbour is not proof of novelty,

because the distance metric may ignore important similarities. The goal is not one magical memorisation threshold, but evidence that complements fidelity and diversity measurements.

Check your understanding

Two conditional generators are evaluated on the same task.

Model A has excellent average single-sample fidelity but produces nearly identical samples for each condition. Model B has slightly worse average fidelity, substantial sample diversity, and much better best-of-5 performance. Shuffling the condition badly hurts Model B but barely changes Model A.

Which model provides stronger evidence of learning a useful conditional distribution?

What additional failure would you still want to rule out before declaring Model B successful?

9.10 Four generative mechanisms, four different contracts

It is tempting to arrange VAE, GAN and diffusion models on a single ladder from old to new. That obscures the more useful comparison: they define different training and sampling contracts.

Mechanism	What training asks for	How sampling begins	Characteristic risk
Deterministic autoencoder	reconstruct each input through a bottleneck	no principled generic rule unless one is added	latent regions used by an arbitrary sampler may be unsupported
VAE	reconstruct while keeping approximate posteriors compatible with a prior	sample a latent from the prior, then decode	prior/posterior mismatch or posterior collapse
GAN	make generated samples indistinguishable from real samples to a learned discriminator	sample noise, then one generator pass	unstable adversarial training or mode collapse
Diffusion	predict/reverse corruption across many noise levels	begin from noise and iteratively denoise	slow sampling and sensitivity to the reverse process/guidance

Table 9.1: Generative families differ not only in architecture but in what their objectives ask the model to learn and how randomness is turned into samples.

No row is universally best. A VAE provides an explicit latent inference mechanism that is convenient when a compact stochastic representation is valuable. GANs can produce sharp samples but are notoriously sensitive to adversarial dynamics. Diffusion models trade iterative computation for a stable denoising objective and strong sample quality. A deterministic autoencoder remains useful when the main goal is representation rather than generation.

The right question is therefore not “which generative model wins?” but “which uncertainty, conditioning structure, evaluation criterion and computational budget define this problem?”

9.11 What to carry into Week 9

This week has changed the object we ask a model to learn. A target can be uncertain even when the input is fixed. In that case, a point estimate may be an inadequate description

of the problem. A generative model introduces randomness in a structured way and trains that randomness to correspond to variation in the data.

The essential distinctions are:

- **point prediction versus distribution modelling:** one answer is not the same object as a probability law over answers;
- **reconstruction versus generation:** an autoencoder can preserve an input without defining how new latent codes should be sampled;
- **posterior versus prior:** a VAE learns from target-aware posterior samples but must generate from the prior available at test time;
- **reconstruction versus KL:** one preserves information about the observation while the other organises latent uncertainty for sampling;
- **fidelity versus diversity:** realistic individual samples do not guarantee coverage, and variety does not guarantee realism;
- **conditional connectivity versus conditional behaviour:** a condition must be shown to matter through interventions or matched controls;
- **one-step versus iterative sampling:** GANs and typical VAEs generate in a small number of passes, whereas diffusion deliberately refines noise across many steps;
- **generation versus alignment:** CLIP-like models align representations across modalities but do not themselves generate images.

There is one question we have mostly postponed. A condition can be represented by one vector, but what if different parts of a long condition matter for different parts of the output? What if a text token needs one image region, another token needs another region, and a generated sequence should retrieve different memories at different steps?

Week 7 motivated attention as an escape from one recurrent summary. Week 9 will now open that mechanism fully. Queries, keys and values will turn conditioning from “attach one context vector” into explicit information routing between positions, memories and modalities.

The generative lesson should remain in view when we do so. Better routing cannot remove genuine uncertainty. Attention can help a model use the evidence that exists; a distribution is still required when the evidence does not determine one unique future.

Chapter 10

Attention, transformers and information routing

Week 7 introduced a difficulty that recurrence cannot entirely remove. A hidden state can carry information forward, but every useful detail must survive repeated compression into the same vector. Attention offered a different possibility: keep several representations available and retrieve from them when a later computation needs something specific.

Week 8 then added another complication. A model may need to generate several plausible outputs, but those outputs should still depend on the evidence that is present. Randomness can represent uncertainty; it cannot substitute for using the condition correctly.

This week joins those two ideas. The central question is no longer only *what information can the model represent?* It is:

which information is allowed to interact with which other information, and how is that choice made?

Attention is useful because it turns that question into a differentiable computation. A query says what is currently needed. Keys describe how stored items can be matched. Values carry the information that will be routed forward. Transformers build most of their sequence computation from repeated versions of this operation.

Key idea

Attention is best understood as content-dependent information routing. A transformer is powerful not because every token is mixed with every other token indiscriminately, but because each representation can construct a query and retrieve a different weighted combination of the available information.

This viewpoint also keeps us from assigning too much meaning to architecture names. Self-attention, causal attention and cross-attention use nearly the same mathematical machinery. What changes is where queries, keys and values come from, which interactions are permitted, and what objective trains the resulting computation.

10.1 From a memory lookup to attention

Suppose a system stores four memories. Each memory has two parts:

$$(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_4, \mathbf{v}_4).$$

The key $\mathbf{k}_i$ describes how memory i should be matched. The value $\mathbf{v}_i$ contains the information to return if that memory is relevant.

Now introduce a query $\mathbf{q}$. A simple hard lookup would choose the key most similar to the query,

$$i^* = \arg \max_i \text{sim}(\mathbf{q}, \mathbf{k}_i),$$

and return $\mathbf{v}_{i^*}$.

That would be a perfectly sensible retrieval rule, but it has an awkward property for gradient-based learning: the identity of the selected item changes discretely. A small change in the query may do nothing until one score crosses another, at which point the returned value jumps.

Attention replaces this hard decision with a soft one. We first compute one score per memory,

$$s_i = \text{score}(\mathbf{q}, \mathbf{k}_i), \quad (10.1)$$

turn the scores into non-negative weights that sum to one,

$$\alpha_i = \frac{e^{s_i}}{\sum_j e^{s_j}}, \quad (10.2)$$

and return the weighted value mixture

$$\mathbf{o} = \sum_i \alpha_i \mathbf{v}_i. \quad (10.3)$$

The output can now change smoothly as the query changes. Near one key, most weight may fall on one memory. Halfway between two compatible keys, the output may become a mixture of their values.

That mixture is not a defect. It is what makes the retrieval differentiable. The network can learn to sharpen or broaden the routing by changing the query and key representations.

10.1.1 Keys and values do different jobs

It is tempting to collapse keys and values into one vector, but separating them is useful. The information that makes an item easy to select need not be the same information we want to retrieve.

Imagine a database of people. A key might encode identity, while the value stores an address. We match on one property and return another. Neural attention uses the same freedom. One projection can create coordinates useful for deciding relevance; another can create coordinates useful for downstream computation.

This distinction becomes especially important in cross-attention. A text-derived query may compare against visual keys that describe image content, then retrieve visual values that preserve a richer set of features than the matching coordinates alone.

10.2 Scaled dot-product attention

Transformers normally use learned linear projections to construct queries, keys and values. Given an input representation $\mathbf{x}$, one head may compute

$$\mathbf{q} = \mathbf{W}_Q \mathbf{x}, \quad \mathbf{k} = \mathbf{W}_K \mathbf{x}, \quad \mathbf{v} = \mathbf{W}_V \mathbf{x}.$$

The matrices are learned. Their role is not merely dimensional bookkeeping: they determine which directions matter for matching and which information is carried in the retrieved value.

For one query $\mathbf{q} \in \mathbb{R}^{d_k}$ and one key $\mathbf{k}_i \in \mathbb{R}^{d_k}$, the basic compatibility score is a dot product

$$\mathbf{q}^\top \mathbf{k}_i.$$

ACTION — create this figure

Draw one query on the left and four memory items on the right. Each memory item should have a key box above a value box. Show query-to-key score arrows, a softmax producing four weights, then weighted arrows from the four values into one output vector. Include two different queries over the same memory to show that routing can change while the stored values stay fixed.

Figure 10.1: Attention separates matching from retrieval. Queries are compared with keys; the resulting weights determine how values are combined. The same memory can therefore be routed differently for different queries.

A large positive score means that the learned query and key coordinates are strongly aligned. Transformer attention uses the scaled score

$$s_i = \frac{\mathbf{q}^\top \mathbf{k}_i}{\sqrt{d_k}}. \quad (10.4)$$

The division by $\sqrt{d_k}$ matters because the magnitude of an unnormalised dot product tends to grow with dimension. Large logits drive softmax toward extremely sharp distributions, where small score differences produce almost one-hot weights and gradients can become less useful. Scaling keeps the numerical range more controlled (Vaswani et al. 2017).

For a whole set of queries, keys and values we write matrices

$$\mathbf{Q} \in \mathbb{R}^{n_q \times d_k}, \quad \mathbf{K} \in \mathbb{R}^{n_k \times d_k}, \quad \mathbf{V} \in \mathbb{R}^{n_k \times d_v}.$$

Then all pairwise scores are computed at once:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (10.5)$$

The softmax is applied across the keys for each query row.

This compact equation contains the entire mechanism:

1. $\mathbf{Q}\mathbf{K}^\top$ asks how compatible every query is with every key;
2. softmax turns those scores into routing weights;
3. multiplying by $\mathbf{V}$ returns one weighted value mixture per query.

PyTorch: one attention head, written explicitly

```
Q = x_q @ W_Q # [B, nq, dk]
K = x_kv @ W_K # [B, nk, dk]
V = x_kv @ W_V # [B, nk, dv]

scores = Q @ K.transpose(-2, -1)
scores = scores / (dk ** 0.5) # [B, nq, nk]
weights = scores.softmax(dim=-1)
out = weights @ V # [B, nq, dv]
```

Optional mathematics — why the square-root scaling appears

Assume the coordinates of $\mathbf{q}$ and $\mathbf{k}$ are independent, mean zero and have variance approximately one. Consider the dot product

$$\mathbf{q}^\top \mathbf{k} = \sum_{j=1}^{d_k} q_j k_j.$$

Under these simplifying assumptions, estimate how the variance of the sum grows with d_k . What happens to that variance after dividing by $\sqrt{d_k}$?

This argument is only a scale heuristic, not a claim that trained query/key coordinates remain independent Gaussians. Its purpose is to explain why the normalisation depends on dimension.

Key idea

Content-dependent attention is conditional computation. The model is not merely representing different inputs; it is allowing different inputs to activate different information paths through the same stored memory.

ACTION — create this figure

Create a two-part figure using the same four token embeddings in two different orders. Left: content-only self-attention followed by mean pooling, emphasising that a permutation simply permutes the internal rows and the pooled result cannot identify order. Right: add distinct positional vectors before attention, so swapping tokens changes the combined representations and therefore the attention score matrix.

Figure 10.2: Self-attention provides direct interaction between positions but not an intrinsic ordering coordinate. Positional information makes order-dependent relations available to the learned attention computation.

Check your understanding

A model uses eight attention heads. On one example, head 3 assigns 0.85 weight to a particular token. Which statement is justified?

1. “Token 3 caused the prediction.”
2. “Under head 3’s learned query/key projections, that token contributed strongly to the value mixture for this query.”

The second statement follows from the computation. The first needs an intervention on the token, value path or resulting representation.

-
-

- 1.
- 2.

PyTorch: one small self-attention block

```
attn_out, weights = mha(x, x, x,  
                        attn_mask=mask,  
                        need_weights=True)  
x = norm1(x + attn_out)  
  
mlp_out = mlp(x)  
x = norm2(x + mlp_out)
```

2014

2015

-
-
-

2021

Key idea

Contrastive alignment and attention solve different problems. Contrastive learning makes two representations comparable in a shared geometry. Attention uses a query to route information among representations inside a particular computation.

Optional mathematics — one row of InfoNCE

Suppose one image has similarities $[4.0, 1.5, 0.5, -1.0]$ to four candidate texts and the first text is the correct pair.

1. Apply a softmax to the four similarities and compute the probability assigned to the correct text.
2. Recompute after dividing all similarities by temperature $\tau = 0.5$.
3. Recompute after changing one negative similarity from 1.5 to 3.8.

Explain how temperature and hard negatives change the learning signal even though the identity of the positive pair is unchanged.

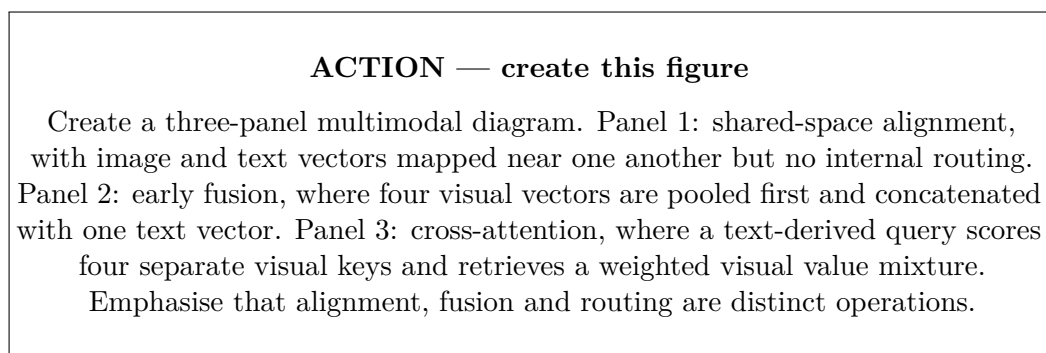


Figure 10.3: Multimodal systems can combine representations in different ways. Shared-space alignment makes modalities comparable; early fusion combines summaries; cross-attention allows one modality to route information selectively from another.

Application to the architecture

The StoryReasoning system provides a useful sequence of attention failures and repairs because the task contains four input frames whose relevance changes from one story window to another.

An early selector used attention weights over the four input positions. The weights looked structured, but the structure was mostly a *positional prior*: frame 3 tended to receive high weight. On near-copy windows, the selector identified the actually closest input only about 29% of the time, scarcely above the 25% four-way chance level. The architecture contained attention, but the learned routing was not yet strongly content-dependent.

Making the query depend on the sequence state and explicitly supervising the selected frame improved selection. Yet coupling that selector too tightly to the shared GRU state damaged text retrieval badly. A mechanism that helped one route changed the representation another head depended on.

The later architecture therefore separates roles more carefully. Frame-selection evidence can be computed from visual latent relationships without forcing the shared sequence state to carry every selection objective. Separately, the text decoder uses cross-attention over a memory containing input-description vectors and character-name information, so different generated words can retrieve different conditioning evidence.

Week 9 supplies the language for these observations. A fixed or weakly content-dependent query can learn a useful average preference without solving per-example routing. Attention weights are inspectable but need behavioural tests. Cross-attention is most valuable when different queries genuinely need different parts of memory. And information-routing components can interfere with the representation that creates their queries, so architecture should be judged by the whole system rather than by one attractive attention map.

-
-
-

-
-
-

Check your understanding

A multimodal transformer obtains excellent validation retrieval. When the text condition is shuffled across examples, retrieval barely changes. Attention maps over the visual tokens look sharp and different across examples.

Which conclusion is best supported?

1. The model has learned sophisticated text-controlled visual reasoning because the maps are sharp.
2. The evidence does not show that text materially affects retrieval; the sharp maps may reflect visual or positional routing that is largely independent of the text condition.

The second conclusion follows from the intervention. The architecture permits cross-modal routing, but the behaviour has not demonstrated dependence on the text.

- [illegible]

Key idea

Scale does not make the scientific questions disappear. It makes it more important to separate *capacity*, *data*, *compute*, *objective* and *measurement*. If a system improves, we should ask which resource removed which limitation. If it does not improve, we should ask whether the limiting factor was ever one that scaling could address.

-
-
-
-
-
-
-

2020

2022

Check your understanding

A larger model has half the training loss of a smaller one. Their held-out accuracy is identical, and both fail on the same rare subgroup. Which conclusions follow?

The larger model is clearly fitting the training objective more closely. It does *not* follow that the model is more useful on the subgroup, nor that more parameters are the

ACTION — create this figure

Create a schematic three-panel scaling figure. Panel 1: validation loss decreasing smoothly as model/data/compute increase. Panel 2: a curve that flattens because an objective-limited metric stops improving even while training loss falls. Panel 3: a triangle labelled model capacity, data and compute, with objective/metric drawn as an outer specification that cannot be repaired merely by moving around the triangle.

Figure 11.1: Scaling studies should separate the resource being increased from the quantity being measured. Capacity, data and compute can remove different limitations, but none automatically changes what the objective rewards.

resource that subgroup performance needs. We would need targeted measurements: subgroup data coverage, calibration, representation quality, objective weighting or other relevant diagnostics.

Key idea

A pretrained or frozen representation is not merely a convenient feature extractor. It changes the optimisation geometry of the downstream problem by providing structure

that has already been learned and, when frozen, a reference that cannot drift to satisfy the current loss trivially.

-
-
-
-
-

PyTorch: a toy top-k expert mixture

```
router_logits = router(x) # [B, E]
weights = router_logits.softmax(dim=-1)

top_w, top_i = weights.topk(k=2, dim=-1)
top_w = top_w / top_w.sum(dim=-1, keepdim=True)

expert_outputs = torch.stack([expert(x) for expert in experts], dim=1)
selected = expert_outputs.gather(
    1, top_i[..., None].expand(-1, -1, expert_outputs.size(-1))
)
y = (top_w[..., None] * selected).sum(dim=1)
```

-
-
-
-
-
-

ACTION — create this figure

Create a 2-D geometric illustration with three panels. Healthy: several separated clusters around the origin. Anisotropic: the same relative clusters shifted strongly in one common direction so vectors from the origin have similar angles. Centred: subtract the shared mean and recover the relative cluster geometry. Add a fourth small inset for true collapse, where all points occupy nearly the same location.

Figure 11.2: A dominant common direction can compress raw cosine differences without destroying all information. Centring can reveal relative structure. True collapse is different: the meaningful variation itself disappears.

-
-
-
-
-
-

Optional mathematics — centre before measuring geometry

Let three two-dimensional embeddings be

$$\mathbf{h}_1 = (10, 1), \quad \mathbf{h}_2 = (10, 0), \quad \mathbf{h}_3 = (10, -1).$$

Compute the pairwise raw cosines. Then subtract the mean embedding and inspect the centred vectors.

Why do the raw cosines make the three examples look extremely similar? What structure becomes visible after centring? Finally compare with the genuinely collapsed case $\mathbf{h}_1 = \mathbf{h}_2 = \mathbf{h}_3 = (10, 0)$.

2020

Check your understanding

Two checkpoints both obtain 98.5% validation accuracy. At the later checkpoint, within-class scatter is 40% lower, class means are farther apart and classifier directions are more closely aligned with those means.

It is correct to say that the checkpoints have similar measured accuracy but different representation geometry. It is not automatically correct to say that the later checkpoint will be better on every distribution shift. Geometry is evidence about the representation, not a universal guarantee of robustness.

2022

2022

ACTION — create this figure

Create a simple superposition figure. Left: a privileged-axis representation with two features aligned to x and y axes. Right: three or four sparse feature directions arranged around the same 2-D plane, illustrating that more feature directions than dimensions can coexist at the cost of interference. Add a note that this is a toy linear picture, not a claim that real networks use only two dimensions.

Figure 11.3: Superposition motivates a direction-based view of representation. Useful features can occupy distributed directions rather than one-neuron-per-feature coordinates.

Key idea

A representation claim should name both the property and the measurement. “The embedding is good” is underspecified. Good for nearest-neighbour retrieval? Linearly separable classification? Causal control of the output? Robustness to a distribution shift? Different measurements answer different questions.

Application to the architecture

The StoryReasoning system provides a compact example of why scaling should be analysed component by component.

A scaled version used more story windows, additional GroundCap data for pretraining, frozen CLIP features and a wider autoencoder. Several quantities improved on the same 5,254 test windows: text retrieval top-10 rose from 41.5% to 51.6%; text cross-entropy fell from 2.65 to 2.40; character F1 increased from 0.44 to 0.46; and autoencoder reconstruction L1 improved from 0.042 to 0.036.

Yet the core next-frame image metrics barely moved: prior-mean L1 changed from 0.131 to 0.132, and best-of-5 sample L1 from 0.123 to 0.124. The extra data and stronger representations helped tasks that could exploit them, but they did not remove the ambiguity built into pointwise image prediction.

The representation diagnostics also exposed geometry that raw loss values hid. Image latents shared a substantial common component, so raw cosine similarity exaggerated similarity between unrelated targets. Centring the target embeddings made retrieval/similarity more discriminative. When a trainable encoder was allowed to move with the target under a cosine-style objective, it could drift toward this shared mean; a frozen target encoder prevented that easy collapse route.

The lesson is not that scaling “failed”. It is that the system had different bottlenecks in different heads. A useful scaling study says what resource changed, which component improved, and which limiting assumption remained untouched.

Metric	Main question	Strength	Important blind spot
Pixel L1	Are aligned pixels numerically close?	precise for registered reconstruction	can reward averages for multimodal targets
Centred CLIP cosine	Is this output semantically aligned with this target in CLIP space?	per-example semantic comparison	not a complete perceptual or fidelity measure
Feature Fréchet	Does the generated set resemble the target set in feature statistics?	distribution-level realism signal	ignores which sample belongs to which target
Laplacian variance	Does the image contain high-frequency structure?	simple blur and sharpness diagnostic	says little about semantics or correctness

Table 11.1: Different metrics operationalise different claims. Calling all of them “quality” hides the distinction between alignment, distribution-level realism and image sharpness.

Optional design exercise — validate before optimising

Suppose you want to add a new image metric M to a training loss. Before training, construct at least four deliberately different candidates: the true target, a blurred target, a real but mismatched related image, and a global average/-median image. Write down the ranking you believe M should produce for the property you claim it measures. Then ask: if the observed ranking disagrees, is the metric wrong, or was the claimed property underspecified? What additional metric would separate the ambiguous cases?

1.

2.

3.

4.

5.

6.

7.

•

•

•

•

•

•

•

•

Stage	Central change in the mental model	Question to keep asking
Weeks 1–2	examples become an explicit model, loss and optimisation problem	What task have we specified, and what behaviour does the loss reward?
Weeks 3–4	networks learn intermediate representations and can fail in optimisation/generalisation	What structure is being learned rather than merely fitted?
Week 5	architecture imposes inductive bias on visual structure	What computation does the architecture make easy?
Week 6	representation and explanation require evidence	What information is present, and what information is causally used?
Week 7	sequence tasks require memory	What must survive through time?
Week 8	ambiguous targets require distributions, not only point estimates	Is there one correct future or several plausible ones?
Week 9	attention makes information flow conditional	Which information should this computation retrieve now?
Week 10	scale and pretraining organise distributed geometry	What resource is limiting, what structure is hidden in the representation, and does our metric see it?

Table 11.2: The compulsory module is one connected progression rather than a catalogue of model names. Each new mechanism adds a new diagnostic question while retaining the earlier ones.

Key idea

The most transferable skill in deep learning is not recognising an architecture diagram. It is being able to look at a learning system and ask: what information enters, what computation is possible, what objective supplies the learning signal, what representation emerges, what evidence shows that the intended signal is used, and what alternative explanation has been ruled out?

Key idea

“Learning during inference” is not one mechanism. To reason clearly, ask what changes: the input context, transient activations, an external memory, persistent parameters or state, or simply the amount of computation performed before answering.

2017

Mechanism	What changes, and what can fail?
In-context learning	The prompt/context and transient activations change; the parameters do not. Demonstrations can be ignored, misread or contradictory.
External retrieval	The retrieved evidence changes while model weights remain fixed; relevant items can be absent, missed or poorly used.
Test-time adaptation	Selected parameters, normalisation state or adapter state change; a proxy objective can adapt in the wrong direction.
Search/reranking	The number and ordering of candidate computations change; the correct candidate may never be proposed or may be misranked.

Table 12.1: Several mechanisms can make inference adaptive, but the location and persistence of the change are different.

- 1.
- 2.
- 3.
- 4.

Check your understanding

A frozen transformer gives label B for a query. You change one demonstration in the prompt and it now gives label D. The parameter tensors are bit-for-bit identical before and after both forward passes.

What changed? The model's learned procedure did not change. The temporary information presented to that procedure changed, producing different activations and therefore a different output.

- 1.
- 2.
- 3.

ACTION — create this figure

Create a small sequence diagram for an episodic key-label task. Show three demonstration pairs followed by a repeated query key. Draw an attention-style arrow from the query back to the matching earlier key and a second arrow carrying the neighbouring label information toward the output. Beside it, show a causal intervention where only that earlier label is changed and the predicted label changes, while model weights remain fixed.

Figure 12.1: An induction-style retrieve-and-copy hypothesis becomes more convincing when an intervention on the retrieved information changes the output, not merely when an attention map looks suggestive.

2021

-
-
-
-

ACTION — create this figure

Draw a retrieval-augmented pipeline with a user query entering a query encoder, a vector search over an editable external memory, top-k retrieved items entering a downstream reader/generator, and a final answer. Mark model parameters and external memory as separate stores. Show an edit to one memory document changing a later answer without changing the model weights.

Figure 12.2: External retrieval separates learned search/representation machinery from an editable store of task information. This improves updateability but introduces retrieval as a new failure point.

PyTorch: a tiny test-time adapter update

```
# base_model is frozen; target_x is unlabelled
for p in base_model.parameters():
    p.requires_grad_(False)

A = nn.Parameter(torch.eye(d))
b = nn.Parameter(torch.zeros(d))
opt = torch.optim.Adam([A, b], lr=1e-2)

for _ in range(adapt_steps):
    x_adapt = target_x @ A.T + b
    loss = moment_matching_loss(x_adapt, source_stats)
    opt.zero_grad()
    loss.backward()
    opt.step()
```

-
-
-
-
-
-

Check your understanding

A test-time adapter reduces its unsupervised entropy/moment-matching objective on every deployment batch. On a labelled audit set, task accuracy first improves and then falls as adaptation continues.

Has optimisation failed? Not necessarily. The optimiser is succeeding on the proxy objective. The scientific problem is that further optimisation no longer aligns with the external task metric.

-
-
-
-
-
-

- 1.
- 2.
- 3.

-
-

ACTION — create this figure

Create a quality-versus-compute diagram for two-stage inference. Show a large candidate memory, cheap Stage-1 scoring over all candidates, top-M selection, expensive Stage-2 reranking over only M, and final output. Add a small graph where increasing M raises Stage-1 recall but also increases second-stage compute, eventually approaching full search.

Figure 12.3: Test-time computation can be allocated selectively. A smaller candidate budget saves work but may discard the correct answer before expensive verification can help.

Application to the architecture

StoryReasoning has repeatedly exposed a problem with direct next-frame generation: after a camera cut, the exact target pixels are not determined by the four inputs. A pixel loss can therefore reward an average-looking output even when a related real frame is more plausible.

One alternative is to change the inference problem. Instead of generating pixels, treat real candidate frames as an external memory. Use frozen representations of the four context frames/descriptions to build a query, retrieve likely fifth-frame candidates, then spend a second signal on a smaller set to rerank them.

The architecture narrative reports a stronger retrieval formulation in which the true next frame was selected from 2,974 candidates about 7% of the time and appeared in the top 10 about 40% of the time. The corresponding chance rates are roughly 0.034% top-1 and 0.34% top-10. Those numbers do not make retrieval equivalent to generation, but they show that the context contains information useful for ranking real future frames far above chance.

This formulation also makes the inference-time budget explicit. A cheap text or image score can shortlist candidates; a more expensive multimodal score can rerank only the shortlist. No full assessment model needs to be retrained when the candidate memory changes.

The trade-off is conceptual as well as computational. Retrieval guarantees that the returned image is an existing real frame, but it cannot invent a genuinely novel continuation that is absent from memory. Generation can create new outputs, but must model plausibility and uncertainty rather than simply rank stored examples.

Location	Typical lifetime	Diagnostic question
Parameters	long-lived	Did any tensor or persistent state change during inference?
Context/activations	one request or context window	Does changing demonstrations or instructions alter behaviour with identical weights?
External memory	separately persistent	Was the needed item present, retrieved and passed downstream?
Search budget	one inference procedure	Does quality change with samples, retrieval depth, reranking depth or verification steps?

Table 12.2: The phrase “the system learned” becomes more useful when replaced by a statement about what information store or computation actually changed.

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.

Optional mathematics — a simple retrieval ceiling

A two-stage system retrieves M items, then uses a perfect reranker that always places the true item first whenever it survives Stage 1. Let Stage-1 recall@ M be r_M .

1. What is the maximum possible final top-1 accuracy?
2. If $r_{10} = 0.62$ and $r_{50} = 0.86$, what final top-1 gains are theoretically available from improving only the reranker?
3. If the observed final top-1 at $M = 50$ is 0.30, which stage still leaves more headroom: retrieval or reranking?

The exercise illustrates a general systems principle: downstream computation cannot recover information discarded upstream.

-
-
-
-

Key idea

The final diagnostic question of the module is therefore not merely “did the model learn?” It is: *what changed, where is the information stored, how long does it persist, what computation uses it, and what evidence shows that this mechanism improved the task we actually care about?*

